

RĪGAS VALSTS TEHNIKUMS

DATORIKAS NODAĻA

Izglītības programma: Programmēšana

Paskaidrojošais raksts 119. lpp.

KVALIFIKĀCIJAS DARBS

“Futbola komandu vadības sistēma - TeamFlow”

Audzēknis:

Vladimirs Avgustins

Prakses vadītājs:

Rēbuks Gundars

Nodaļas vadītājs:

Normunds Barbāns

Rīga 2026

ANOTĀCIJA

Kvalifikācijas darba tēma ir tīmekļa lietojumprogramma “Futbola komandu vadības sistēma - TeamFlow”. Sistēma ir izstrādāta futbola komandas ikdienas darba organizēšanai vienotā vidē: lietotāji var reģistrēties un autorizēties, treneris var izveidot komandu, spēlētājs var pievienoties komandai ar komandas kodu, tiek uzturēts notikumu grafiks, apmeklētības uzskaite, spēļu sastāva plānošana, spēlētāju statistika, grupas tērēšana, privātās ziņas, profila attēli, komandas logotips, kontaktforma un administratora pārvaldības funkcijas. Frontend daļas izstrādē izmantots Vue.js ar Vite ātrai lietotāja saskarnes izstrādei, Pinia lietotnes stāvokļa pārvaldībai, Vue Router lapu maršrutēšanai, Vue I18n latviešu un angļu valodas atbalstam, Axios saziņai ar servera API, Chart.js statistikas vizualizācijai un Day.js datumu apstrādei. Backend daļā izmantots Node.js un Express REST API izveidei, SQLite datu glabāšanai, Socket.IO reāllaika saziņai, JWT lietotāju autentifikācijai, bcrypt parolu šifrēšanai, Nodemailer e-pasta nosūtīšanai paroles atjaunošanas un kontaktformas vajadzībām, kā arī dotenv konfigurācijas datu glabāšanai vides mainīgajos.

Paskaidrojošajā rakstā ir iekļauts ievads, uzdevuma nostādne, prasību specifikācija, uzdevuma risināšanas līdzekļu izvēles pamatojums, programmatūras produkta modelēšana un projektēšana, datu struktūru apraksts, lietotāja ceļvedis, programmatūras lietojamības testēšana, nobeigums, informācijas avoti un pielikumi. Dokumentā aprakstīta projekta aktualitāte, mērķauditorija, lietotāju lomas, funkcionālās un nefunkcionālās prasības, sistēmas arhitektūra, datu plūsmas, datubāzes tabulas un to savstarpējās saites, API grupas, drošības mehānismi, sistēmas instalācija un galvenie lietošanas scenāriji.

Kvalifikācijas darbs sastāv no 119 lappusēm, 53 attēliem, 22 tabulas un 1 pielikuma. Pielikums satur programmas pirmkodu fragmentus.

ANNOTATION

The topic of the qualification work is the web application “Football team management system - TeamFlow”. The system was developed to organize the daily workflow of a football team in one environment: users can register and authenticate, a coach can create a team, a player can join a team using a team code, and the system maintains an event schedule, attendance tracking, match squad planning, player statistics, team chat, direct messages, profile images, team logo, contact form and administrator management functions. The frontend part was developed using Vue.js with Vite for fast user interface development, Pinia for application state management, Vue Router for page routing, Vue I18n for Latvian and English language support, Axios for communication with the server API, Chart.js for statistics visualization and Day.js for date processing. The backend part uses Node.js and Express for REST API development, SQLite for data storage, Socket.IO for real-time communication, JWT for user authentication, bcrypt for password hashing, Nodemailer for sending password reset and contact form e-mails, and dotenv for storing configuration data in environment variables.

The explanatory report contains the introduction, problem statement, requirements specification, justification of selected development tools, software modelling and design, data structure description, user guide, usability testing, conclusion, references and appendices. The document describes the relevance of the project, target audience, user roles, functional and non-functional requirements, system architecture, data flows, database tables and their relationships, API groups, security mechanisms, system installation and main usage scenarios.

The qualification work consists of 119 pages, 53 figures, 22 tables and 1 appendice. The appendice contains code fragments.

SATURS

IEVADS.....	5
1. UZDEVUMA NOSTĀDNE	6
2. PRASĪBU SPECIFIKĀCIJA	8
2.1. Ieejas un izejas informācijas apraksts	8
2.1.1. Ieejas informācijas apraksts.....	8
2.1.2. Izejas informācijas apraksts.....	9
2.1.3. Ārējās informācijas apraksts.....	9
2.2. Funkcionālās prasības	9
2.3. Nefunkcionālās prasības.....	15
3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU IZVĒLES PAMATOJUMS	17
4. PROGRAMMATŪRAS PRODUKTA MODELĒŠANA UN PROJEKTĒŠANA	19
4.1. Sistēmas struktūras modelis	19
4.1.1. Sistēmas arhitektūra	19
4.1.2. Sistēmas ER modelis.....	21
4.2. Funkcionālais sistēmas modelis	24
4.2.1. Datu plūsmu modelis.....	24
5. DATU STRUKTŪRU APRAKSTS	31
6. LIETOTĀJA CEĻVEDIS	37
6.1. Sistēmas prasības aparatūrai un programmatūrai	37
6.2. Sistēmas instalācija un palaišana.....	37
6.3. Programmas apraksts.....	40
7. PROGRAMMATŪRAS LIETOJAMĪBAS TESTĒŠANA	69
NOBEIGUMS.....	71
INFORMĀCIJAS AVOTI.....	72
PIELIKUMI.....	73
1. pielikums.....	73

IEVADS

Futbola komandas ikdienas darbā, informācijas apmaiņa un datu uzskaitē ir tikpat svarīga kā pats treniņu process. Trenerim regulāri jāplāno treniņi, spēles un komandas sapulces, jāseko spēlētāju apmeklētībai, jāanalizē sportiskie rādītāji, jāuztur aktuāla informācija par komandas sastāvu un jānodrošina operatīva saziņa ar spēlētājiem. Ja šie procesi tiek veikti atsevišķos rīkos, piemēram, izklājlappās, ziņapmaiņas lietotnēs un e-pastos, informācija kļūst sadrumstalota, dublējas un ir grūti pārskatāma.

Īpaši amatieru un jauniešu futbola komandām bieži nav pieejams centralizēts risinājums, kas apvienotu visus nepieciešamos procesus vienā sistēmā. Esošās starptautiskās sporta pārvaldības platformas parasti ir paredzētas lielām organizācijām vai pieejamas par maksu, savukārt bezmaksas risinājumi nodrošina tikai daļēju funkcionalitāti. Piemēram, izklājlappas ļauj uzglabāt statistiku, bet nenodrošina piekļuves kontroli, savukārt ziņapmaiņas lietotnes nodrošina saziņu, bet neveido strukturētu datu modeli.

Šī kvalifikācijas darba ietvaros izstrādātā sistēma “TeamFlow” ir paredzēta tieši futbola komandu darba organizēšanai vienotā tīmekļa vidē. Sistēma nodrošina iespēju izveidot komandu, piesaistīt spēlētājus ar komandas kodu, apstrādāt asistenta trenera pievienošanās pieprasījumus, uzturēt grafiku ar notikumu tipiem treniņš, spēle, sapulce un cits, atzīmēt apmeklētību, pārvaldīt spēlētāju statistiku un nodrošināt saziņu reāllaikā.

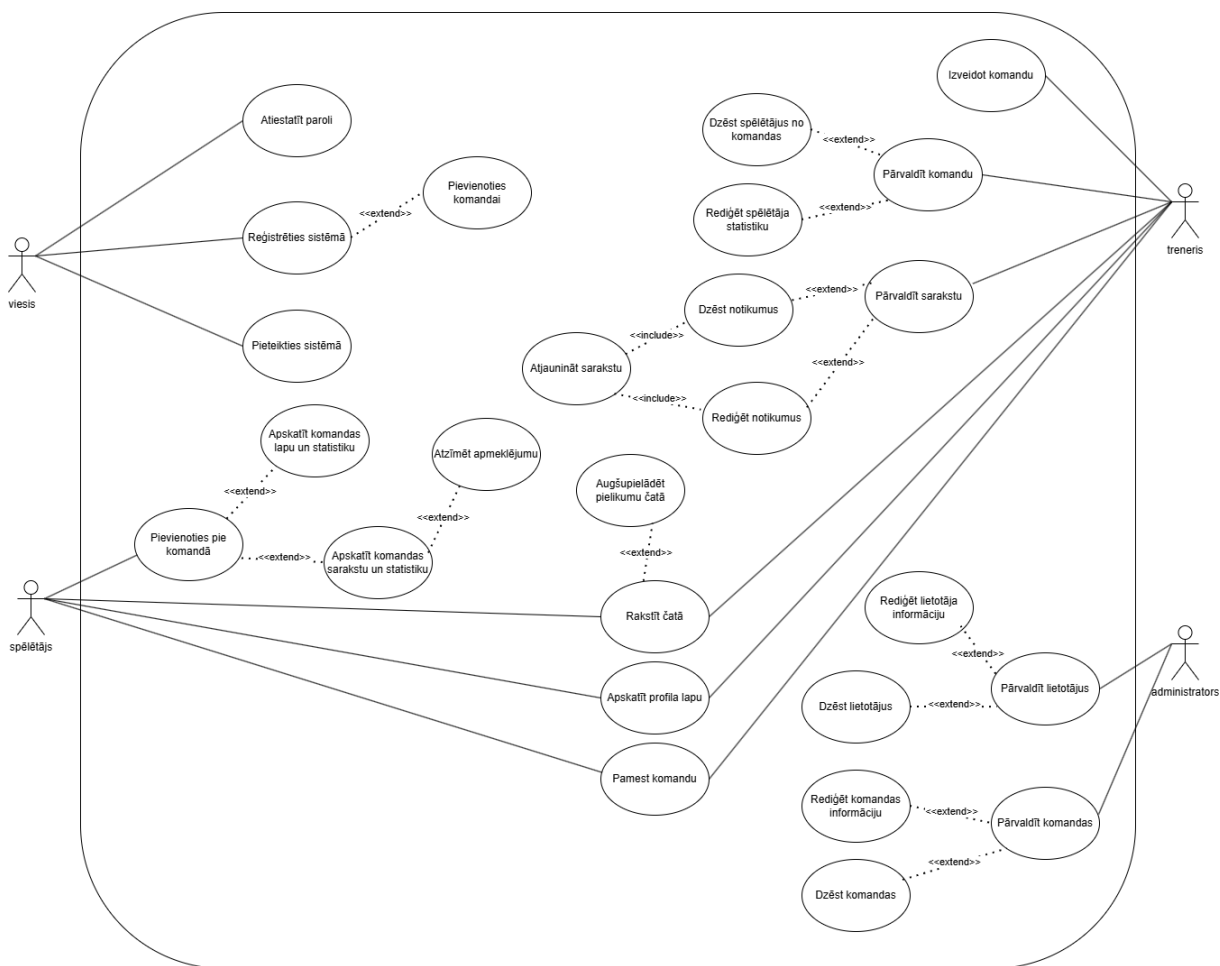
Sistēmas mērķauditorija ir futbola komandu treneri, spēlētāji. Trenerim sistēma nodrošina rīkus komandas darba organizēšanai un pārvaldībai, spēlētājam - ērtu piekļuvi savas komandas informācijai, grafikam un saziņai, savukārt administratoram - iespēju pārvaldīt lietotāju un komandu pamatdatus. Neautorizētam lietotājam (viesim) sistēma piedāvā iespēju apskatīt publiskās sadaļas, reģistrēties, pieslēgties vai pieprasīt paroles atjaunošanu.

Projekta aktualitāti nosaka nepieciešamība pēc praktiska, viegli uzturama un konkrēti futbola komandām pielāgota risinājuma, kas nav pārslogots ar lielu sporta organizāciju funkcionalitāti, bet vienlaikus nodrošina visus būtiskos procesus vienotā datu vidē. Sistēma ir izstrādāta kā klienta-servera tīmekļa lietojumprogramma, kur frontend daļa darbojas pārlūkprogrammā, bet backend daļa apstrādā API pieprasījumus un nodrošina datu glabāšanu SQLite datubāzē.

1. UZDEVUMA NOSTĀDNE

Kvalifikācijas darba uzdevums ir izstrādāt tīmekļa lietojumprogrammu futbola komandas pārvaldībai, kurā dažādu lomu lietotāji var centralizēti strādāt ar komandai svarīgu informāciju: lietotāju kontiem, komandas sastāvu, grafiku, apmeklētību, spēlētāju statistiku un saziņu.

Sistēmas galvenā funkcionalitāte un lietotāju mijiedarbība ar to ir attēlota lietojumgadījumu diagrammā (skat. 1. att.).



1. att. Lietojumgadījumu diagramma.

Sistēmai jānodrošina šādas galvenās funkcionalitātes:

- lietotāju reģistrācija ar “spēlētājs” vai “treneris” lomu, autorizācija ar e-pastu un paroli, kā arī paroles atjaunošana ar e-pasta tokenu,
- komandas izveide trenerim, spēlētāja pievienošanās komandai ar unikālu komandas kodu un trenera pieteikums asistenta lomai,
- galvenā trenera darbības ar asistenta trenera pieprasījumiem, spēlētāju pārvaldību un spēlētāju izņemšanu no komandas,
- komandas grafika uzturēšana nedēļas un saraksta skatā, notikumu tipu “treniņš”, “spēle”, “sapulce” un “cits” izmantošana, kā arī konfliktu pārbaude pēc datuma un laika,
- Apmeklētības uzskaitē ar statusiem “klāt”, “prombūtnē”,
- spēlētāju statistikas pārvaldība un eksports CSV/XLS formātā,
- komandas tērēšana, privātās ziņas, ziņu pielikumu augšupielāde un neizlasīto ziņu apstrāde
- administratoram paredzēta lietotāju un komandu pamatdatu apskate, rediģēšana un dzēšana.

Sistēmā tiek izmantotas šādas lietotāju lomas:

Viesis ir lietotājs, kurš nav autorizējies sistēmā. Viņam ir pieejamas publiskās lapas, kā arī iespēja reģistrēties, pieslēgties sistēmai, pieprasīt paroles atjaunošanu un izmantot kontaktformu.

Spēlētājs ir autorizēts lietotājs ar lomu “spēlētājs”. Viņš var pievienoties komandai, izmantojot komandas kodu, apskatīt komandas grafiku, savu statistiku un komandas informāciju, kā arī atzīmēt savu apmeklētības statusu treniņiem un spēlēm. Papildus spēlētājs var izmantot komandas tērēšanu, privātās ziņas un pārvaldīt savu profila attēlu.

Treneris ir autorizēts lietotājs ar lomu “treneris”. Trenerim ir iespēja izveidot komandu un pārvaldīt tās darbību. Tas ietver komandas grafika plānošanu, spēlētāju statistikas rediģēšanu, spēlētāju apmeklētības datu apskati, spēlētāju profila attēlu un komandas logotipa uzturēšanu, kā arī saziņas nodrošināšanu komandas ietvaros.

Galvenais treneris nav atsevišķa sistēmas loma, bet gan treneris, kurš ir konkrētās komandas īpašnieks. Tas tiek noteikts pēc komandas “coach_id” vērtības. Papildus trenera funkcijām galvenais treneris var apstiprināt vai noraidīt asistenta trenera pieprasījumus, kā arī izņemt spēlētājus no komandas.

Administrators ir autorizēts lietotājs ar administratora lomu. Administrators var apskatīt visu lietotāju sarakstu sistēmā, rediģēt lietotāju datus, tostarp lomu un komandas piesaisti, kā arī dzēst lietotāju kontus. Tāpat administrators var apskatīt un rediģēt komandu pamatinformāciju, taču jaunu komandu izveide administratoram nav paredzēta.

2. PRASĪBU SPECIFIKĀCIJA

2.1. Ieejas un izejas informācijas apraksts

2.1.1. Ieejas informācijas apraksts

Sistēmā tiks nodrošināta šādas ieejas informācijas, ko lietotāji ievadīs no tastatūras vai caur formām, apstrāde.

1. Informācija par lietotājiem :
 - Vārds - burtu teksts līdz 20 rakstzīmēm (piem., "Jānis ").
 - Uzvārds - burtu teksts līdz 20 rakstzīmēm (piem., "Kalniņš").
 - Parole - teksts no 8 līdz 30 rakstzīmēm, var saturēt burtus, ciparus un speciālās rakstzīmes, tiek saglabāta hash veidā (piem., "Sport123!").
 - E-pasts - teksts līdz 40 rakstzīmēm (piem., "janis.kalnins@gmail.com").
 - Loma - izvēle no saraksta (Treneris, Spēlētājs).
2. Informācija par komandām:
 - Nosaukums - burtu teksts līdz 40 rakstzīmēm (piem., "Rīga FC").
 - Komandas kods - unikāls teksts (piem., "RTG2025").
 - Trenera ID - ārējā atslēga uz lietotāju ar trenera lomu.
 - Logo - grafiskais fails.
3. Informācija par spēlētājiem
 - Spēlētāja ID - ārējā atslēga uz lietotāju ar spēlētāja lomu.
 - Statistika - aizvadītās spēles, gūtie vārti, piespēles, kartītes.
 - Foto - attēla fails.
4. Informācija par treniņiem un spēlēm:
 - Nosaukums - teksts līdz 50 rakstzīmēm (piem., "Skonto halle").
 - Datums - datuma formāts (piem., "2025-03-14").
 - Laiks - laika formāts (piem., "18:00").
 - Norises vieta - teksts līdz 50 rakstzīmēm (piem., "RTU stadions").
 - Tips - izvēle no saraksta (treniņš, spēle, sapulce, cits).
5. Informācija par apmeklētību:
 - Spēlētāja ID - ārējā atslēga uz lietotāju ar lomu "Player".
 - Notikuma ID - ārējā atslēga uz konkrētu treniņu vai spēli.
 - Apmeklējuma statuss - izvēle no saraksta (klāt vai prombūtnē).
 - Piezīmes - brīvs teksts papildu informācijai par apmeklējumu.
 - Atzīmēšanas datums un laiks - automātiski ģenerēts laika zīmogs.
6. Informācija par spēļu sastāvu:
 - Spēles ID - ārējā atslēga uz grafika notikumu ar tipu "spēle".
 - Spēlētāja ID - ārējā atslēga uz lietotāju ar lomu "player".
 - Dalības statuss - izvēle no saraksta (apstiprināts, noraidīts).
 - Piezīmes - brīvs teksts papildu informācijai.
 - Atzīmēšanas datums un laiks - automātiski ģenerēts laika zīmogs.
7. Informācija par čata ziņojumiem:
 - Sūtītāja ID - ārējā atslēga uz lietotāju, kurš nosūta ziņojumu.
 - Saņēmēja ID - ārējā atslēga uz lietotāju privātās ziņas gadījumā.
 - Čata istabas ID - ārējā atslēga uz komandas čatu grupas ziņas gadījumā.
 - Ziņojuma teksts - brīvs teksts ziņas saturam.
 - Pielikums - atbalstīts fails, piemēram, attēls vai dokuments.

- Nosūtīšanas datums un laiks - automātiski ģenerēts laika zīmogs.
- Ziņojuma statuss - norāde, vai privātā ziņa ir izlasīta.

2.1.2. Izejas informācijas apraksts

Sistēmas datu apstrādes rezultātā lietotāji varēs iegūt šādu izejas informāciju:

- Autentifikācijas rezultāts - veiksmīgas pieslēgšanās gadījumā lietotājs saņem JWT tokens un profila pamatinformāciju (ekrāns).
- Komandas panelis - tiek attēlots komandas nosaukums, kods, logotips, sastāvs un tuvākie notikumi (ekrāna skats).
- Grafika pārskats - komandas notikumi sakārtoti nedēļas vai saraksta skatā ar filtrēšanu pēc tipa un datuma (ekrāna skats).
- Apmeklētības kopsavilkums - spēlētāju statusi konkrētiem treniņu notikumiem un procentuālie rādītāji (ekrāna tabula).
- Spēļu sastāva pārskats - trenerim izveidotais spēlētāju saraksts ar katra dalībnieka apstiprinājuma statusu (ekrāna tabula).
- Statistikas pārskats - kopējie vārti, piespēles, kartītes, līderu saraksti un diagrammas (ekrāna skats).
- Statistikas eksports - spēlētāju statistikas dati ārējai apstrādei (CSV un XLS faili).
- Čata vēsture - grupas un privāto ziņu sarakste ar pielikumiem un laika atzīmēm (ekrāna skats).
- Administratīvais pārskats - lietotāju saraksts ar lomām un komandas piesaisti, kā arī komandu pamatdati (ekrāna tabula).
- Kļūdu un veiksmes paziņojumi - validācijas, autorizācijas, saglabāšanas un piekļuves kontroles rezultāti (ekrāna paziņojumi).

2.1.3. Ārējās informācijas apraksts

Sistēma neizmanto autorizāciju ar ārējiem sociālajiem tīkliem, internetbanku vai valsts identifikācijas risinājumiem. Ārējā saziņa tiek izmantota e-pasta nosūtīšanai paroles atjaunošanas procesā un kontaktformas darbībā. Backend serveris izmanto Nodemailer un SMTP konfigurāciju, kas tiek glabāta .env failā. Lietotāja ievadītais e-pasts tiek izmantots atjaunošanas saites nosūtīšanai, bet paroles atjaunošanas tokens netiek glabāts atklātā veidā - datubāzē saglabā tā hash vērtību.

2.2. Funkcionālās prasības

1. Jauna lietotāja konta reģistrācija

Identifikators	PS 2.2.1
Ievads	
Funkcija paredzēta, lai jauns lietotājs varētu izveidot kontu sistēmā ar spēlētāja vai trenera lomu un pēc tam izmantot sistēmas funkcijas atbilstoši savai lomai.	
Ievaddati	
1. Vārds - obligāts lauks.	

2. Uzvārds - obligāts lauks. 3. E-pasts - obligāts un unikāls lauks. 4. Parole - obligāts lauks, vismaz 8 rakstzīmes. 5. Loma - spēlētājs vai treneris. 6. Komandas kods - neobligāts lauks spēlētājam, ja viņš vēlas pievienoties komandai reģistrācijas laikā.
Apstrāde
1. Sistēma pārbauda, vai visi obligātie lauki ir aizpildīti. 2. Sistēma pārbauda, vai e-pasts vēl nav reģistrēts datubāzē. 3. Sistēma pārbauda ievadīto datu formātu. 4. Parole tiek šifrēta, izmantojot bcrypt. 5. Lietotāja dati tiek saglabāti datubāzē. 6. Ja spēlētājs norāda derīgu komandas kodu, sistēma piesaista spēlētāju attiecīgajai komandai. 7. Spēlētājam tiek izveidots sākotnējais statistikas ieraksts.
Izvaddati
1. Veiksmīgas reģistrācijas paziņojums. 2. Izveidotā lietotāja konta dati.
Kļūdas paziņojumi
1. Nav aizpildīti obligātie lauki. 2. Lietotājs ar šādu e-pastu jau pastāv. 3. Ievadīto datu formāts nav derīgs. 4. Norādītais komandas kods nav atrasts.
Izsaucamās prasības
Funkciju var izmantot neautorizēts lietotājs. Publiskajā reģistrācijā var izveidot spēlētāja vai trenera kontu.

2. Komandas izveide un pievienošanās komandai

Identifikators	PS 2.2.2
Ievads	
Funkcija paredzēta komandas izveidei un lietotāju piesaistei komandai. Treneris var izveidot jaunu komandu, bet spēlētājs var pievienoties komandai, izmantojot komandas kodu.	
Ievaddati	
1. Komandas nosaukums - obligāts lauks. 2. Komandas kods - automātiski ģenerēts sistēmā. 3. Pievienošanās kods - ievada spēlētājs vai treneris.	
Apstrāde	
1. Sistēma pārbauda, vai komandas nosaukums ir aizpildīts. 2. Sistēma ģenerē unikālu komandas kodu. 3. Komanda tiek saglabāta datubāzē. 4. Treneris tiek piesaistīts komandai kā galvenais treneris. 5. Sistēma izveido komandas čata telpu. 6. Ja lietotājs ievada komandas kodu, sistēma pārbauda tā derīgumu. 7. Ja kods ir derīgs, lietotājs tiek piesaistīts komandai. 8. Ja treneris piesakās citai komandai, tiek izveidots pieprasījums.	
Izvaddati	
1. Izveidotās komandas dati. 2. Komandas kods. 3. Paziņojums par veiksmīgu pievienošanos komandai.	
Kļūdas paziņojumi	
1. Komandas nosaukums nav aizpildīts.	

2. Komandas kods nav atrasts.
3. Lietotājs jau ir piesaistīts komandai.
Izsaucamās prasības
Komandu var izveidot tikai treneris. Spēlētājs var pievienoties komandai ar kodu. Administrators komandas neveido.

3. Komandas grafika pārvaldīšana

Identifikators	PS 2.2.3
Ievads	
Funkcija paredzēta komandas treniņu un spēļu plānošanai un attēlošanai nedēļas vai saraksta skatā.	
Ievaddati	
1. Notikuma nosaukums - obligāts lauks. 2. Datums - obligāts lauks. 3. Laiks - obligāts lauks. 4. Tips - treniņš, spēle, sapulce vai cits. 5. Norises vieta - neobligāts lauks.	
Apstrāde	
1. Sistēma pārbauda ievaddatus. 2. Sistēma pārbauda datuma un laika formātu. 3. Sistēma pārbauda, vai nav konflikta ar citu notikumu. 4. Ja konflikta nav, notikums tiek saglabāts datubāzē. 5. Notikumi tiek attēloti nedēļas vai saraksta skatā. 6. Lietotājs var filtrēt notikumus pēc datuma un tipa.	
Izvaddati	
1. Atjaunināts grafiks. 2. Paziņojums par veiksmīgu darbību.	
Kļūdas paziņojumi	
1. Nav aizpildīti obligātie lauki. 2. Nepareizs datuma vai laika formāts. 3. Konflikts ar citu notikumu.	
Izsaucamās prasības	
Grafiku var pārvaldīt treneris. Spēlētājs var tikai skatīt grafiku.	

4. Komandas saziņa un privātās ziņas

Identifikators	PS 2.2.4
Ievads	
Funkcija paredzēta lietotāju saziņai komandas ietvaros, izmantojot grupas tērzēšanu un privātās ziņas.	
Ievaddati	
1. Ziņojuma teksts. 2. Sūtītāja identifikators. 3. Saņēmēja identifikators vai čata telpas identifikators. 4. Pielikums - neobligāts fails.	
Apstrāde	
1. Sistēma pārbauda lietotāja autorizāciju. 2. Sistēma pārbauda piekļuves tiesības. 3. Ziņojums tiek saglabāts datubāzē. 4. Pielikums tiek saglabāts mapē "uploads". 5. Ziņa tiek nosūtīta reāllaikā.	

6. Privātajām ziņām tiek saglabāts izlasīšanas statuss.
Izvaddati
1. Jauns ziņojums tērēšanā. 2. Privātā saruna. 3. Pielikuma saite.
Kļūdas paziņojumi
1. Tukšs ziņojums. 2. Lietotājs nav autorizēts. 3. Nav piekļuves tiesību. 4. Neizdevās augšupielādēt failu.
Izsaucamās prasības
Funkcija pieejama tikai autorizētiem lietotājiem savas komandas ietvaros.

5. Autorizācija:

- 5.1. Sistēmai jānodrošina lietotāja pieslēgšanās, ievadot e-pastu un paroli.
- 5.2. Sistēmai jāpārbauda, vai lietotājs ar norādīto e-pastu eksistē datubāzē.
- 5.3. Sistēmai jāpārbauda, vai ievadītā parole atbilst datubāzē saglabātajai šifrētajai parolei.
- 5.4. Pēc veiksmīgas pieslēgšanās sistēmai jāizveido autorizācijas tokens.
- 5.5. Lietotājam jābūt iespējai izrakstīties no sistēmas.
- 5.6. Pēc izrakstīšanās lietotāja autorizācijas dati jādzēš no pārlūka lokālās glabātuves.
- 5.7. Ja pieslēgšanās dati nav pareizi, sistēmai jāizvada kļūdas paziņojums.

6. Paroles atjaunošana:

- 6.1. Lietotājam jābūt iespējai pieprasīt paroles atjaunošanu, ievadot savu e-pastu.
- 6.2. Sistēmai jāpārbauda, vai konts ar norādīto e-pastu eksistē datubāzē.
- 6.3. Sistēmai jāizveido unikāls paroles atjaunošanas tokens.
- 6.4. Sistēmai jānosūta paroles atjaunošanas saite uz lietotāja e-pastu.
- 6.5. Paroles atjaunošanas saitei jābūt derīgai ierobežotu laiku.
- 6.6. Lietotājam jābūt iespējai ievadīt jaunu paroli un apstiprināt to.
- 6.7. Ja paroles nesakrīt, jaunā parole ir īsāka par 8 rakstzīmēm vai tokens nav derīgs, sistēmai jāizvada kļūdas paziņojums.
- 6.8. Pēc veiksmīgas paroles nomaiņas izmantotais paroles atjaunošanas tokens vairs nedrīkst būt derīgs.

7. Lietotāja profila pārvaldīšana:

- 7.1. Lietotājam jābūt iespējai apskatīt savu profila informāciju.
- 7.2. Profilā jāattēlo lietotāja vārds, uzvārds, loma, e-pasts un komandas informācija.
- 7.3. Lietotājam jābūt iespējai parādīt vai paslēpt savu e-pastu.
- 7.4. Lietotājam jābūt iespējai parādīt vai paslēpt komandas kodu.
- 7.5. Lietotājam jābūt iespējai augšupielādēt profila attēlu.
- 7.6. Lietotājam jābūt iespējai dzēst savu profila attēlu.
- 7.7. Ja augšupielādētais fails nav attēls vai pārsniedz atļauto izmēru, sistēmai jāizvada kļūdas paziņojums.
- 7.8. Lietotājam jābūt iespējai pieprasīt paroles maiņu no profila sadaļas.
- 7.9. Lietotājam jābūt iespējai pamest pašreizējo komandu.
- 7.10. Pirms komandas pamešanas sistēmai jāizvada brīdinājums ar iespēju darbību atcelt.

8. Komandas pārskata attēlošana:

- 8.1. Lietotājam jābūt iespējai apskatīt savas komandas pārskata lapu.
- 8.2. Komandas pārskatā jāattēlo komandas nosaukums, komandas kods, logotips un galvenie rādītāji.

8.3. Sistēmai jāattēlo spēlētāju skaits, tuvākie notikumi un komandas statistikas kopsavilkums.

8.4. Sistēmai jāattēlo labākie spēlētāji pēc vārtiem, piespēlēm un spēļu skaita.

8.5. Trenerim jābūt iespējai augšupielādēt komandas logotipu.

8.6. Trenerim jābūt iespējai dzēst komandas logotipu.

8.7. Ja logotipa fails neatbilst prasībām, sistēmai jāizvada kļūdas paziņojums.

9. Spēlētāju saraksta pārvaldīšana:

9.1. Komandas dalībniekiem jābūt iespējai apskatīt komandas spēlētāju sarakstu.

9.2. Spēlētāju sarakstā jāattēlo spēlētāja vārds, uzvārds, e-pasts, profila attēls un statistika.

9.3. Sistēmai jānodrošina spēlētāju attēlošana kartīšu un tabulas skatā.

9.4. Sistēmai jānodrošina spēlētāju meklēšana pēc vārda, uzvārda vai e-pasta.

9.5. Sistēmai jānodrošina spēlētāju kārtošana pēc vārda, vārtiem, piespēlēm un spēļu skaita.

9.6. Trenerim jābūt iespējai augšupielādēt vai dzēst spēlētāja profila attēlu.

9.7. Galvenajam trenerim jābūt iespējai noņemt spēlētāju no komandas.

9.8. Pirms spēlētāja noņemšanas sistēmai jāizvada brīdinājums.

10. Spēlētāju statistikas pārvaldīšana:

10.1. Trenerim jābūt iespējai labot spēlētāju statistikas datus.

10.2. Sistēmai jāļauj ievadīt spēļu skaitu, vārtus, piespēles, dzeltenās kartītes un sarkanās kartītes.

10.3. Statistikas vērtībām jābūt skaitliskām un tās nedrīkst būt negatīvas.

10.4. Ja ievadītie dati nav derīgi, sistēmai nav jāļauj saglabāt izmaiņas.

10.5. Sistēmai jāaprēķina komandas kopējie statistikas rādītāji.

10.6. Sistēmai jāattēlo labākie vārtu guvēji un piespēļu autori.

10.7. Trenerim jābūt iespējai apskatīt statistiku tabulas un diagrammu veidā.

10.8. Sistēmai jānodrošina statistikas eksportēšana CSV formātā.

10.9. Sistēmai jānodrošina statistikas eksportēšana Excel formātā.

11. Apmeklējuma pārvaldīšana:

11.1. Spēlētājam jābūt iespējai atzīmēt savu dalību treniņā vai spēlē.

11.2. Spēlētājam jābūt iespējai norādīt, ka viņš nevar piedalīties treniņā vai spēlē.

11.3. Ja spēlētājs nevar piedalīties, sistēmai jāļauj ievadīt iemeslu.

11.4. Trenerim jābūt iespējai apskatīt visu spēlētāju atbildes konkrētam notikumam.

11.5. Trenerim jābūt iespējai apskatīt komandas apmeklējuma statistiku.

11.6. Apmeklējuma statusam jābūt: klāt vai prombūtnē.

11.7. Apmeklējumu drīkst atzīmēt treniņa un spēles tipa notikumiem.

11.8. Sistēmai jāaprēķina spēlētāju apmeklējuma procents.

12. Trenera pievienošanās pieprasījumu pārvaldīšana:

12.1. Trenerim jābūt iespējai nosūtīt pieprasījumu pievienoties komandai kā trenera asistentam.

12.2. Pieprasījumu var nosūtīt, ievadot komandas kodu.

12.3. Sistēmai jāpārbauda, vai trenerim jau nav aktīva pievienošanās pieprasījuma.

12.4. Galvenajam trenerim jābūt iespējai apskatīt savas komandas pievienošanās pieprasījumus.

12.5. Galvenajam trenerim jābūt iespējai apstiprināt pieprasījumu.

12.6. Galvenajam trenerim jābūt iespējai noraidīt pieprasījumu.

12.7. Pēc pieprasījuma apstiprināšanas treneris tiek piesaistīts komandai.

12.8. Pieprasījumam jāglabā statuss: gaida, apstiprināts vai noraidīts.

13. Administratora panelis:

13.1. Administratoram jābūt iespējai apskatīt visu lietotāju sarakstu.

- 13.2. Administratoram jābūt iespējai meklēt lietotājus pēc vārda vai e-pasta.
- 13.3. Administratoram jābūt iespējai filtrēt lietotājus pēc lomas.
- 13.4. Administratoram jābūt iespējai rediģēt lietotāja vārdu, uzvārdu, e-pastu, lomu un komandas ID.
- 13.5. Administratoram jābūt iespējai dzēst lietotāju.
- 13.6. Pirms lietotāja dzēšanas sistēmai jāizvada brīdinājums.
- 13.7. Administratoram jābūt iespējai apskatīt visu komandu sarakstu.
- 13.8. Administratoram jābūt iespējai meklēt komandas pēc nosaukuma, koda vai trenera.
- 13.9. Administratoram jābūt iespējai rediģēt komandas nosaukumu un komandas kodu.
- 13.10. Komandas kodam jābūt unikālam.
- 13.11. Administratora panelim jābūt pieejamam tikai lietotājiem ar administratora lomu.
14. Piekļuves tiesību kontrole:
 - 14.1. Sistēmai jāierobežo piekļuve funkcijām atbilstoši lietotāja lomai.
 - 14.2. Neautorizēts lietotājs nedrīkst piekļūt aizsargātām sistēmas sadaļām.
 - 14.3. Spēlētājam jābūt pieejai tikai savas komandas informācijai.
 - 14.4. Trenerim jābūt pieejai savas komandas pārvaldības funkcijām.
 - 14.5. Galvenajam trenerim jābūt pieejai asistenta trenera pieprasījumu izskatīšanai.
 - 14.6. Administratoram jābūt pieejai lietotāju un komandu pārvaldības sadaļām.
 - 14.7. Ja lietotājam nav nepieciešamo tiesību, sistēmai jāliedz darbība un jāizvada paziņojums.
15. Meklēšana, filtrēšana un kārtošana:
 - 15.1. Sistēmā jānodrošina spēlētāju meklēšana pēc vārda, uzvārda vai e-pasta.
 - 15.2. Sistēmā jānodrošina notikumu meklēšana pēc nosaukuma vai vietas.
 - 15.3. Sistēmā jānodrošina notikumu filtrēšana pēc tipa.
 - 15.4. Administratora panelī jānodrošina lietotāju meklēšana un filtrēšana pēc lomas.
 - 15.5. Administratora panelī jānodrošina komandu meklēšana pēc nosaukuma, koda vai trenera.
 - 15.6. Meklēšanas un filtrēšanas rezultātiem jāatbilst ievadītajiem kritērijiem.
16. Sistēmas saskarnes iestatījumi:
 - 16.1. Lietotājam jābūt iespējai pārslēgt sistēmas valodu starp latviešu un angļu valodu.
 - 16.2. Lietotājam jābūt iespējai pārslēgt gaišo un tumšo režīmu.
 - 16.3. Sistēmai jāpielāgo saskarne mobilajām ierīcēm.
 - 16.4. Sistēmai jānodrošina navigācija uz profilu, komandu, grafiku, čatu un administrācijas sadaļu.
 - 16.5. Sistēmai jāattēlo veiksmīgu darbību un kļūdu paziņojumi pēc lietotāja darbībām.
17. Spēļu sastāva pārvaldīšana:
 - 17.1. Trenerim jābūt iespējai izveidot spēlētāju sarakstu konkrētai spēlei.
 - 17.2. Trenerim jābūt iespējai izvēlēties spēlētājus no komandas sastāva.
 - 17.3. Nākamās spēles sarakstam jābūt automātiski aizpildītam ar iepriekšējās spēles spēlētājiem.
 - 17.4. Trenerim jābūt iespējai mainīt automātiski aizpildīto sarakstu pirms saglabāšanas.
 - 17.5. Spēlētājam jābūt iespējai apstiprināt savu dalību spēlē.
 - 17.6. Spēlētājam jābūt iespējai norādīt, ka viņš nevar piedalīties spēlē.
 - 17.7. Ja spēlētājs nevar piedalīties, sistēmai jāļauj ievadīt iemeslu.
 - 17.8. Trenerim jābūt iespējai apskatīt visu iekļauto spēlētāju apstiprinājuma statusus.
 - 17.9. Sastāvu drīkst veidot tikai spēles tipa notikumiem.

17.10. Apmeklējuma statuss spēlei jā saglabā tabulā "attendance" ar tiem pašiem statusiem kā treniņiem.

2.3. Nefunkcionālās prasības

1. Sistēmas saskarnei ir jābūt latviešu un angļu valodā.
2. Sistēmai jābūt responsīvai - pielāgoties dažādiem ekrāna izmēriem.
3. Sistēmai jābūt pieejamai populārākajos pārlūkos (Chrome, Firefox, Edge, Safari).
4. Dati jāšifrē ar drošiem algoritmiem.
5. Sistēmai jānodrošina kļūdu paziņojumi (veiksmes/kļūda).
6. Dizainam jābūt mūsdienīgam, ar tumšā režīma atbalstu.
7. Tekstiem jābūt tumšā krāsā uz gaiša fona.
8. Lietotāja pieslēgšanās formai jāizskatās sekojoši (skat. 2. att.).



The diagram shows a login form within a rounded rectangle. At the top, the word "Login" is centered. Below it, the label "Username:" is centered, followed by a horizontal input field. Below the input field, the label "Password:" is centered, followed by another horizontal input field.

2. att. Pieslēgšanās forma.

9. Notikuma pievienošana jāizskatās sekojoši (skat. 3. att.).

Add Event

Title

Practice, Game, etc.

Date

23.10.2025

Time

18:00

Type

Game

Location

Skonto stadium

Cancel

Add

3. att. Notikuma pievienošanas formas skice.

10. Spēlētāju statistika jāizskatās sekojoši (skat. 4. att).

Player name and Surname

Matches:

10

Goals:

4

Assists:

6

Yellow Cards:

2

Red Cards:

0

4. att. Spēlētāju statistikas formas skice.

3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU IZVĒLES PAMATOJUMS

Sistēma sastāv no divām galvenajām daļām: lietotāja puses (frontend), kas darbojas pārlūkprogrammā un arī mobilajās ierīcēs, un servera puses (backend), kas atbild par biznesa loģiku, piekļuves kontroli, datu apstrādi un glabāšanu datubāzē.

Lietotāja puses izstrādei tika izvēlēts Vue.js 3.5.33 kopā ar Vite 6.4.2. Šī kombinācija ļauj ātri izstrādāt lietotni, ērti strukturēt kodu komponentēs un vienkārši to uzturēt. Navigācijai tiek izmantots Vue Router 4.6.4, kas nodrošina pārejas starp lapām, bet lietotnes stāvokļa pārvaldībai - Pinia 3.0.4. Saziņai ar serveri (API pieprasījumiem) tiek izmantots Axios 1.15.2. Day.js 1.11.20 izmantots datumu apstrādei, statistikas datu vizualizācijai - Chart.js 4.5.1. Reāllaika saziņas funkcionalitātei klienta pusē izmantots Socket.io-client 4.8.3, kas ļauj lietotājiem apmainīties ar ziņām bez lapas pārlādes.

Servera puse ir realizēta, izmantojot Node.js vidi un Express 4.22.1 ietvaru. Šī pieeja nodrošina elastīgu REST API izveidi un efektīvu pieprasījumu apstrādi. Lietotāju autentifikācijai un autorizācijai tiek izmantots jsonwebtoken 9.0.3, kas ļauj droši pārvaldīt piekļuvi sistēmai. Paroļu drošai glabāšanai izmantots bcrypt 5.1.1, savukārt e-pasta paziņojumu nosūtīšanai - nodemailer 8.0.6. Starpdomēnu piekļuves kontrolei tiek izmantots cors 2.8.6, bet konfigurācijas parametru pārvaldībai - dotenv 16.6.1. Reāllaika tērzēšanas funkcionalitātei servera pusē izmantots Socket.IO 4.8.3. Datu glabāšanai izvēlēta SQLite (sqlite3 5.1.7), jo tā ir pietiekama šī projekta vajadzībām, neprasa atsevišķa datubāzes servera uzturēšanu un ir ērti integrējama prototipā.

Sistēmas izstrādei tika izmantota Visual Studio Code vide. Tā tika izvēlēta, jo nodrošina ērtu darbu ar JavaScript, Vue un Node.js tehnoloģijām, piedāvā iebūvētu termināli, plašu paplašinājumu klāstu un palīdz efektīvi pārvaldīt projekta struktūru. Projekta versiju kontrolei tika izmantots Git, bet pirmkods tiek glabāts GitHub repozitorijā. Tas ļauj sekot līdzi izmaiņu vēsturei, droši uzglabāt datus un organizēt izstrādes procesu strukturētā veidā.

Izmantotie izstrādes rīki un bibliotēkas.

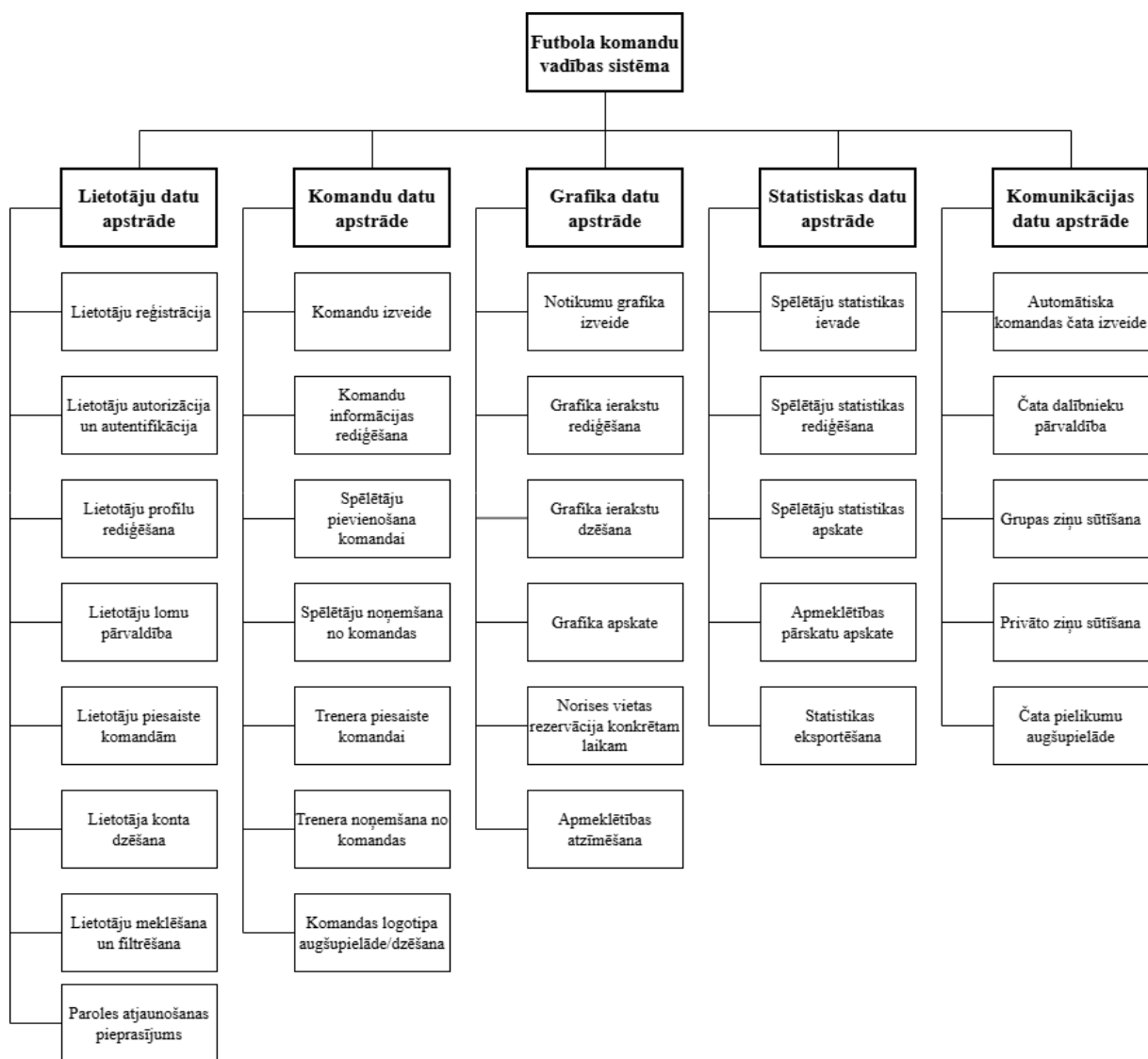
Komponents	Tehnoloģija	Versija	Pamatojums
Frontend ietvars	Vue.js	3.5.33	Komponentu pieeja, ātra SPA izstrāde.
Būvēšanas rīks	Vite	6.4.2	Ātrs izstrādes serveris un optimizēta būvēšana.
Maršrutēšana	Vue Router	4.6.4	Lapu navigācija un aizsargāto maršrutu kontrole.
Stāvokļa pārvaldība	Pinia	3.0.4	Centralizēta lietotāja un sistēmas datu glabāšana frontend pusē.
API klients	Axios	1.15.2	Ērta HTTP pieprasījumu un kļūdu apstrāde.
Diagrammas	Chart.js	4.5.1	Statistikas vizualizācija.
Datumi	Day.js	1.11.20	Datumu formatēšana un salīdzināšana.
Backend ietvars	Express	4.22.1	REST API maršruti un middleware.
Datubāze	SQLite/sqlite3	5.1.7	Viegla failu bāzēta relāciju datubāze.
Reāllaika saziņa	Socket.IO	4.8.3	Grupu čats un privātās ziņas bez lapas pārlādes.
Autentifikācija	jsonwebtoken	9.0.3	JWT tokenu ģenerēšana un pārbaude.
Paroles	bcrypt	5.1.1	Droša hash + salt pieeja.

4. PROGRAMMATŪRAS PRODUKTA MODELĒŠANA UN PROJEKTĒŠANA

4.1. Sistēmas struktūras modelis

4.1.1. Sistēmas arhitektūra

"TeamFlow" tiks veidota no piecām apakšsistēmām (skat. 5. att.): lietotāju datu apstrāde, komandu datu apstrāde, grafika datu apstrāde, statistikas datu apstrāde un komunikācijas datu apstrāde.



5. att. Sistēmas arhitektūras diagramma.

Zemāk ir uzskaitītas visas apakšsistēmas un tajās ietilpstošie moduļi ar to funkcionālo nozīmi:

Lietotāju datu apstrāde

- Lietotāju reģistrācija - nodrošina jaunu lietotāju kontu izveidi sistēmā.
- Lietotāju autorizācija un autentifikācija - veic lietotāju identitātes pārbaudi drošai piekļuvei sistēmas resursiem.
- Lietotāju profilu rediģēšana - ļauj lietotājiem pārvaldīt savus datus un veikt nepieciešamās izmaiņas.
- Lietotāju lomu pārvaldība - nodrošina tiesību sadali starp treneriem, spēlētājiem un administratoriem.
- Lietotāju piesaiste komandām - realizē spēlētāju un personāla reģistrēšanu pie konkrētām sporta vienībām.
- Lietotāja konta dzēšana - modulis lietotāja informācijas pilnīgai vai daļējai izņemšanai no sistēmas.
- Lietotāju meklēšana un filtrēšana - rīks lietotāju saraksta ērtai pārlūkošanai pēc dažādiem parametriem.
- Paroles atjaunošanas pieprasījums - automatizēts process piekļuves atgūšanai paroles nozaudēšanas gadījumā.

Komandu datu apstrāde

- Komandu izveide - nodrošina jaunu komandu struktūru reģistrēšanu.
- Komandu informācijas rediģēšana - iespēja rediģēt datus par komandu.
- Spēlētāju pievienošana un noņemšana no komandas - rīks komandas sastāva dinamiskai pārvaldībai.
- Trenera piesaiste komandai - nodrošina vadības un atbildības saiti starp treneri un sportistiem.
- Komandas logotipa augšupielāde/dzēšana - ļauj trenerim pievienot, mainīt vai dzēst komandas logotipu

Grafika datu apstrāde

- Notikumu grafika izveide - centrālais modulis notikumu plānošanai un laika vadībai.
- Grafika ierakstu rediģēšana - ļauj veikt izmaiņas jau izveidotu notikumu datos
- Grafika ierakstu dzēšana - nodrošina atceltu notikumu izņemšanu no sistēmas grafika.
- Grafika apskate - vizuāls interfeiss ērtai notikumu plāna pārskatīšanai.
- Norises vietas rezervācija konkrētam laikam - kontrolē resursu noslodzi un novērš notikumu pārklāšanos.
- Grafika filtrēšana pēc laika perioda - nodrošina iespēju atlasīt pasākumus konkrētā datumā vai nedēļā.
- Apmeklētības atzīmēšana - nodrošina disciplīnas kontroli un klātbūtnes uzskaiti pasākumos.

Statistiskas datu apstrāde

- Spēlētāju statistikas ievade - jaunu sportisko rādītāju (vārti, piespēles) fiksēšana pēc notikušas spēles.
- Spēlētāju statistikas rediģēšana - iespēja rediģēt iepriekš ievadītos statistikas datus.
- Spēlētāju statistikas apskate - analītisks rīks individuālā snieguma un progresa vērtēšanai.

- Apmeklētības pārskatu apskate - ļauj treneriem vizuāli pārskatīt apkopotos datus par spēlētāju klātbūtni.

Komunikācijas datu apstrāde

- Automātiska komandas čata izveide - nodrošina tūlītēju kopējās saziņas grupas izveidi brīdī, kad sistēmā tiek reģistrēta jauna komanda.
- Čata dalībnieku pārvaldība - automātiski kontrolē treneru un spēlētāju pievienošanu komandas čatam atkarībā no viņu piederības komandai.
- Grupas ziņu sūtīšana - nodrošina ziņojumu apmaiņu starp visiem komandas biedriem kopējā tērzētavā.
- Privāto ziņu sūtīšana - realizē tiešu saziņu starp diviem lietotājiem, atļaujot sūtīt ziņas tikai savas komandas biedru ietvaros.
- Čata pielikumu augšupielāde - nodrošina iespēju lietotājiem pievienot čata ziņojumiem atbalstītus failus

4.1.2. Sistēmas ER modelis

ER modelis raksturo sistēmas galvenās entītijas un to savstarpējās saites konceptuālā līmenī. ER diagramma (skat. 6. att.) sastāv no vienpadsmit entītijām, kas nodrošina lietotāju, komandu, grafika, statistikas, apmeklētības un komunikācijas datu uzglabāšanu un apstrādi sistēmā.

“Lietotājs” - uzskaita visus sistēmas lietotājus. Šī entītija satur informāciju par lietotāja vārdu, uzvārdu, e-pastu, paroli, avatāru un lietotāja lomu. Lietotājs var būt spēlētājs, treneris vai administrators.

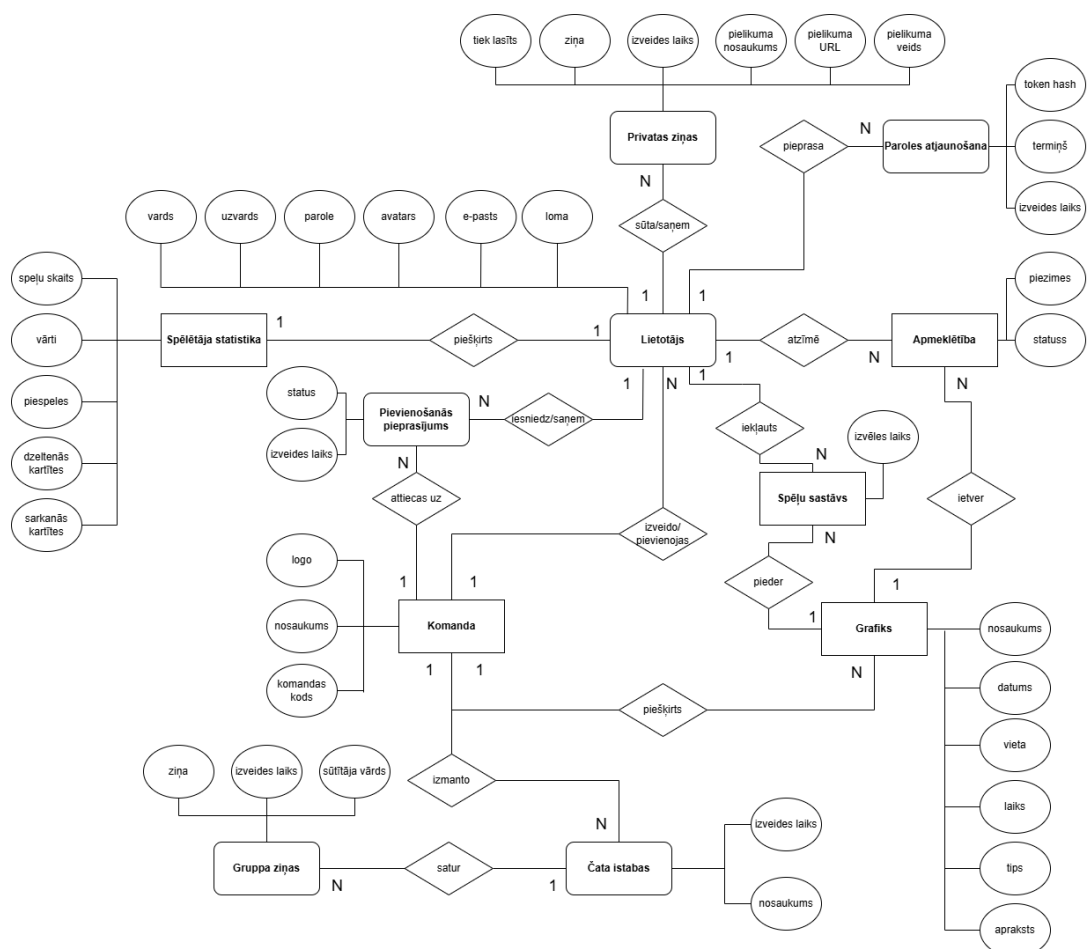
“Komanda” - satur informāciju par futbola komandām. Entītijas atribūti ietver komandas nosaukumu, unikālu komandas kodu un logotipu. Komandas kods tiek izmantots lietotāju piesaistei konkrētai komandai.

“Grafiks” - glabā informāciju par komandas treniņiem un spēlēm. Entītija satur notikuma nosaukumu, datumu, laiku, norises vietu, tipu un aprakstu.

“Spēlētāja statistika” - satur informāciju par spēlētāju individuālajiem sportiskajiem rādītājiem, piemēram, spēļu skaitu, gūtajiem vārtiem, piespēlēm, dzeltenajām un sarkanajām kartītēm.

“Apmeklētība” - nodrošina informāciju par spēlētāju piedalīšanos treniņos un dalības apstiprinājumu spēlēs. Entītija satur apmeklējuma statusu un piezīmes, kā arī ir saistīta ar konkrētu lietotāju un grafika notikumu.

"Spēļu sastāvs" - glabā informāciju par trenerim izveidoto spēlētāju sarakstu konkrētai spēlei. Entūtijā satur spēlētāja ID, spēles notikuma ID, dalības statusu un piezīmes.



6. att. ER model diagramma.

ER modeļa relācijas ir aprakstītas tālāk:

Starp entītijām “Komanda” un “Lietotājs” pastāv attiecība viens pret daudziem, jo vienai komandai var būt vairāki lietotāji, bet katrs lietotājs var būt piesaistīts tikai vienai komandai.

Starp entītijām “Komanda” un “Grafiks” pastāv attiecība viens pret daudziem, jo vienai komandai var būt vairāki grafika notikumi, bet katrs notikums pieder tikai vienai komandai.

Starp entītijām “Komanda” un “Čata istabas” pastāv attiecība viens pret daudziem, jo vienai komandai var būt viena vai vairākas čata istabas komandas komunikācijai, bet katra čata istaba pieder vienai komandai.

Starp entītijām “Čata istabas” un “Grupu ziņas” pastāv attiecība viens pret daudziem, jo vienā čata istabā var būt vairākas grupas ziņas, bet katra grupas ziņa pieder vienai čata istabai.

Starp entītijām “Lietotājs” un “Grupu ziņas” pastāv attiecība viens pret daudziem, jo viens lietotājs var nosūtīt vairākas grupas ziņas, bet katru grupas ziņu nosūta viens lietotājs.

Starp entītijām “Lietotājs” un “Privātas ziņas” pastāv attiecība viens pret daudziem, jo viens lietotājs var nosūtīt vairākas privātās ziņas.

Starp entītijām “Lietotājs” un “Privātas ziņas” pastāv attiecība viens pret daudziem arī saņēmēja nozīmē, jo viens lietotājs var saņemt vairākas privātās ziņas.

Starp entītijām “Lietotājs” un “Spēlētāja statistika” pastāv attiecība viens pret vienu, jo katram spēlētājam ir viens individuālās statistikas ieraksts.

Starp entītijām “Lietotājs”, “Grafiks” un “Apmeklētība” pastāv attiecība daudzi pret daudziem, kas realizēta ar starpentītijū “Apmeklētība”, jo viens lietotājs var būt atzīmēts vairākos notikumos, un vienam notikumam var būt vairāki lietotāju apmeklētības ieraksti.

Starp entītijām “Lietotājs” un “Paroles atjaunošana” pastāv attiecība viens pret daudziem, jo viens lietotājs var izveidot vairākus paroles atjaunošanas pieprasījumus, bet katrs paroles atjaunošanas pieprasījums pieder vienam lietotājam.

Starp entītijām “Komanda” un “Trenera pievienošanās pieprasījumi” pastāv attiecība viens pret daudziem, jo viena komanda var saņemt vairākus trenera pievienošanās pieprasījumus, bet katrs pieprasījums attiecas uz vienu konkrētu komandu.

Starp entitijām "Lietotājs" un "Trenera pievienošanās pieprasījumi" pastāv attiecība viens pret daudziem, jo viens treneris var iesniegt vairākus pievienošanās pieprasījumus, bet katru pieprasījumu iesniedz viens lietotājs.

Starp entitijām "Lietotājs" un "Trenera pievienošanās pieprasījumi" pastāv attiecība viens pret daudziem arī pieprasījuma izskatīšanas nozīmē, jo viens galvenais treneris var izskatīt vairākus pievienošanās pieprasījumus, bet katru izskatītu pieprasījumu apstiprina vai noraida viens lietotājs.

Starp entitijām "Grafiks" un "Spēļu sastāvs" pastāv attiecība viens pret daudziem, jo vienam spēles notikumam var būt vairāki spēlētāju saraksta ieraksti, bet katrs ieraksts pieder vienam notikumam.

Starp entitijām "Lietotājs" un "Spēļu sastāvs" pastāv attiecība viens pret daudziem, jo viens spēlētājs var figurēt vairāku spēļu sastāvos, bet katrs sastāva ieraksts attiecas uz vienu spēlētāju.

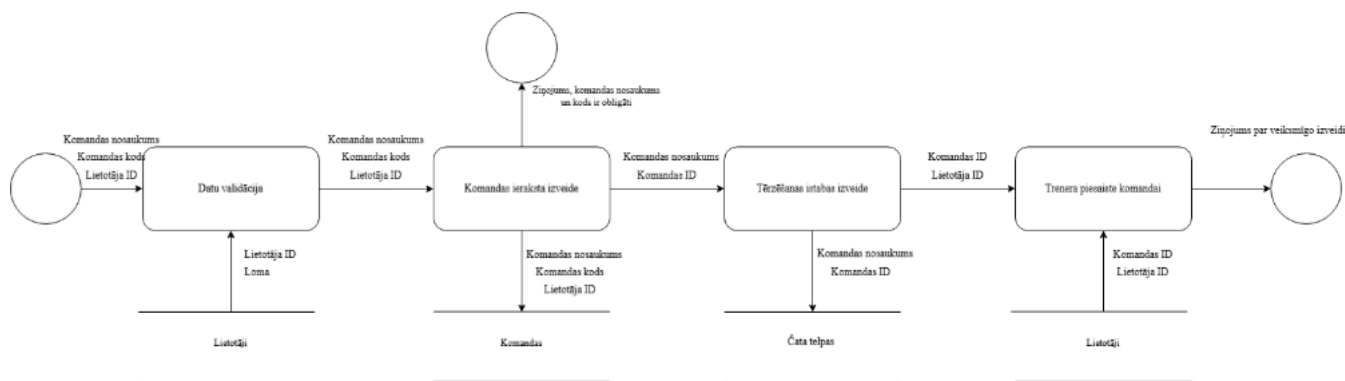
4.2. Funkcionālais sistēmas modelis

4.2.1. Datu plūsmu modelis

Šajā nodaļā ir sniegts detalizēts "TeamFlow" sistēmas procesu apraksts, izmantojot datu plūsmas diagrammu (DPD) metodi. Katram procesam ir aprakstīta datu transformācija no ievades līdz gala rezultātam.

Komandas izveide (skat. 7. att.)

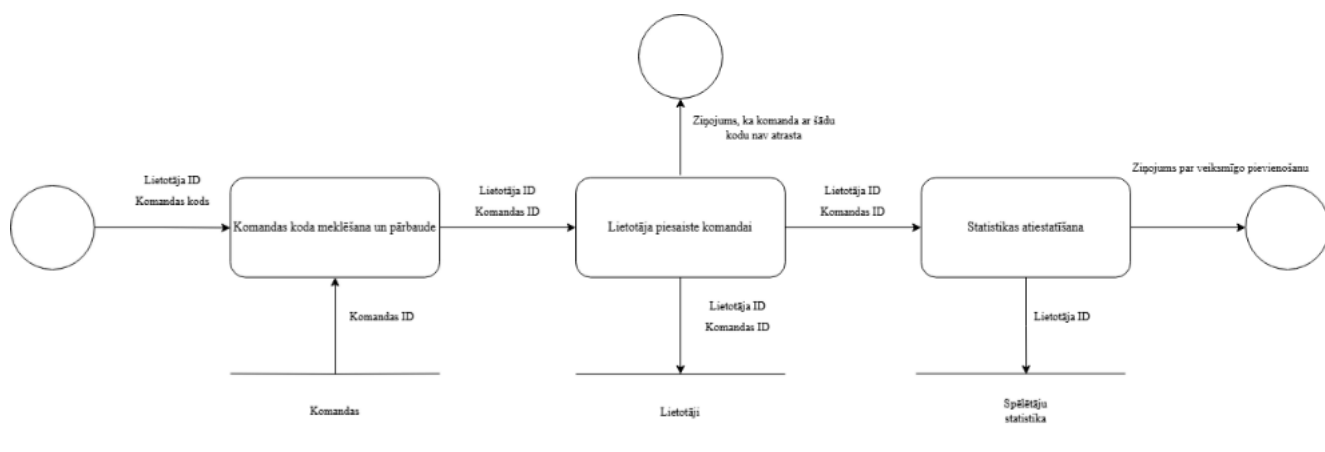
Treneris iesniedz pieprasījumu ar komandas nosaukumu un unikālu komandas kodu. Vispirms notiek komandas ieraksta izveide - datu krātuvē "Komandas" tiek saglabāts jauns ieraksts ar komandas nosaukumu, komandas kodu un trenera identifikatoru. Pēc tam notiek tērēšanas telpas izveide - sistēma automātiski izveido ierakstu datu krātuvē "Tērēšanas telpas" ar komandas identifikatoru un telpas nosaukumu formātā Komandas nosaukums + tērēšana, lai komandas dalībnieki varētu nekavējoties sākt saziņu. Visbeidzot notiek trenera piesaiste komandai - datu krātuvē "Lietotāji" tiek atjaunināts trenera ieraksts, iestatot komandas identifikatoru uz jaunizveidotās komandas identifikatoru. Trenerim tiek atgriezts veiksmīgas izveides apstiprinājums ar komandas datiem.



7. att. Komandas izveides datu plūsmas diagramma.

Pievienošanās komandai (skat. 8. att.)

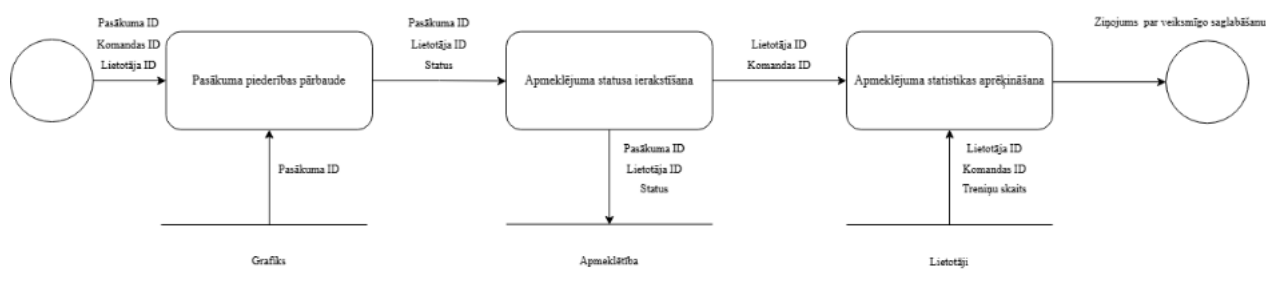
Spēlētājs ievada komandas kodu, lai pievienotos komandai. Vispirms notiek komandas koda meklēšana un pārbaude - no datu krātuves "Komandas" tiek nolasīts ieraksts ar atbilstošo komandas koda vērtību un iegūts komandas identifikators. Ja komanda ar norādīto kodu nav atrasta, tiek atgriezts kļūdas paziņojums. Pēc veiksmīgas pārbaudes notiek lietotāja piesaiste komandai un statistikas atiestatīšana - datu krātuvē "Lietotāji" tiek atjaunināts lietotāja ieraksts, iestatot komandas identifikatoru uz atrasto komandas identifikatoru. Vienlaikus datu krātuvē "Spēlētāju statistika" spēlētāja rādītāji tiek iestatīti uz nulles vērtībām (spēles, vārti, piespēles, dzeltenās kartītes, sarkanās kartītes), izmantojot ievietošanas vai atjaunināšanas darbību, lai nodrošinātu korektu sākuma stāvokli jaunajā komandā. Lietotājam tiek atgriezts veiksmīgas pievienošanās apstiprinājums.



8. att. Pievienošanās komandai datu plūsmas diagramma.

Apmeklējuma uzskaite (skat. 9. att.)

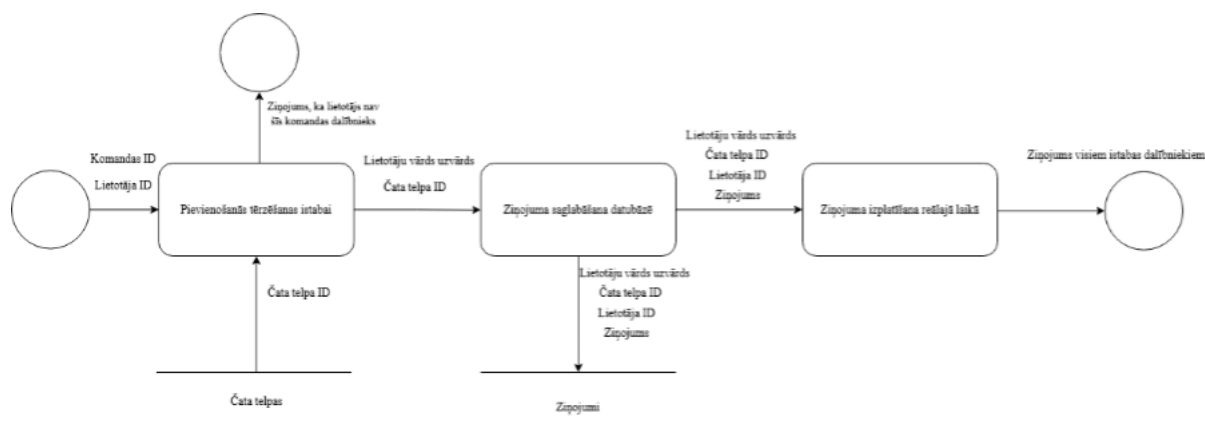
Spēlētājs atver konkrētu treniņa notikumu grafikā un iesniedz savu apmeklējuma statusu. Vispirms sistēma pārbauda, vai lietotājs ir autorizēts, vai viņš ir piesaistīts attiecīgajai komandai un vai izvēlētais notikums pieder šai komandai. Papildus tiek pārbaudīts, vai notikuma tips ir treniņš vai spēle, jo apmeklējuma atzīmēšana tiek veikta tikai treniņu un spēļu notikumiem. Spēles gadījumā statuss tiek izmantots kā dalības apstiprinājums. Ja spēlētājs plāno piedalīties treniņā, viņš izvēlas statusu “klāt”. Ja spēlētājs nevar piedalīties, viņš izvēlas statusu “prombūtnē” un pēc nepieciešamības norāda neierašanās iemeslu. Sistēma saglabā apmeklējuma informāciju datu krātuvē “Apmeklējums”, izveidojot jaunu ierakstu vai atjauninot esošo ierakstu konkrētajam spēlētājam un notikumam. Šāda pieeja nodrošina, ka vienam spēlētājam vienā treniņā ir tikai viens apmeklējuma ieraksts. Pēc veiksmīgas saglabāšanas spēlētājam tiek parādīts apstiprinājuma paziņojums. Treneris šajā procesā apmeklējumu neatzīmē spēlētāja vietā, bet var apskatīt visu spēlētāju atbildes, neierašanās iemeslus un komandas apmeklējuma kopsavilkumu konkrētajam treniņam.



9. att. Apmeklējuma uzskaites datu plūsmas diagramma.

Komandas tērēšana reālajā laikā (skat. 10. att.)

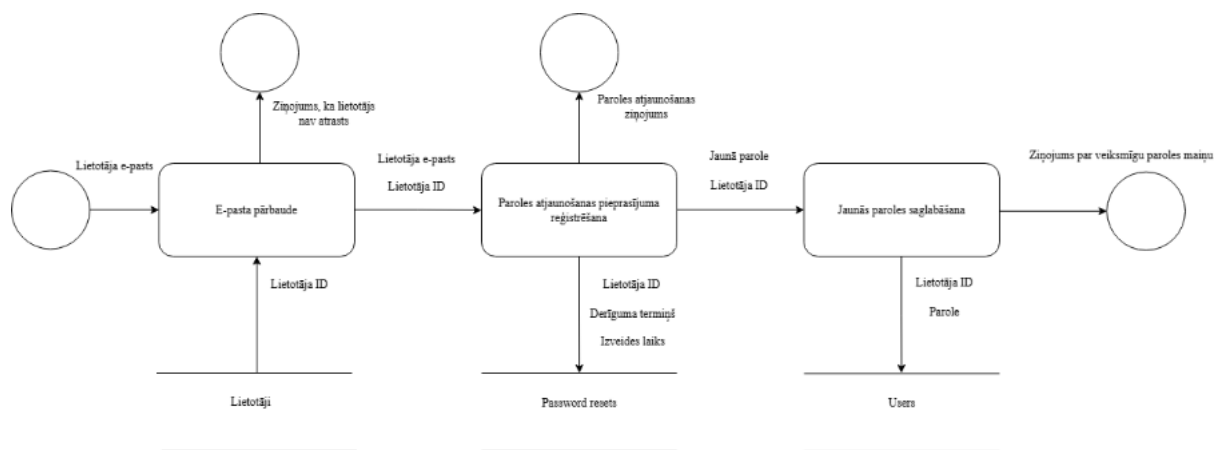
Lietotājs pieslēdzas sistēmai caur reāllaika savienojumu. Vispirms notiek pievienošanās tērēšanas telpai - no datu krātuves "Tērēšanas telpas" tiek nolasīts telpas ieraksts, lai pārbaudītu, vai telpa eksistē un pieder lietotāja komandai. Ja lietotājs nav komandas dalībnieks, tiek izvadīts kļūdas ziņojums, ka lietotājs nav šīs komandas dalībnieks. Ja autentifikācija neizdodas, tiek izvadīts kļūdas ziņojums par autentifikācijas kļūdu. Pēc veiksmīgas pievienošanās, kad lietotājs nosūta ziņojumu, notiek ziņojuma saglabāšana datubāzē - datu krātuvē "Ziņojumi" tiek izveidots jauns ieraksts ar telpas identifikatoru, lietotāja identifikatoru, lietotāja vārdu un ziņojuma tekstu. Ja saglabāšana neizdodas, tiek izvadīts kļūdas ziņojums par neizdevušos ziņojuma nosūtīšanu. Pēc veiksmīgas saglabāšanas notiek ziņojuma izplatīšana reālajā laikā - serveris nosūta ziņojumu visiem telpas dalībniekiem. Ja visi procesi norit veiksmīgi, ziņojums tiek attēlots visiem tērēšanas dalībniekiem.



10. att. Komandas tērēšanas reālajā laikā datu plūsmas diagramma.

Paroles atjaunošana (skat. 11. att.)

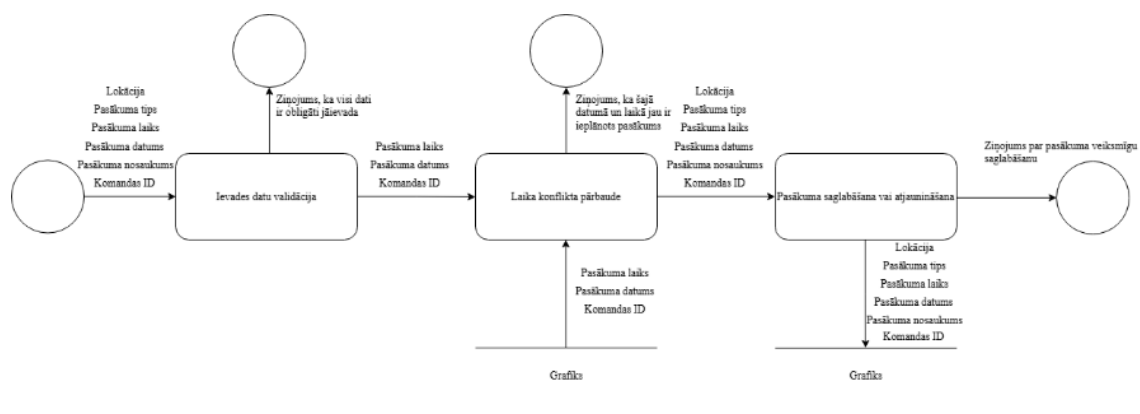
Lietotājs ievada savu e-pasta adresi paroles atjaunošanas formā. Vispirms notiek atjaunošanas pieprasījuma apstrāde - no datu krātuves Lietotāji tiek meklēts lietotāja ieraksts pēc e-pasta adreses, ģenerēts vienreizējs atjaunošanas tokens un aprēķināta tā hash vērtība ar derīguma termiņu. Ja e-pasts nav norādīts, tiek izvadīts kļūdas ziņojums "E-pasts ir obligāts". Pēc tam notiek atjaunošanas tokena saglabāšana un saites nosūtīšana - datu krātuvē "Paroles atjaunošana" tiek dzēsti iepriekšējie novecojušie ieraksti un izveidots jauns ieraksts ar tokena hash un derīguma laiku, pēc kā lietotājam tiek nosūtīta paroles atjaunošanas saite uz e-pastu. Ja e-pasta nosūtīšana neizdodas, tiek izvadīts kļūdas ziņojums "Neizdevās nosūtīt atjaunošanas e-pastu". Kad lietotājs atver saiti un ievada jaunu paroli, notiek tokena validācija un paroles nomaiņa - sistēma pārbauda, vai tokens nav izmantots un nav beidzies, pēc tam datu krātuvē Lietotāji tiek atjaunināts paroles hash, bet datu krātuvē "Paroles atjaunošana" tokens tiek atzīmēts kā izmantots. Ja tokens nav derīgs vai ir beidzies, tiek izvadīts kļūdas ziņojums "Atjaunošanas tokens nav derīgs vai ir beidzis darbību". Ja visi procesi noritējuši veiksmīgi, lietotājam tiek atgriezts paziņojums "Parole veiksmīgi atjaunota".



11. att. Paroles atjaunošanas datu plūsmas diagramma.

Pasākumu grafika pārvaldība (skat. 12. att.)

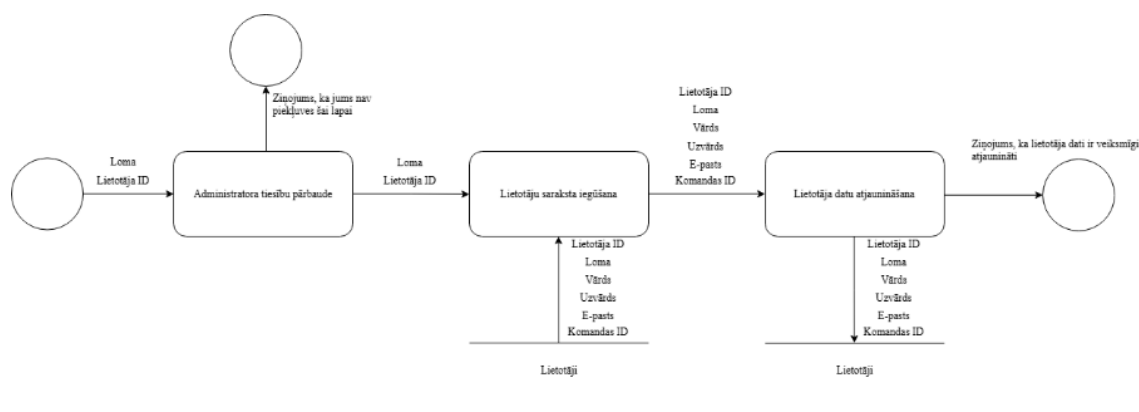
Treneris iesniedz pieprasījumu izveidot vai rediģēt komandas pasākumu. Vispirms notiek ievades datu validācija - sistēma pārbauda, vai ir norādīts pasākuma nosaukums un datums. Pēc tam notiek laika konflikta pārbaude - no datu krātuves Grafiks tiek nolasīti ieraksti pēc komandas ID, pasākuma datuma un laika, lai noteiktu, vai tajā pašā laikā jau neeksistē cits pasākums. Ja konflikts tiek atrasts, tiek izvadīts kļūdas ziņojums "Šajā datumā un laikā jau ir iepļānots pasākums". Ja konflikta nav, notiek pasākuma saglabāšana datubāzē - datu krātuvē Grafiks tiek izveidots jauns ieraksts vai atjaunināts esošais ieraksts ar pasākuma datiem. Pēc veiksmīgas saglabāšanas lietotājam tiek atgriezts apstiprinājums ar pasākuma datiem. Dzēšanas gadījumā sistēma dzēš pasākuma ierakstu no datu krātuves Grafiks, ja ieraksts nav atrasts, tiek izvadīts kļūdas ziņojums "Notikums nav atrasts". Ja visi procesi noritējuši veiksmīgi, komandas kalendārs tiek atjaunināts.



12. att. Pasākumu grafika pārvaldības datu plūsmas diagramma.

Lietotāju administrēšana (skat. 13. att.)

Administrators iesniedz pieprasījumu lietotāju saraksta iegūšanai, lietotāja datu rediģēšanai vai lietotāja dzēšanai. Vispirms notiek administratora tiesību pārbaude - sistēma pārbauda lietotāja lomu un piekļuves tiesības. Ja lietotājam nav administratora tiesību, tiek izvadīts kļūdas ziņojums "Aizliegts". Pēc veiksmīgas autorizācijas, pieprasot sarakstu, no datu krātuves Lietotāji tiek nolasīti visi lietotāju ieraksti. Rediģēšanas gadījumā sistēma atjaunina izvēlētajā lietotāja laukus: vārds, uzvārds, e-pasts, loma, komandas ID. Dzēšanas gadījumā no datu krātuves Lietotāji tiek dzēsts lietotāja ieraksts, saistītie ieraksti saistītajās tabulās tiek dzēsti pēc ārējo atslēgu noteikumiem. Ja datubāzes operācija neizdodas, tiek izvadīts kļūdas ziņojums par apstrādes kļūdu. Ja visi procesi noritējuši veiksmīgi, administratoram tiek atgriezts veiksmīgas darbības apstiprinājums.

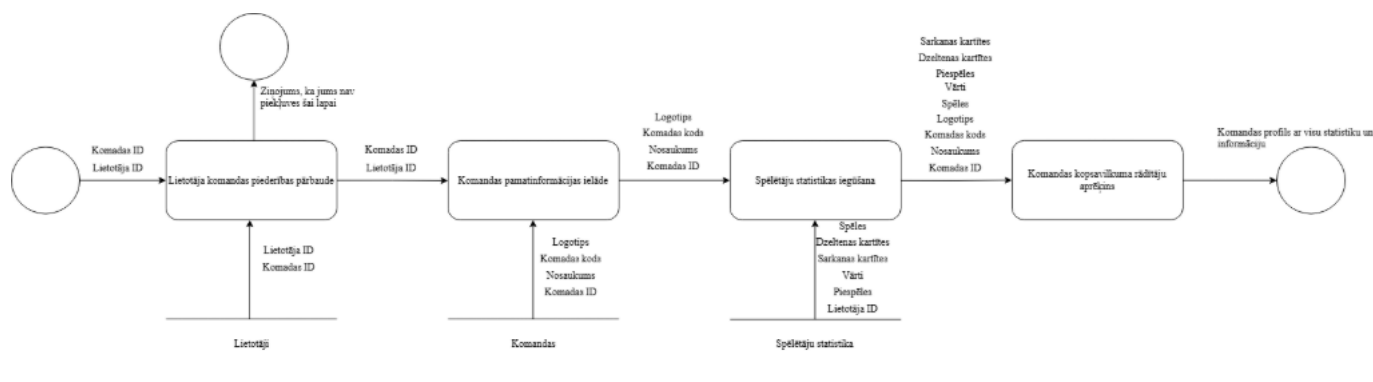


13. att. Lietotāju administrēšanas datu plūsmas diagramma.

Komandas profila informācijas iegūšanas (skat. 14. att.)

Lietotājs (treneris vai spēlētājs) atver komandas profilu, lai apskatītu pilnu informāciju par komandu. Vispirms notiek lietotāja autorizācijas pārbaude - sistēma pārbauda, vai lietotājs ir komandas dalībnieks, un nolasa komandas ID no datu krātuves Lietotāji. Pēc tam notiek komandas pamatinformācijas ielāde - no datu krātuves Komandas tiek nolasīts komandas nosaukums, kods, trenera dati un logotips. Nākamajā solī notiek komandas dalībnieku iegūšana - no datu krātuves Lietotāji ar filtrēšanu pēc komandas ID tiek atlasīti visi komandas dalībnieki. Vienlaicīgi notiek spēlētāju statistikas iegūšana - no datu krātuves Spēlētāju statistika tiek nolasīti katram spēlētājam atbilstoši statistikas ieraksti. Pēc tam notiek komandas kopsavilkuma rādītāju aprēķins - tiek apkopoti vārti, piespēles, kartītes un vidējais apmeklējums pēc visiem spēlētājiem. Visbeidzot notiek atbildes komplektēšana - visi dati tiek apvienoti atbildes objektā un nosūtīti lietotājam. Ja lietotājs nav komandas dalībnieks, tiek

atgriezts kļūdas paziņojums "Piekļuve ir aizliegta". Ja dati nav atrasti, tiek atgriezts paziņojums "Komanda nav atrasta".



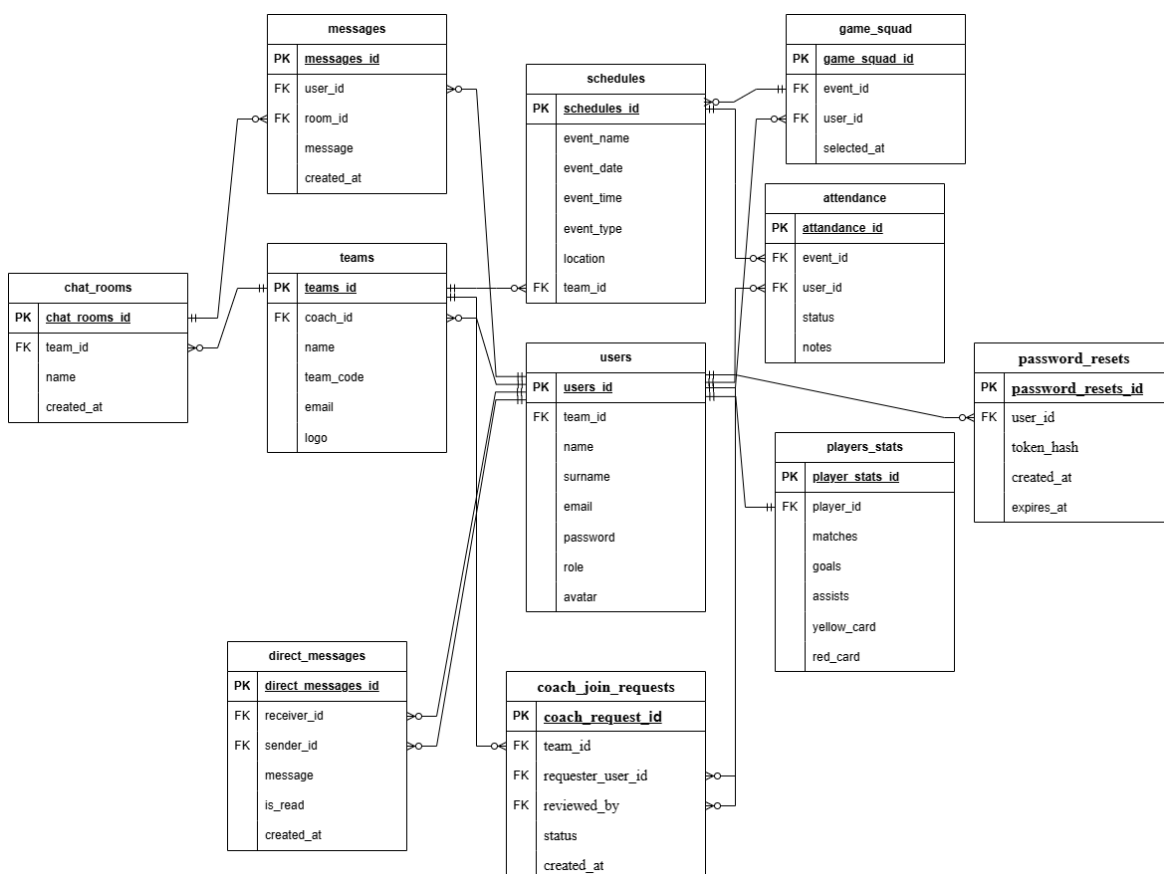
14. att. Komandas profila informācijas iegūšanas datu plūsmas diagramma.

5. DATU STRUKTŪRU APRAKSTS

Datubāzes izveidei tika izmantota relāciju datubāze SQLite, kas nodrošina datu strukturētu uzglabāšanu un efektīvu piekļuvi tiem. Datubāzes struktūra tika veidota, balstoties uz izstrādāto ER diagrammu, kas atspoguļo galvenās entitijas un to savstarpējās attiecības sistēmā.

Izveidotā datubāze sastāv no vairākām savstarpēji saistītām tabulām, kas nodrošina lietotāju pārvaldību, komandu organizēšanu, notikumu plānošanu, statistikas uzskaiti, apmeklētības kontroli un saziņas funkcionalitāti.

Datubāzes tabulu saišu shēma ir attēlota attēlā (skat. 15. att.).



15. att. Datubāzes tabulu saišu shēma.

Sistēmā tiek izmantotas šādas galvenās tabulas:

- Tabula “**users**” glabā informāciju par reģistrētajiem lietotājiem, tostarp viņu lomu, autentifikācijas datus un piesaisti komandai.
- Tabula “**teams**” satur informāciju par komandām, to nosaukumiem, kodiem, logotipiem un galvenajiem treneriem.
- Tabula “**schedules**” glabā informāciju par komandas notikumiem, piemēram, treniņiem, spēlēm un citiem pasākumiem.
- Tabula “**player_stats**” satur spēlētāju statistikas datus, piemēram, gūtos vārtus, piespēles un kartītes.
- Tabula “**attendance**” nodrošina informāciju par spēlētāju piedalīšanos treniņos un dalības apstiprinājumu spēlēs..
- Tabula “**chat_rooms**” satur informāciju par komandas čata telpām.
- Tabula “**messages**” glabā grupas čata ziņojumus un to metadatus.
- Tabula “**direct_messages**” nodrošina privāto ziņu funkcionalitāti starp lietotājiem.
- Tabula “**password_resets**” tiek izmantota paroles atjaunošanas funkcionalitātes nodrošināšanai.
- Tabula “**coach_join_requests**” glabā trenera pievienošanās pieprasījumus komandām un to statusus.
- Tabula “**game_squad**” glabā trenerim izveidoto spēlētāju sarakstu konkrētām spēlēm un katra spēlētāja dalības apstiprinājuma statusu.

5.1.tabula

Tabulas “users” struktūra

Kolonna	Datu tips	Apraksts
users_id	integer (PK)	Unikāls lietotāja identifikators sistēmā.
name	varchar(255)	Lietotāja vārds.
email	varchar(255)	Lietotāja unikālā e-pasta adrese, kas tiek izmantota autorizācijai.
password	varchar(255)	Šifrēta lietotāja parole (hash formātā).
role	varchar(255)	Lietotāja loma sistēmā (spēlētājs, treneris, administrators).
team_id	integer (FK)	Saite uz komandu, kurai lietotājs ir piesaistīts (team_id).

5.2.tabula

Tabulas “teams” struktūra

Kolonna	Datu tips	Apraksts
teams_id	integer (PK)	Unikāls komandas identifikators.
name	varchar(255)	Komandas nosaukums.
teamCode	varchar(255)	Unikāls komandas kods, kas tiek izmantots lietotāju pievienošanai komandai.
logo	varchar(255)	Komandas logotipa atrašanās vieta vai attēlojuma dati.
coach_id	integer (FK)	Galvenā trenera identifikators (saite uz users.id).

5.3.tabula

Tabulas “schedules” struktūra

Kolonna	Datu tips	Apraksts
schedules_id	integer (PK)	Unikāls notikuma identifikators.
event_name	varchar(255)	Notikuma nosaukums.
event_date	date	Notikuma norises datums.
event_time	time	Notikuma sākuma laiks.
event_type	varchar(255)	Notikuma tips (piemēram, treniņš, spēle, sapulce vai cits).
location	varchar(255)	Notikuma norises vieta.
team_id	integer (FK)	Saite uz komandu, kurai pieder konkrētais notikums.

5.4 tabula

Tabulas “player_stats” struktūra

Kolonna	Datu tips	Apraksts
player_stats_id	integer (PK)	Unikāls statistikas ieraksta identifikators.
matches	integer	Aizvadīto spēļu skaits.
goals	integer	Gūto vārtu skaits.
assists	integer	Rezultatīvo piespēļu skaits.
yellow_card	integer	Saņemto dzelteno kartīšu skaits.
red_card	integer	Saņemto sarkano kartīšu skaits.
player_id	integer (FK)	Saite uz lietotāju (spēlētāju), kuram pieder statistikas dati (users.id).

5.5.tabula

Tabulas “attendance” struktūra

Kolonna	Datu tips	Apraksts
attendance_id	integer (PK)	Unikāls apmeklētības ieraksta identifikators.
notes	varchar(255)	Papildinformācija vai paskaidrojums par neierašanos vai kavējumu.
status	varchar(255)	Apmeklētības statuss (klāt, prombūtnē).
player_id	integer (FK)	Saite uz spēlētāju (users.id), uz kuru attiecas ieraksts.
event_id	integer (FK)	Saite uz konkrēto notikumu (schedule_id).

5.6.tabula

Tabulas “messages” struktūra

Kolonna	Datu tips	Apraksts
messages_id	integer (PK)	Unikāls ziņojuma identifikators.
message	text	Ziņojuma tekstuālais saturs.
created_at	datetime	Datums un laiks, kad ziņojums tika izveidots.
user_id	integer (FK)	Lietotāja identifikators, kurš nosūtīja ziņojumu.
room_id	integer (FK)	Saite uz čata telpu, kurā ziņojums tika nosūtīts.

5.7 tabula

Tabulas “chat_rooms” struktūra

Kolonna	Datu tips	Apraksts
chat_room_id	integer (PK)	Unikāls čata telpas identifikators.
name	varchar(255)	Čata telpas nosaukums.
created_at	datetime	Čata telpas izveidošanas datums un laiks.
team_id	integer (FK)	Saite uz komandu, kurai pieder čata telpa.

5.8.tabula

Tabulas “direct_messages” struktūra

Kolonna	Datu tips	Apraksts
direct_messages_id	integer (PK)	Unikāls privātās ziņas identifikators.
message	text	Privātās ziņas teksts.
is_read	boolean	Norāda, vai ziņa ir izlasīta (true/false).
created_at	datetime	Ziņas nosūtīšanas datums un laiks.
receiver_id	integer (FK)	Saņēmēja identifikators (users.id).
sender_id	integer (FK)	Sūtītāja identifikators (users.id).

5.9.tabula

Tabulas “coach_join_requests” struktūra

Kolonna	Datu tips	Apraksts
coach_request_id	integer (PK)	Trenera pievienošanās pieprasījuma identifikators
team_id	integer (FK)	Komanda, kurai treneris vēlas pievienoties
requester_user_id	integer (FK)	Treneris, kurš nosūtīja pieprasījumu
status	varchar(255)	Pieprasījuma statuss: gaida, apstiprināts vai noraidīts
created_at	datetime	Pieprasījuma izveides laiks
reviewed_by	integer (FK)	Galvenais treneris, kurš izskatīja pieprasījumu

5.10.tabula.

Tabulas “password_resets” struktūra

Kolonna	Datu tips	Apraksts
password_resets_id	integer (PK)	Paroles atjaunošanas ieraksta identifikators
token_hash	varchar(255)	Šifrēts paroles atjaunošanas tokens
expires_at	datetime	Tokena derīguma beigu termiņš
created_at	datetime	Paroles atjaunošanas pieprasījuma izveides laiks
user_id	integer (FK)	Lietotājs, kuram pieprasīta paroles atjaunošana

5.11.tabula.

Tabulas “game_squad” struktūra

Kolonna	Datu tips	Apraksts
game_squad_id	integer (PK)	Unikāls ieraksta identifikators.
event_id	integer (FK)	Saite uz spēles notikumu (schedules.id).
user_id	integer (FK)	Saite uz spēlētāju, kurš iekļauts sastāvā (users.id).
selected_at	datetime	Laiks, kad spēlētājs tika pievienots sarakstam.

Relācijas:

- users.team_id → teams.id (1:N) - Lietotājs pieder vienai komandai, bet vienai komandai var būt vairāki lietotāji.
- teams.coach_id → users.id (1:1) - Viena komanda ir piesaistīta trenerim un viens treneris var būt atbildīgs par vienu komandu.
- schedules.team_id → teams.id (1:N) - Grafika ieraksts ir piesaistīts konkrētai komandai, bet vienai komandai var būt vairāki grafika ieraksti.
- player_stats.user_id → users.id (1:1) - Katram spēlētājam ir sava unikāla statistika un katrs statistikas ieraksts pieder tikai vienam spēlētājam.
- attendance.player_id → users.id (1:N) - Viens lietotājs var būt saistīts ar vairākiem apmeklētības ierakstiem, bet katrs apmeklētības ieraksts pieder vienam lietotājam.
- attendance.event_id → schedules.id (1:N) - Viens grafika notikums var būt saistīts ar vairākiem apmeklētības ierakstiem, bet katrs apmeklētības ieraksts pieder vienam notikumam.
- chat_rooms.team_id → teams.id (1:N) - Vienai komandai var būt vairākas čata istabas, bet katra čata istaba pieder vienai komandai.
- messages.room_id → chat_rooms.id (1:N) - Vienā čata istabā var būt vairākas ziņas, bet katra ziņa pieder vienai čata istabai.
- messages.user_id → users.id (1:N) - Viens lietotājs var nosūtīt vairākas ziņas, bet katra ziņa ir nosūtīta no viena lietotāja.
- direct_messages.sender_id → users.id (1:N) - Viens lietotājs var nosūtīt daudzas privātās ziņas, bet katrai privātajai ziņai ir viens sūtītājs.
- direct_messages.receiver_id → users.id (1:N) - Viens lietotājs var saņemt daudzas privātās ziņas, bet katrai privātajai ziņai ir viens saņēmējs.
- password_resets.user_id → users.id (1:N) - Vienam lietotājam var būt vairāki paroles atjaunošanas pieprasījumi, bet katrs pieprasījums pieder vienam lietotājam.

- `game_squad.event_id` → `schedules.id` (1:N) - Vienam spēles notikumam var būt vairāki sastāva ieraksti, bet katrs sastāva ieraksts pieder vienam spēles notikumam.
- `game_squad.user_id` → `users.id` (1:N) - Viens spēlētājs var figurēt vairāku spēļu sastāvos, bet katrs sastāva ieraksts attiecas uz vienu spēlētāju.
- `coach_join_requests.team_id` → `teams.id` (1:N) - Viena komanda var saņemt vairākus trenera pievienošanās pieprasījumus, bet katrs pieprasījums attiecas uz vienu komandu.
- `coach_join_requests.requester_user_id` → `users.id` (1:N) - Viens treneris var iesniegt vairākus pievienošanās pieprasījumus, bet katrs pieprasījums ir iesniegts no viena lietotāja.
- `coach_join_requests.reviewed_by` → `users.id` (1:N) - Viens galvenais treneris var izskatīt vairākus pievienošanās pieprasījumus, bet katru pieprasījumu izskata viens lietotājs.

6. LIETOTĀJA CEĻVEDIS

6.1. Sistēmas prasības aparatūrai un programmatūrai

Futbola komandu vadības sistēma “TeamFlow” ir tīmekļa lietotne, kas ir piemērota ērtai lietošanai lielākajā daļā ierīču, ja tās atbilst sekojošām prasībām:

- Operētājsistēma:
 - Windows 10 vai jaunāka versija,
 - macOS 10.15 vai jaunāka versija,
 - jebkura moderna Linux versija,
 - Android 9 vai jaunāka versija,
 - iOS 14 vai jaunāka versija.
- Interneta pārlūks:
 - Google Chrome 89 vai jaunāka versija,
 - Mozilla Firefox 86 vai jaunāka versija,
 - Safari 14 vai jaunāka versija,
 - Microsoft Edge 89 vai jaunāka versija.

Ir jāņem vērā, ka, lai ierīce spētu efektīvi darboties ar futbola komandu vadības sistēmu “TeamFlow” un nodrošinātu lietotājam ērtu darbu, izmantotajam pārlūkam jāatbalsta JavaScript. JavaScript ir nepieciešams lietotāja saskarnes darbībai, formu apstrādei, lapu pārslēgšanai un datu attēlošanai bez pilnas lapas pārlādes.

Papildus pārlūkprogrammai jāatbalsta lokālā krātuve un WebSocket savienojumi. Lokālā krātuve tiek izmantota lietotāja autorizācijas datu saglabāšanai pārlūkprogrammā, bet WebSocket savienojumi ir nepieciešami komandas tērēšanas un privāto ziņu darbībai reāllaikā.

6.2. Sistēmas instalācija un palaišana

Lai instalētu un palaistu sistēmu, lietotājam vispirms ir jāpārlicinās, ka uz izmantojamās ierīces ir instalēts nepieciešamais programmnodrošinājums:

- Visual Studio Code vai cits koda redaktors,
- Docker Desktop vai Docker Engine,
- Docker Compose,
- Git.

Ja sistēma tiek palaista bez Docker, papildus nepieciešams:

- Node.js,
- npm pakotņu pārvaldnieks.

Sistēmas lokālai palaišanai ieteicams izmantot Docker Compose, jo tas ļauj vienlaicīgi palaist gan servera daļu, gan klienta daļu. Projekta servera daļa atrodas mapē backend, savukārt lietotāja saskarnes daļa atrodas mapē frontend.

1. Projekta klonēšana no GitHub

Lai lejupielādētu projekta failus, jāatver terminālis un jāizpilda komanda:

git clone https://github.com/VladimirsAvgustin/Football-team-management-system.git

Pēc komandas izpildes datorā tiek izveidota projekta mape ar visiem nepieciešamajiem failiem.

Pēc tam jāieiet projekta mapē:

cd Football-team-management-system

2. Konfigurācijas faila sagatavošana

Projekta saknes mapē jāizveido .env fails no piemēra faila .env.example.

Windows PowerShell vidē jāizpilda komanda:

Copy-Item .env.example .env

Linux vai macOS terminālī jāizpilda komanda:

cp .env.example .env

.env failā jānorāda sistēmas konfigurācijas vērtības:

FRONTEND_URL=http://localhost:5173

JWT_SECRET=your_jwt_secret

EMAIL_USER=e-pasts, no kura tiks nosūtīti ziņojumi

EMAIL_PASS=e-pasta_SMTP_parole

Mainīgais FRONTEND_URL norāda klienta daļas adresi. Mainīgais JWT_SECRET tiek izmantots autorizācijas tokenu parakstīšanai. Mainīgie EMAIL_USER un EMAIL_PASS ir nepieciešami paroles atjaunošanas un kontaktformas e-pastu nosūtīšanai.

3. Sistēmas palaišana ar Docker Compose

Pirms sistēmas palaišanas jāpārlicinās, ka terminālis atrodas projekta saknes mapē, kur atrodas fails docker-compose.yml.

Sistēmu palaiž ar komandu:

docker compose up --build

Šī komanda izveido nepieciešamos Docker attēlus un palaiž divus konteinerus:

- backend konteineru, kurā darbojas Node.js/Express servera daļa,
- frontend konteineru, kurā darbojas Vue/Vite lietotāja saskarne.

Pirmajā palaišanas reizē process var aizņemt vairākas minūtes, jo tiek lejupielādēti nepieciešamie Docker attēli un instalētas projekta atkarības.

Pēc veiksmīgas palaišanas sistēma ir pieejama pārlūkprogrammā adresē:

<http://localhost:5173>

Servera API daļa darbojas adresē:

<http://localhost:3000>

Lietotājam ikdienas darbam ir jāatver tikai klienta daļas adrese <http://localhost:5173>.

4. Sistēmas apturēšana

Lai pilnībā apturētu un noņemtu palaistos konteinerus, jāizmanto komanda:

docker compose down

5. Alternatīva palaišana bez Docker

Ja Docker nav pieejams, sistēmu var palaist ar Node.js un npm. Šādā gadījumā servera un klienta daļas jāpalaiž atsevišķos termināļa logos.

Servera daļas palaišana:

cd backend

npm install

npm run dev

Klienta daļas palaišana:

cd frontend

npm install

npm run dev

Pēc abu daļu palaišanas sistēma ir pieejama adresē:

<http://localhost:5173>

Pašlaik sistēma nav izvietota publiskā tīmekļa domēnā, tāpēc pārbaudes laikā tā tiek palaista lokāli.

Lietotāju pieslēgšanas dati:

Administratora profilam var piekļūt, izmantojot e-pastu demo.admin@teamflow.lv un paroli Admin!123,

Trenera profilam var piekļūt, izmantojot e-pastu demo.coach@teamflow.lv un paroli Coach!123,

Spēlētāja profilam var piekļūt, izmantojot e-pastu demo.player@teamflow.lv un paroli Player!123.

Sistēma ir pieejama arī tiešsaistē.

Lietotāji var izmantot sistēmu bez lokālas instalēšanas, atverot to tīmekļa pārlūkprogrammā pēc saites - <https://TeamFlow.rvtdev.tech>

6.3. Programmas apraksts

Navigācijas josla

Navigācijas josla ir redzama sistēmas augšdaļā un sastāv no vairākām pogu grupām. Pirmā grupa satur komandas nosaukuma pogu, tēmas pārslēgšanas pogu, valodas pārslēgšanas pogu un publisko lapu saites. Attēlā redzamais teksts “TeamFlow” apzīmē komandas nosaukumu. Nospiežot komandas nosaukuma pogu, lietotājs pāriet uz sākumlapu vai komandas sākuma skatu. Tēmas poga pārslēdz gaišo un tumšo režīmu. Valodas poga pārslēdz saskarnes valodu. Poga “Sākums” atver sākumlapu, bet poga “Kontakti” atver kontaktu lapu (skat. 16. att.).



16. att. Navigācijas joslas publisko lapu un iestatījumu pogu grupa.

Nepierakstītam lietotājam navigācijas joslā tiek rādīta autentifikācijas pogu grupa. Poga “Reģistrēties” atver reģistrācijas lapu, kur lietotājs var izveidot jaunu kontu. Poga “Pierakstīties” atver pierakstīšanās modālo logu. Šī pogu grupa tiek rādīta tikai tad, ja lietotājs vēl nav autentificēts sistēmā (skat. 17. att.).



17. att. Nepierakstīta lietotāja autentifikācijas pogu grupa.

Lietotājam, kurš ir piesaistīts komandai, navigācijas joslā tiek rādīta komandas funkciju grupa. Poga ar komandas nosaukumu atver komandas paneli. Poga “Spēlētāji” atver komandas sastāva lapu. Poga “Statistika” atver komandas statistikas lapu un tiek rādīta trenerim. Poga “Grafiks” atver treniņu un spēļu grafiku. Poga “Čats” atver komandas čatu un tiešo ziņojumu sadaļu (skat. 18. att.).



18. att. Komandas funkciju pogu grupa.

Pierakstītam lietotājam navigācijas joslā tiek rādīta profila un sesijas pārvaldības grupa. Poga ar lietotāja vārdu atver profila lapu. Attēlā redzamais teksts “New Coach” apzīmē lietotāja vārdu. Poga “Izlogoties” beidz lietotāja sesiju, dzēš autentifikācijas datus no pārlūkprogrammas lokālās krātuves un pāradresē lietotāju uz sākulapam (skat. 19. att.).



19. att. Profila un izlogošanās pogas

Ja lietotājam vēl nav komandas, navigācijas joslā tiek rādīta komandas piesaistes darbību grupa. Poga “Izveidot komandu” atver komandas izveides lapu un ir paredzēta trenerim. Poga “Pievienoties komandai” atver modālo logu komandas koda ievadei. Spēlētājam šī darbība pievieno viņu komandai, bet trenerim nosūta pievienošanās pieprasījumu galvenajam trenerim (skat. 20. att.).



20. att. Komandas izveides un pievienošanās pogas

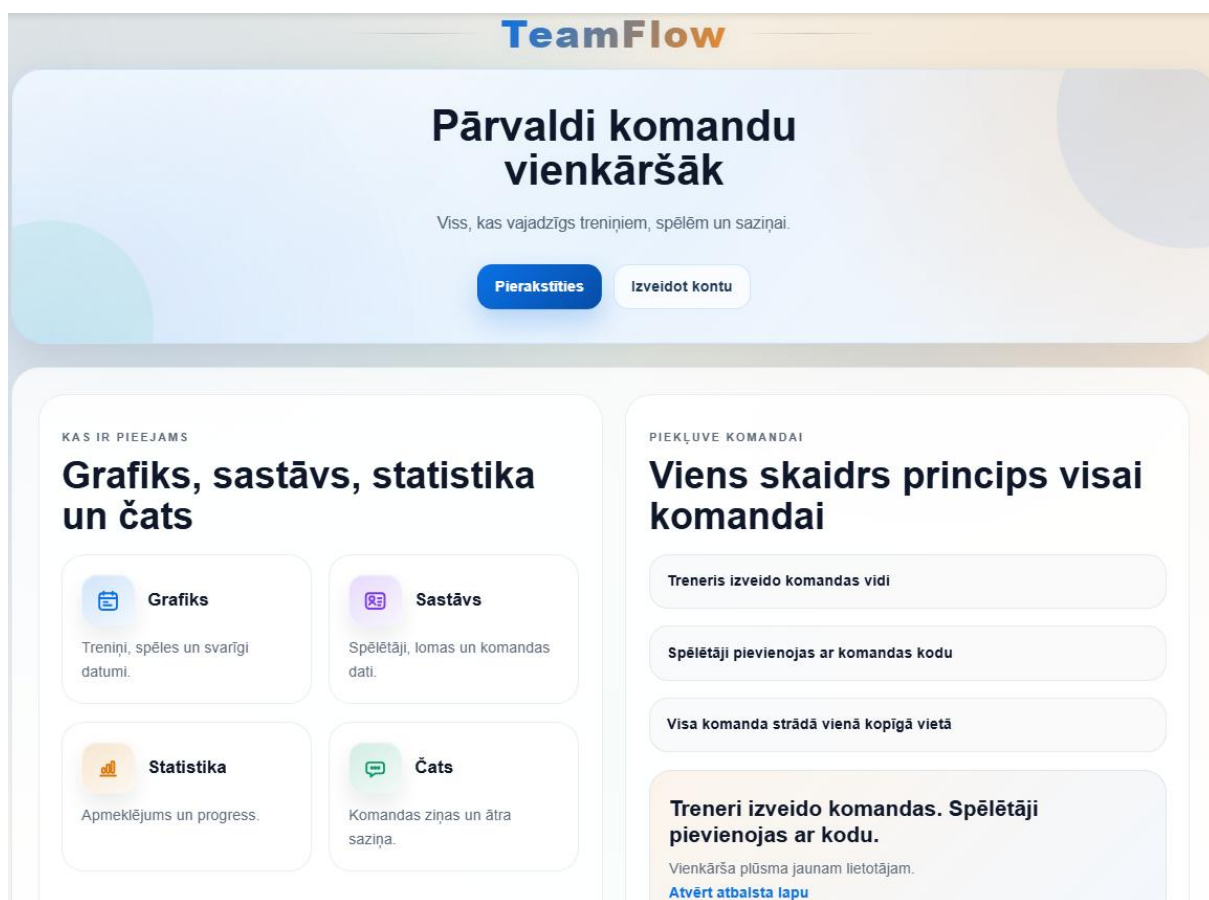
Administrators lomas lietotājam navigācijas joslā papildus tiek rādīta poga “Administrators”. Nospiežot šo pogu, tiek atvērta administrators darba vide, kur iespējams pārvaldīt lietotājus un komandas. Šī poga nav redzama spēlētājiem un treneriem bez administrators tiesībām (skat. 21. att.).



21. att. Administrators piekļuves poga

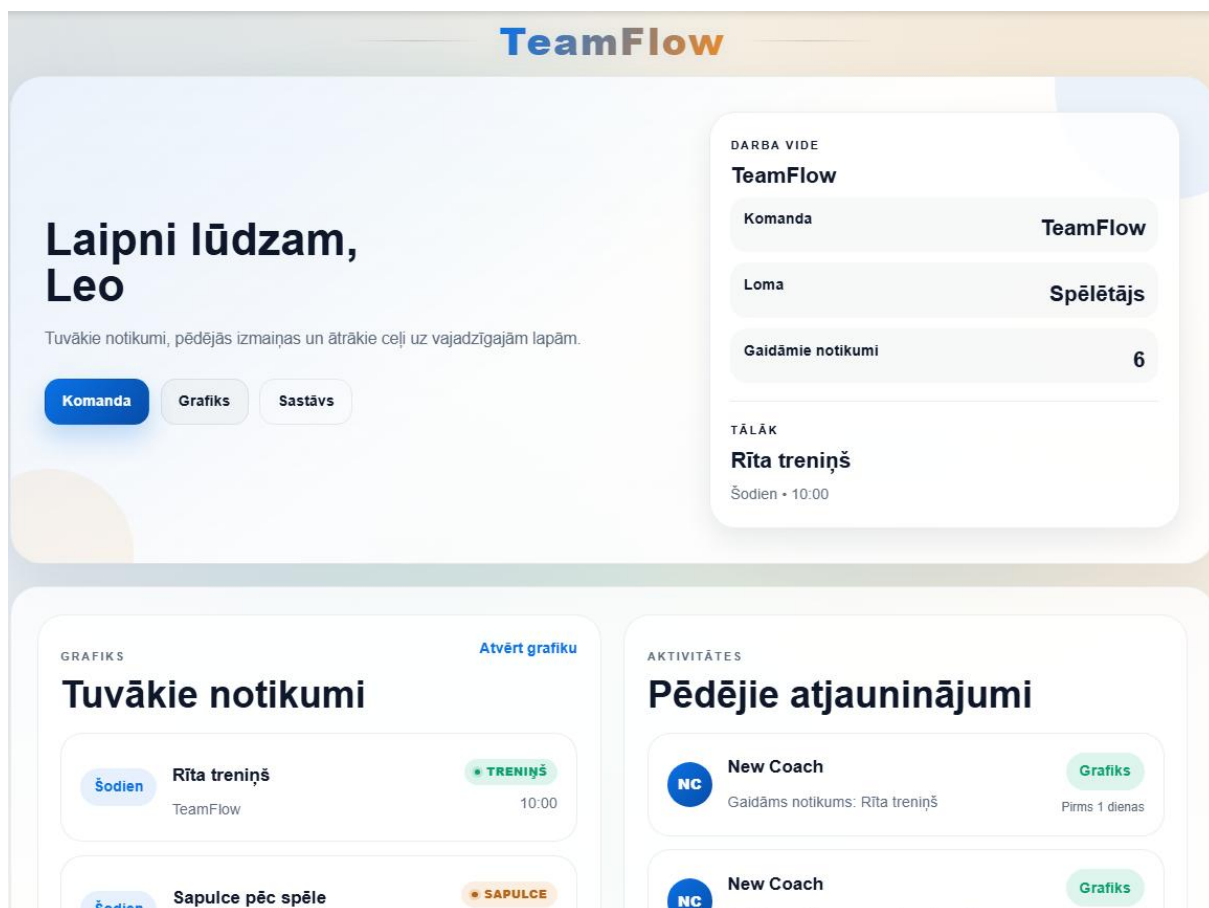
Sākulapa

Sākulapa ir sistēmas sākuma ekrāns. Nepierakstītam lietotājam tajā ir pieejama poga “Pierakstīties”, kas atver pierakstīšanās modālo logu, un poga “Izveidot kontu”, kas pāriet uz reģistrācijas lapu. Sākulapā redzami informācijas bloki paskaidro sistēmas galvenās iespējas: grafiku, sastāvu, statistiku un čatu (skat. 22. att.).



22. att. Sākulapa nepierakstītam lietotājam

Pēc pierakstīšanās sākumlapa kļūst par lietotāja darba paneli. Sistēma ielādē lietotāja profilu, komandas datus, tuvākos notikumus un nesenās aktivitātes. Lietotājs no šīs lapas var pāriet uz komandas lapu, grafiku, sastāvu (skat. 23. att.). Ja lietotājam vēl nav komandas, sākumlapa rāda darbības, kas palīdz izveidot komandu vai pievienoties tai.



23. att. Sākumlapa pierakstītam lietotājam

Pierakstīšanās

Pierakstīšanās logs (skat. 24. att.) tiek atvērts, nospiežot pogu “Pierakstīties”. Logā ir e-pasta ievades lauks, paroles ievades lauks, saite “Aizmirsu paroli?”, poga “Pierakstīties” un aizvēršanas poga. Lietotājam jāievada e-pasts un parole, pēc tam jānospiež poga “Pierakstīties”. Ja kāds lauks nav aizpildīts, sistēma parāda kļūdas paziņojumu. Ja dati tiek nosūtīti, autentifikācijas serviss pārbauda e-pastu un paroli. Veiksmīgas autentifikācijas gadījumā sistēma saglabā autentifikācijas tokens, ielādē lietotāja datus un aizver pierakstīšanās logu.

The image shows a mobile app login screen for 'TEAMFLOW'. At the top left is the 'TEAMFLOW' logo, and at the top right is a close button (X). The main title is 'Pierakstīties' (Login). Below it are two input fields: 'E-pasts' (Email) with the placeholder 'ievadiet savu e-pastu' and 'Parole' (Password) with the placeholder 'ievadiet paroli'. To the right of the password field is a link 'Aizmirsu paroli?'. At the bottom is a large blue button labeled 'Pierakstīties'.

24. att. Pierakstīšanās forma

Pierakstīšanās logu var aizvērt ar aizvēšanas pogu vai nospiežot ārpus loga. Saite “Aizmirsu paroli?” aizver pierakstīšanās logu un atver paroles atjaunošanas pieprasījuma lapu. Ja autentifikācijas serviss atgriež kļūdu, lietotājam jālabo ievadītie dati un darbība jāatkārto.

Reģistrācija

Reģistrācijas lapa tiek atvērta ar navigācijas pogu “Reģistrēties” vai sākumlapas pogu “Izveidot kontu”. Formā ir lauki “Vārds”, “Uzvārds”, “E-pasts”, “Parole”, “Komandas kods” un “Loma” (skat. 25. att.). Lietotājam jāaizpilda obligātie personas un konta dati, jāizvēlas loma un jānospiež poga “Reģistrēties”. Ja izvēlēta spēlētāja loma, lietotājs var ievadīt komandas kodu, lai reģistrācijas laikā tiktu piesaistīts komandai. Ja izvēlēta trenera loma, komandas koda lauks tiek atspējots, jo treneris komandu izveido vai piekļuvi pieprasa pēc reģistrācijas.

Reģistrēties

VĀRDS:

UZVĀRDS:

E-PASTS:

PAROLE:

KOMANDAS KODS (NAV OBLIGĀTS):

LOMA:

Izvēlieties lomu

Reģistrēties

25. att. Lietotāja reģistrācijas forma

Pēc pogas “Reģistrēties” nospiešanas reģistrācijas serviss pārbauda obligātos laukus, e-pasta unikalitāti, e-pasta formātu un to, vai parole satur vismaz 8 rakstzīmes. Ja reģistrācija ir veiksmīga, konts tiek izveidots, spēlētājam tiek sagatavots statistikas ieraksts, un lietotājs tiek pāradresēts uz sākumlapu. Ja serveris atgriež kļūdu, tā tiek parādīta zem formas, un lietotājam jāizlabo dati.

Paroles atjaunošana

Paroles atjaunošanas pieprasījuma lapā lietotājam jāievada sava konta e-pasts un jānospiež poga “Sūtīt atiestatīšanas saiti” (skat. 26. att.). Serviss pārbauda, vai konts eksistē, izveido vienreizēju atiestatīšanas tokens un nosūta e-pastu ar paroles maiņas saiti.

PAROLES ATJAUNOŠANA

Aizmirsu paroli

Ievadiet e-pastu, kas piesaistīts jūsu TeamFlow kontam, un mēs nosūtīsim paroles atiestatīšanas saiti.

E-PASTS

Ievadiet e-pastu

Sūtīt atiestatīšanas saiti

Atpakaļ uz sākumu

26. att. Paroles atjaunošanas pieprasījuma forma

Atverot saiti no e-pasta, lietotājs nonāk paroles atiestatīšanas lapā (skat. 27. att.). Šajā formā jāievada jaunā parole un tā jāapstiprina otrajā laukā. Pēc pogas “Saglabāt jauno paroli” nospiešanas sistēma pārbauda, vai saites tokens ir derīgs un vai abas paroles sakrīt. Ja pārbaude ir veiksmīga, parole tiek nomainīta un tokens vairs nav izmantojams.

KONTA DROŠĪBA

Atiestatīt paroli

Izveidojiet jaunu TeamFlow konta paroli. Atiestatīšanas saiti var izmantot tikai vienu reizi.

JAUNA PAROLE

Ievadiet jauno paroli **Rādīt**

APSTIPRINIET PAROLI

Atkārtojiet jauno paroli **Rādīt**

Saglabāt jauno paroli

Pieprasīt jaunu saiti

27. att. Paroles atiestatīšanas forma

Kontaktforma

Kontaktu lapā lietotājs var apskatīt atbalsta e-pastu, atrašanās vietu un nosūtīt ziņojumu sistēmas uzturētājam. Formā jāaizpilda lauki “Vārds”, “E-pasts”, jāizvēlas tēma un jāievada ziņojuma teksts (skat. 28. att.). Pēc pogas “Sūtīt ziņojumu” nospiešanas kontaktu serviss pārbauda, vai visi lauki ir aizpildīti, un nosūta e-pastu administratoram. Ja ziņojums ir nosūtīts, lapā parādās veiksmīgas nosūtīšanas paziņojums. Ja nosūtīšana neizdodas, lietotājam tiek parādīts kļūdas paziņojums.

The screenshot shows a contact form titled "SŪTIET MUMS ZIŅOJUMU" (Write to me a message). In the top right corner, there is a blue button that says "Mēs izlasām katru ziņojumu" (We read every message). The form contains several input fields: "VĀRDS" (Name) with the placeholder "Jūsu vārds", "E-PASTS" (Email) with the placeholder "jūsu@e-pasts.lv", and "TĒMA" (Topic) with a dropdown menu showing "Izvēlieties tēmu". Below these is a large text area for "ZIŅOJUMS" (Message) with the placeholder "Pastāstiet mums, kas gaitā...". At the bottom of the form is a prominent blue button labeled "Sūtīt ziņojumu" (Send message).

28. att. Kontaktu lapa un ziņojuma nosūtīšanas forma

Kontaktu lapā ir arī biežāk uzdoto jautājumu sadaļa (skat. 29. att.). Lietotājs nospiež jautājuma virsrakstu, lai atvērtu atbildi, un nospiež to vēlreiz, lai atbildi aizvērtu. Šī darbība nemaina servera datus un darbojas tikai lietotāja saskarnē.

BIEŽI UZDOTIE JAUTĀJUMI

Bieži uzdotie jautājumi

Kā man izveidot komandu? –

Reģistrējieties kā treneri, tad noklikšķiniet uz "Izveidot komandu" navigācijas joslā. Jums būs jāizvēlas nosaukums un unikāls komandas kods.

Kā spēlētāji pievienojas komandai? +

Vai treneris var pievienoties esošai komandai? +

Kā darbojas apmeklētības izsekošana? +

Kā atiestatīt paroli? +

Ko var darīt asistents-treneris? +

29. att. Kontaktu lapas biežāk uzdoto jautājumu sadaļa

Komandas izveide

Komandas izveides lapa ir pieejama trenerim, kurš vēl nav piesaistīts komandai (skat. 30. att.). Formā trenerim jāievada komandas nosaukums. Komandas kods tiek ģenerēts automātiski un tiek rādīts tikai lasāmā laukā. Ja treneris vēlas nomainīt ģenerēto kodu, viņš nospiež pogu "Ģenerēt no jauna". Lai izveidotu komandu, trenerim jānospiež poga "Izveidot jaunu komandu".

KOMANDAS IESTATĪŠANA

Izveidot jaunu komandu

Izveidojiet kopīgu darba vidi savai komandai un uzaiciniet spēlētājus ar ģenerēto komandas kodu.

01
Nosauciet komandu
Izmantojiet nosaukumu, ko spēlētāji lietotnē uzreiz atpazīs.

02
Kopīgojiet kodu
Spēlētāji pēc reģistrācijas varēs pievienoties sastāvam ar šo kodu.

TRENERA DARBA VIDE

Izveidot komandas centru

KOMANDAS NOSAUKUMS

KOMANDAS KODS (AUTOMĀTISKI ĢENERĒTS)

EC8UKK

Generēt jauns

Izveidot jaunu komandu

30. att. Komandas izveides forma

Pēc komandas izveides pogas nospiešanas komandas serviss pārbauda autentifikāciju, komandas nosaukumu un kodu. Ja dati ir derīgi, tiek izveidots komandas ieraksts, treneris tiek piesaistīts komandai, un sistēma automātiski izveido komandas čata telpu. Pēc veiksmīgas izveides lietotājs tiek pāradresēts uz sākumlapu.

Pievienošanās komandai

Poga “Pievienoties komandai” atver modālo logu ar komandas koda ievades lauku (skat. 31. att.). Spēlētājam jāievada komandas kods un jānospiež poga “Pievienoties”. Sistēma pārbauda kodu un, ja komanda ir atrasta, piesaista spēlētāju komandai. Modālo logu var aizvērt ar aizvēršanas pogu, pogu “Atcelt” vai nospiežot ārpus loga.

SPĒLĒTĀJA PIEKĻUVE

Pievienoties komandai

Ievadiet trenera nosūtīto kodu, lai piesaistītu kontu komandas darba videi.

KOMANDAS KODS

0/6

- - - - -

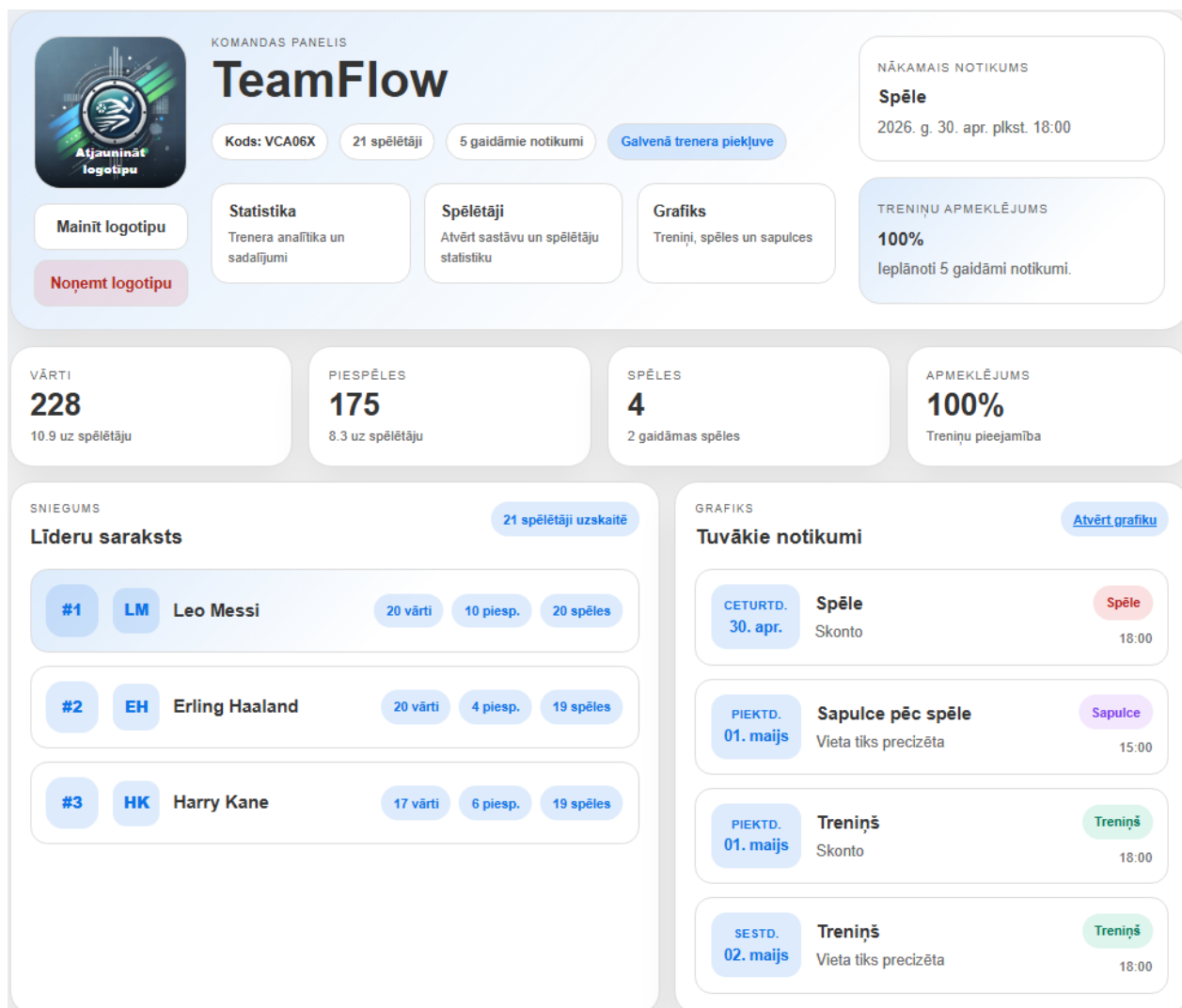
Atcelt Pievienoties komandai

31. att. Pievienošanās komandai forma

Ja pievienošanās logu izmanto treneris, darbība netiek pabeigta uzreiz. Sistēma nosūta pieprasījumu galvenajam trenerim. Pēc pieprasījuma nosūtīšanas galvenais treneris to var apstiprināt vai noraidīt komandas panelī (skat. 32. att.). Ja koda lauks ir tukšs vai kods nav derīgs, sistēma parāda kļūdu.

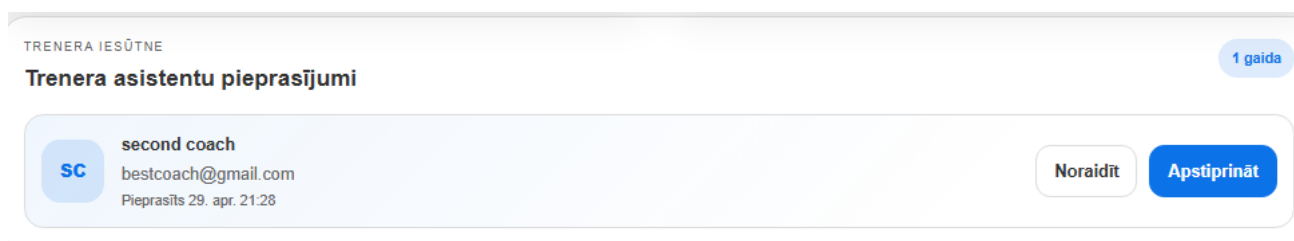
Komandas panelis

Komandas panelis rāda komandas nosaukumu, komandas kodu, spēlētāju skaitu, tuvākos notikumus un komandas kopsavilkuma rādītājus (skat. 33. att.). Lietotājs var izmantot ātrās saites, lai pārietu uz sastāvu, statistiku, grafiku vai čatu. Trenerim pieejama logotipa pārvaldība. Nospiežot uz logotipa zonas vai pogas “Mainīt logotipu”, tiek atvērta attēla faila izvēle. Sistēma pārbauda failu un saglabā logotipu komandai. Ja logotips ir pievienots, poga “Noņemt logotipu” to dzēš.



32. att. Komandas panelis

Galvenajam trenerim komandas panelī tiek rādīti treneru pievienošanās pieprasījumi (skat. 33. att.). Katram pieprasījumam ir poga “Apstiprināt” un poga “Noraidīt”. Nospiežot “Apstiprināt”, pieprasījuma iesniedzējs tiek piesaistīts komandai kā treneris. Nospiežot “Noraidīt”, pieprasījums tiek noraidīts un lietotājs komandai netiek piesaistīts.



33. att. Trenera pievienošanās pieprasījuma apstrāde

Lietotāja profils

Profila lapā lietotājs redz savu identitāti, lomu, komandu un profila attēlu (skat. 34. att.). Poga “Mainīt profila attēlu” atver faila izvēli. Lietotājam jāizvēlas attēla fails. Sistēma pārbauda faila tipu un izmēru, pēc tam saglabā attēlu lietotāja profilā. Ja attēls jau ir pievienots, poga “Noņemt profila attēlu” dzēš to no profila.

The screenshot shows a user profile for 'Leo Messi' with the role 'TeamFlow'. The profile includes a profile picture, a 'Spēlētājs' (Player) badge, and buttons for 'Mainīt profila attēlu' (Change profile picture) and 'Noņemt profila attēlu' (Remove profile picture). There is also an 'Izrakstīties' (Logout) button. Below the profile information, there is a section titled 'DROŠA INFORMĀCIJA' (Secure Information) with the heading 'Slēptie lietotāja dati' (Hidden user data). This section contains three items: 'E-PASTS' (Email) showing 'le*****@gmail.com' with a 'Parādīt' (Show) button; 'PAROLE' (Password) showing '*****' with a 'Mainīt paroli' (Change password) button and a note 'Paroli nevar apskatīt' (Cannot view password); and 'KOMANDAS KODS' (Team code) showing 'V****X' with a 'Parādīt' (Show) button. At the bottom, there is a section titled 'DALĪBA KOMANDĀ' (Team membership) with the heading 'Pamest pašreizējo komandu' (Leave current team) and a note 'Izejot no komandas, jūs tiksiet noņemts no sastāva un komandas čata, līdz pievienosities komandai vēlreiz.' (When leaving the team, you will be removed from the roster and team chat until you rejoin the team). There is a 'Pamest komandu' (Leave team) button.

34. att. Lietotāja profila lapa

Drošās informācijas sadaļā e-pasts un komandas kods sākotnēji tiek rādīti slēpti. Poga “Parādīt” atklāj attiecīgo vērtību, bet poga “Paslēpt” to atkal noslēpj. Poga “Mainīt paroli” nosūta paroles atjaunošanas e-pastu uz lietotāja konta adresi. Poga “Izrakstīties” izbeidz sesiju. Ja lietotājs ir komandā, sadaļā “Dalība komandā” ir poga “Pamest komandu”. Pirms komandas pamešanas sistēma prasa apstiprinājumu. Spēlētājs pēc apstiprinājuma tiek noņemts no komandas. Treneris zaudē piekļuvi komandai. Galvenais treneris var pamest komandu tikai tad, ja sistēma var nodot komandas īpašumtiesības citam trenerim. Ja treneris vēlas izslēgt savu komandu, viņam jāraksta kontaktforma.

Komandas sastāvs

Sastāva lapā redzami komandas spēlētāji(skāt. 35. att.). Meklēšanas lauks filtrē spēlētājus pēc ievadītā teksta. Kārtošanas izvēlne sakārto sarakstu pēc vārda, vārtiem, piespēlēm vai spēlēm. Skata pārslēdzējs maina rādījumu starp kartēm un tabulu. Navigācijas pogas lapas augšdaļā ļauj pāriet uz sastāvu, statistiku vai grafiku.

MEKLĒT Meklēt spēlētājus...

Pēc vārda Kartītes **Tabula**

Parādīti 21 no 21 spēlētājiem

SPĒLĒTĀJS	SPĒLES	VĀRTI	PIESPĒLES	DZELTENĀS	SARKANĀS	DARBĪBA
AH Achraf Hakimi team3.achraf.hakimi@example.com	23	5	11	4	1	Noņemt
AM Alvaro Morata team3.alvaro.morata@example.com	17	12	4	2	0	Noņemt
BS Bukayo Saka team3.bukayo.saka@example.com	17	9	11	2	0	Noņemt
EH Erling Haaland team3.erling.haaland@example.com	19	20	4	1	0	Noņemt
FV Federico Valverde team3.federico.valverde@example.com	21	6	8	4	0	Noņemt
FW Florian Wirtz team3.florian.wirtz@example.com	18	10	12	2	0	Noņemt
HK Harry Kane team3.harry.kane@example.com	19	17	6	1	0	Noņemt
JM Jamal Musiala team3.jamal.musiala@example.com	15	7	8	2	0	Noņemt
JB Jude Bellingham team3.jude.bellingham@example.com	20	11	8	4	0	Noņemt
KD Kevin DeBruyne team3.kevin.debruyne@example.com	17	6	14	3	0	Noņemt
KM Kylian Mbappe	18	16	7	2	0	Noņemt

35. att. Komandas sastāva lapa

Trenerim sastāva lapā ir pieejama spēlētāju datu pārvaldība. Spēlētāja statistikas laukos var mainīt spēļu, vārtu, piespēļu, dzelteni kartīšu un sarkano kartīšu skaitu. Vērtību var ievadīt tieši laukā vai mainīt ar plusa un mīnusa pogām. Pēc vērtības maiņas sistēma saglabā atjaunināto statistiku. Treneris var arī nomainīt vai noņemt spēlētāja profila attēlu. Galvenajam trenerim papildus ir pieejama poga spēlētāja noņemšanai no komandas. Pirms noņemšanas sistēma prasa apstiprinājumu(skāt. 35. att.).

Komandas statistika

Komandas statistikas lapa apkopo spēlētāju un komandas sniegumu(skat. 36. att.). Treneris tajā redz kopējos vārtus, piespēles, spēles, vidējo apmeklējumu, rezultatīvākos spēlētājus, slodzes sadalījumu un detalizētu statistikas tabulu. Poga “Eksportēt CSV” sagatavo tabulas datus CSV formātā. Poga “Eksportēt Excel” sagatavo datus izklājlapas formātā. Ja nav datu eksportēšanai, eksportēšanas pogas ir atspējotas.

DETALIZĒTA TABULA

Eksportēt CSV

Eksportēt Excel

21 rindas

Pilna spēlētāju statistika

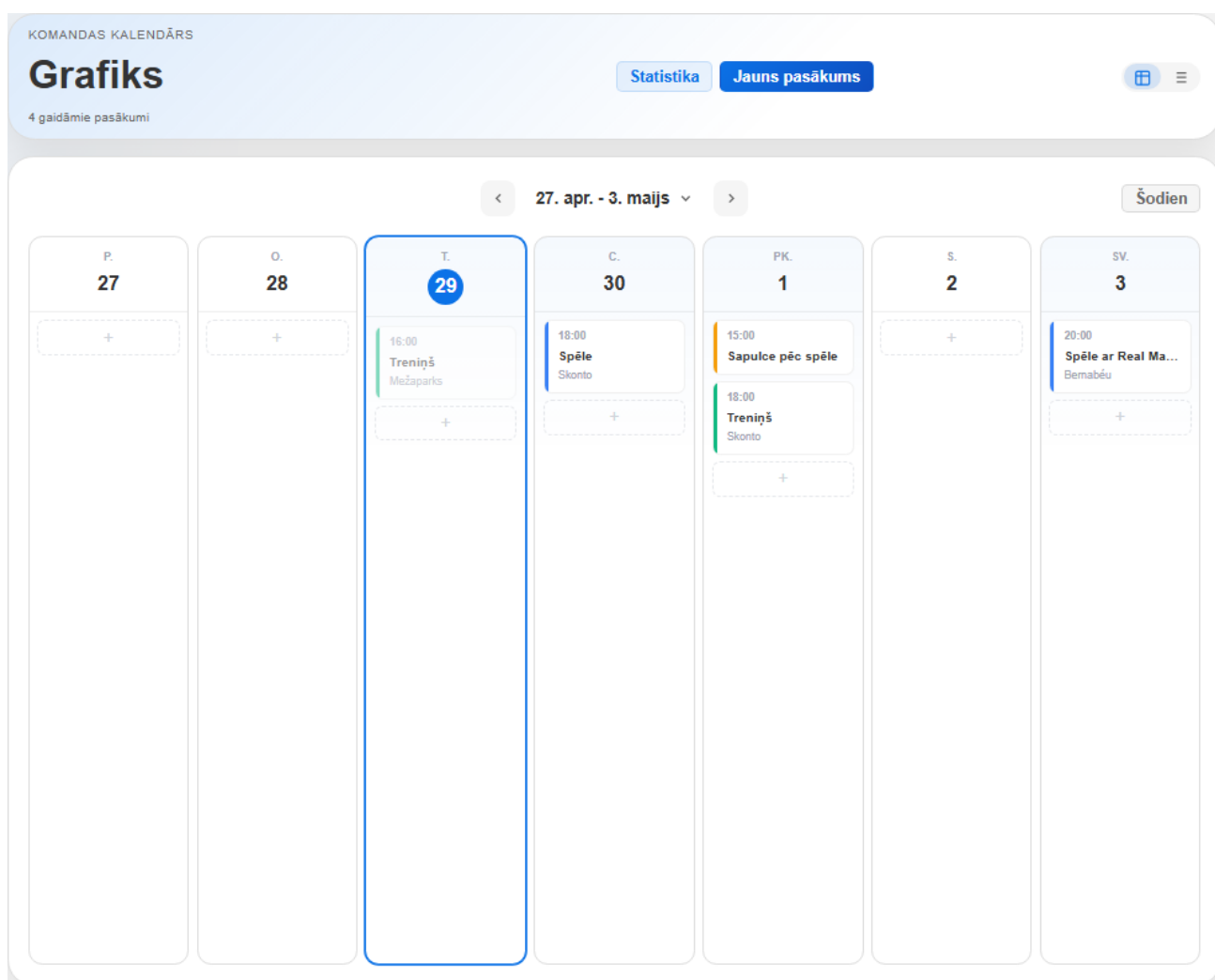
#	SPĒLĒTĀJS	SPĒLES	VĀRTI	PIESPĒLES	DZELTENĀS	SARKANĀS
1	 Leo Messi 20 spēles, 20 vārti, 10 piespēles	20	20	10	0	0
2	 Erling Haaland 19 spēles, 20 vārti, 4 piespēles	19	20	4	1	0
3	 Harry Kane 19 spēles, 17 vārti, 6 piespēles	19	17	6	1	0
4	 Kylian Mbappe 18 spēles, 16 vārti, 7 piespēles	18	16	7	2	0
5	 Robert Lewandowski 18 spēles, 16 vārti, 3 piespēles	18	16	3	1	0
6	 Lautaro Martinez 16 spēles, 15 vārti, 5 piespēles	16	15	5	3	0
7	 Mohamed Salah 18 spēles, 14 vārti, 10 piespēles	18	14	10	2	0
8	 Vinicius Junior 18 spēles, 13 vārti, 9 piespēles	18	13	9	3	0
9	 Alvaro Morata 17 spēles, 12 vārti, 4 piespēles	17	12	4	2	0
10	 Jude Bellingham 20 spēles, 11 vārti, 8 piespēles	20	11	8	4	0
11	 Florian Wirtz 18 spēles, 10 vārti, 12 piespēles	18	10	12	2	0
12	 Bukayo Saka 17 spēles, 9 vārti, 11 piespēles	17	9	11	2	0

36. att. Komandas statistikas lapa

Ja statistikas lapu atver spēlētājs, sistēma neļauj rediģēt vai eksportēt komandas analītiku. Spēlētājam tiek parādīta piekļuves informācija un saite atpakaļ uz spēlētāju sastāva lapu.

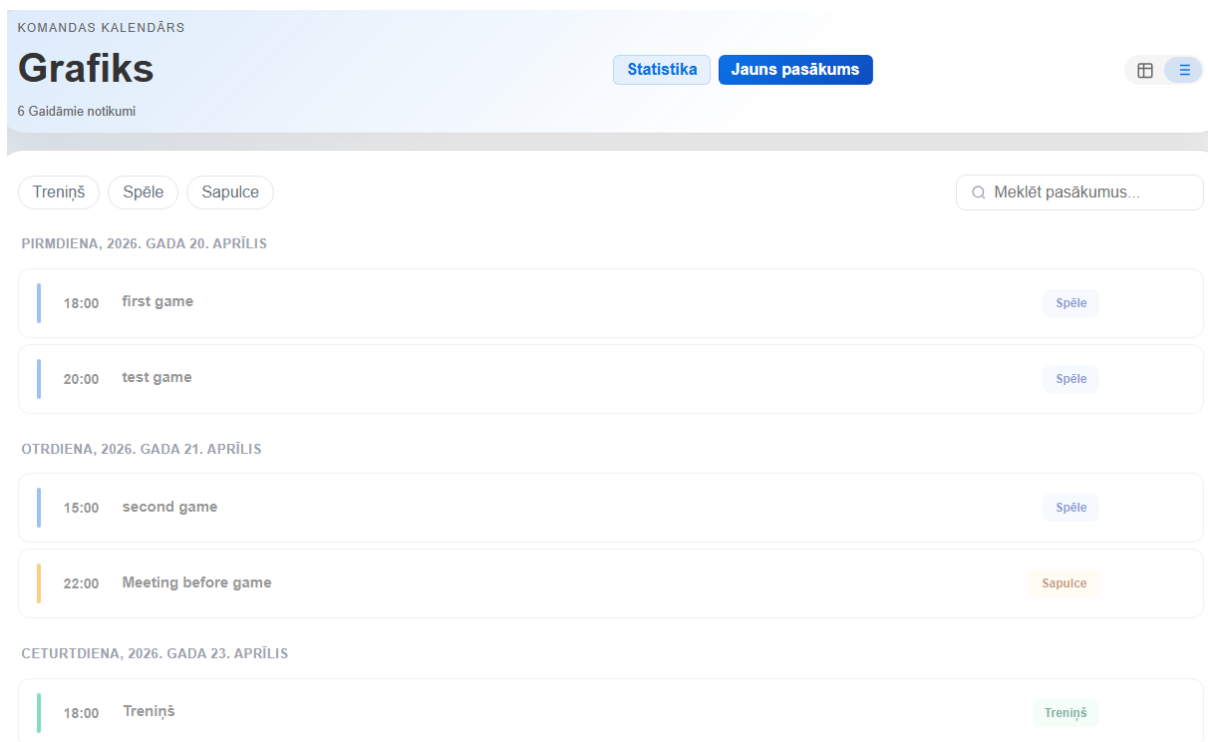
Grafiks

Grafika lapa rāda komandas notikumus(skat. 37. att.). Skata pārslēdzējs maina grafiku starp nedēļas skatu un saraksta skatu. Nedēļas skatā lietotājs var pāriet uz iepriekšējo vai nākamo nedēļu, atgriezties uz pašreizējo nedēļu ar pogu “Šodien” un atvērt datuma izvēlni, lai izvēlētos citu nedēļu.



37. att. Komandas grafika nedēļas skats

Saraksta skatā lietotājs var filtrēt notikumus pēc tipa un meklēt notikumus pēc ievadītā teksta. (skat. 38. att.).



38. att. Komandas grafika saraksta skats

Treneris var pievienot jaunu notikumu ar pogu “Jauns pasākums” vai nospiežot pievienošanas zonu konkrētajā dienā. Notikuma formā jāaizpilda nosaukums, datums, laiks, tips, vieta un apraksts (skat. 39. att.). Lai saglabātu jaunu notikumu, jānospiež poga “Pievienot”. Lai labotu esošu notikumu, treneris atver notikumu, nospiež “Rediģēt”, maina datus un nospiež “Saglabāt”. Lai dzēstu notikumu, treneris nospiež dzēšanas pogu un apstiprina darbību.

Pievienot notikumu

Virsraksts

Treniņš, Spēle, Cits

Datums

30.04.2026

Laiks

18:00

Tips

Treniņš

Vieta

Vieta?

Apraksts

Pievienojiet piezīmes, tikšanās vietu vai sagatavošanās detaļas

Atcelt **Pievienot**

39. att. Notikuma pievienošanas un rediģēšanas forma

Nospiežot uz notikuma, tiek atvērts notikuma detalizētais logs (skat. 40. att.). Tajā redzams notikuma tips, nosaukums, datums, laiks, vieta, apraksts un statuss. Ja notikums ir treniņš un vēl nav sācies, spēlētājs var atzīmēt savu apmeklējumu ar pogu “Būšu treniņā” (skat. 41. att.) vai izvēlēties “Nevaru atnākt”. Izvēloties neierašanos, tiek atvērts iemesla ievades logs (skat. 42. att.). Poga “Apstiprināt” saglabā neierašanās statusu un iemeslu, bet poga “Atcelt” aizver logu bez saglabāšanas.

SPĒLE

Spēle ar Real Madrid

DATUMS
svētdiena, 3 maijs 2026

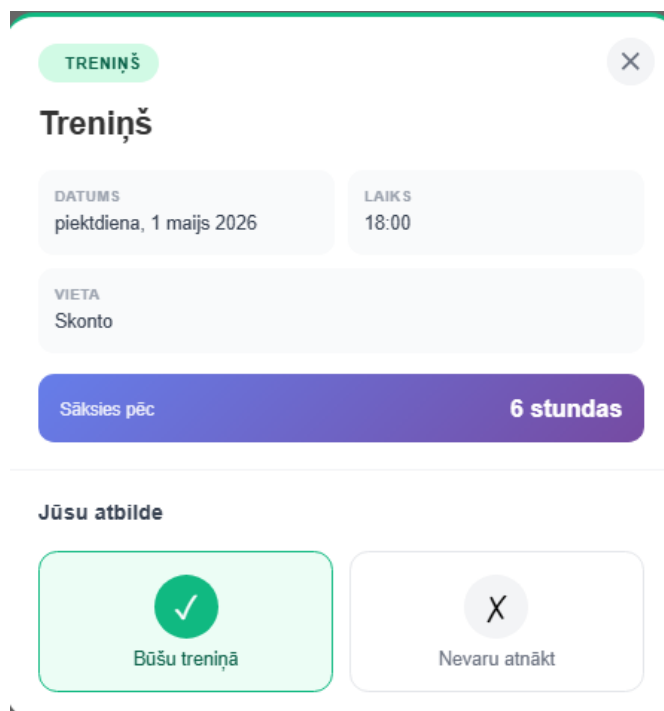
LAIKS
20:00

VIETA
Bernabéu

Sāksies pēc **2 dienas**

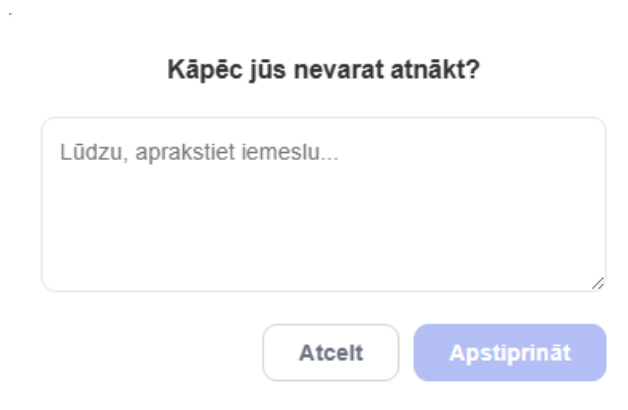
40. att. Notikuma detalizētais logs

Ja notikums ir treniņš un vēl nav sācies, spēlētājs var atzīmēt savu apmeklējumu ar pogu “Būšu treniņā” (skat. 41. att.) vai izvēlēties



41. att. Spēlētāja apmeklējuma atzīmēšana logs

“Nevaru atnākt”. Izvēloties neierašanos, tiek atvērts iemesla ievades logs (skat. 42. att.). Poga “Apstiprināt” saglabā neierašanās statusu un iemeslu, bet poga “Atcelt” aizver logu bez saglabāšanas.



42. att. Neierašanās iemesla ievades forma

Ja treniņa notikumu atver treneris, detalizētajā logā papildus tiek parādīts komandas spēlētāju saraksts un katra spēlētāja atzīmētais apmeklējuma statuss. Treneris var redzēt, kuri spēlētāji ir norādījuši, ka piedalīsies treniņā, kuri nevarēs ierasties, kā arī neierašanās iemeslus, ja tie ir ievadīti (skat. 43. att.).

TRENIŅŠ

Treniņš

DATUMS
piektdiena, 1 maijs 2026

LAIKS
18:00

VIETA
Skonto

Sāksies pēc: **6 stundas**

Komandas atbildes

1 1 20

✓ Apmeklē (1) ✗ Nevarēs ierasties (1) ? Nav atbildes (20)

- Leo Messi**
Apmeklē ✓
- Kylian Mbappe**
Attaisnots ✗
Stipras sāpes kājā
- Achraf Hakimi**
Nav atbildes ?
- Alvaro Morata**
Nav atbildes ?

Rediģēt Dzēst

43. att. Treniņa apmeklējuma pārskats trenerim

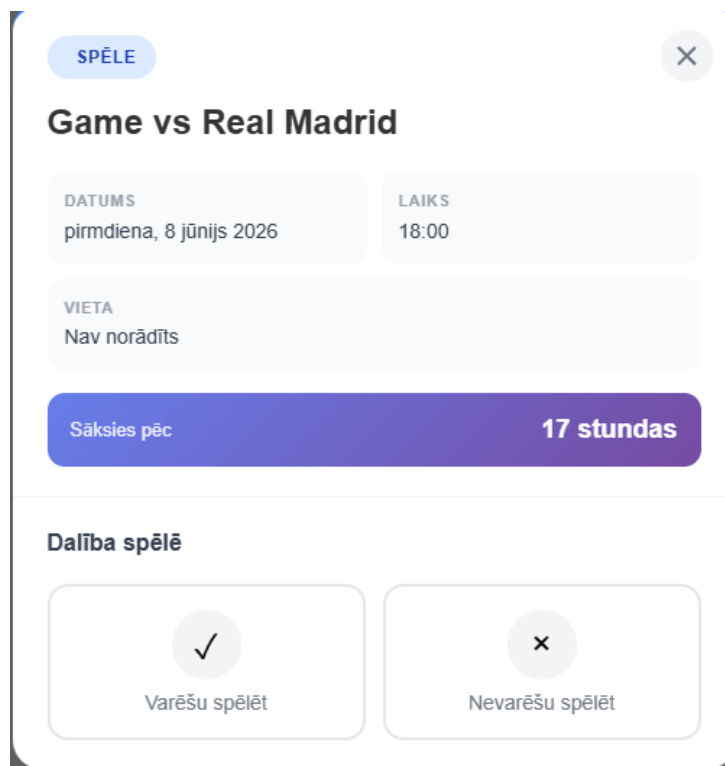
Ja notikums ir spēle, treneris detalizētajā logā var izveidot spēles sastāvu. Sadaļā “Sastāvs” tiek parādīts komandas spēlētāju saraksts, kurā treneris atzīmē spēlētājus, kuri būs iekļauti konkrētās spēles sastāvā. Lai saglabātu izvēlētos spēlētājus, treneris nospiež pogu “Saglabāt sastāvu” (skat. 44. att.). Ja nākamajai spēlei sastāvs vēl nav saglabāts, sistēma automātiski piedāvā iepriekšējās saglabātās spēles sastāvu, kuru treneris var mainīt pirms saglabāšanas.

The screenshot displays a mobile application interface for managing a football match. At the top, a blue header bar contains the word "SPĒLE" and a close button. Below this, the title "Game vs Real Madrid" is prominently displayed. The match details section includes the date "pirmdiena, 8 jūnijs 2026" and the time "18:00". A purple button indicates the match will start in "17 stundas". The "Sastāvs" (Squad) section shows "13 izvēlēti" (13 selected) players. A blue box contains a message: "Iepriekšējās spēles sastāvs ielādēts. Saglabāji to, lai izmantotu šai spēlei." Below this, a grid of player cards is shown, each with a checkbox, a circular icon with initials, and the player's name. The players are arranged in two columns. The first column includes Jude Bellingham (JB), Phil Foden (PF), Erling Haaland (EH), Rodri Hernandez (RH), and Harry Kane (HK). The second column includes Kevin DeBruyne (KD), Pedri Gonzalez (PG), Achraf Hakimi (AH), Vinicius Junior (VJ), and Robert Lewandowski (RL). The first four players in each column have their checkboxes checked, while Rodri Hernandez and Harry Kane have theirs unchecked.

Player	Selected
Jude Bellingham (JB)	Yes
Kevin DeBruyne (KD)	Yes
Phil Foden (PF)	Yes
Pedri Gonzalez (PG)	Yes
Erling Haaland (EH)	Yes
Achraf Hakimi (AH)	Yes
Rodri Hernandez (RH)	No
Vinicius Junior (VJ)	Yes
Harry Kane (HK)	No
Robert Lewandowski (RL)	Yes

44. att. Spēles sastāva izvēle trenerim

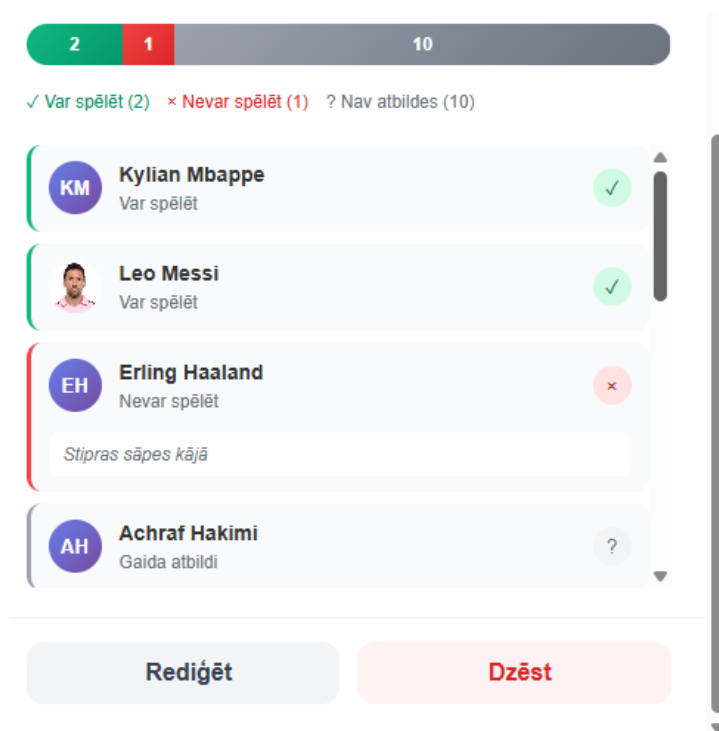
Pēc spēles sastāva saglabāšanas izvēlētie spēlētāji detalizētajā spēles logā var atzīmēt savu dalību spēlē. Spēlētājs var izvēlēties pogu “Varēšu spēlēt” vai “Nevarēšu spēlēt”. Ja spēlētājs izvēlas, ka nevarēs spēlēt, tiek atvērts iemesla ievades logs, kurā jānorāda atteikuma iemesls. Poga “Apstiprināt” saglabā spēlētāja atbildi un iemeslu, bet poga “Atcelt” aizver logu bez saglabāšanas (skat. 45. att.).



The screenshot shows a mobile application interface for a game titled "Game vs Real Madrid". At the top left is a blue button labeled "SPĒLE" and a close button with an "X" icon. Below the title, there are three information boxes: "DATUMS" (pirmidiena, 8 jūnijs 2026), "LAIKS" (18:00), and "VIETA" (Nav norādīts). A large purple button at the bottom of this section says "Sāksies pēc 17 stundas". Below this is a section titled "Dalība spēlē" containing two buttons: "Varēšu spēlēt" with a checkmark icon and "Nevarēšu spēlēt" with an "X" icon.

45. att. Spēlētāja dalības atzīmēšana spēlei

Treneris spēles detalizētajā logā redz saglabāto spēles sastāvu un spēlētāju atbildes. Pārskatā tiek parādīts, kuri spēlētāji var piedalīties spēlē, kuri nevarēs spēlēt un kuri vēl nav snieguši atbildi. Ja spēlētājs ir norādījis atteikuma iemeslu, tas tiek parādīts pie attiecīgā spēlētāja (skat. 46. att.).



46. att. Spēles sastāva un spēlētāju atbilžu pārskats trenerim

Lapas augšdaļā trenerim ir poga “Statistika”. To nospiežot, tiek atvērts apmeklējuma statistikas logs (skat. 47. att.). Tajā redzams spēlētāju skaits, treniņu skaits, vidējais apmeklējums un labākais apmeklējuma rādītājs. Meklēšanas lauks filtrē spēlētājus statistikā. Poga “Aizvērt” aizver statistikas logu.

Treniņu apmeklējuma statistika

22 no 22 spēlētājiem

x

SPĒLĒTĀJI

22

TRENIŅI

5

VIDĒJAIS PROCENTS

5%

LABĀKAIS PROCENTS

100%



Meklēt spēlētājus...

22 no 22 spēlētājiem

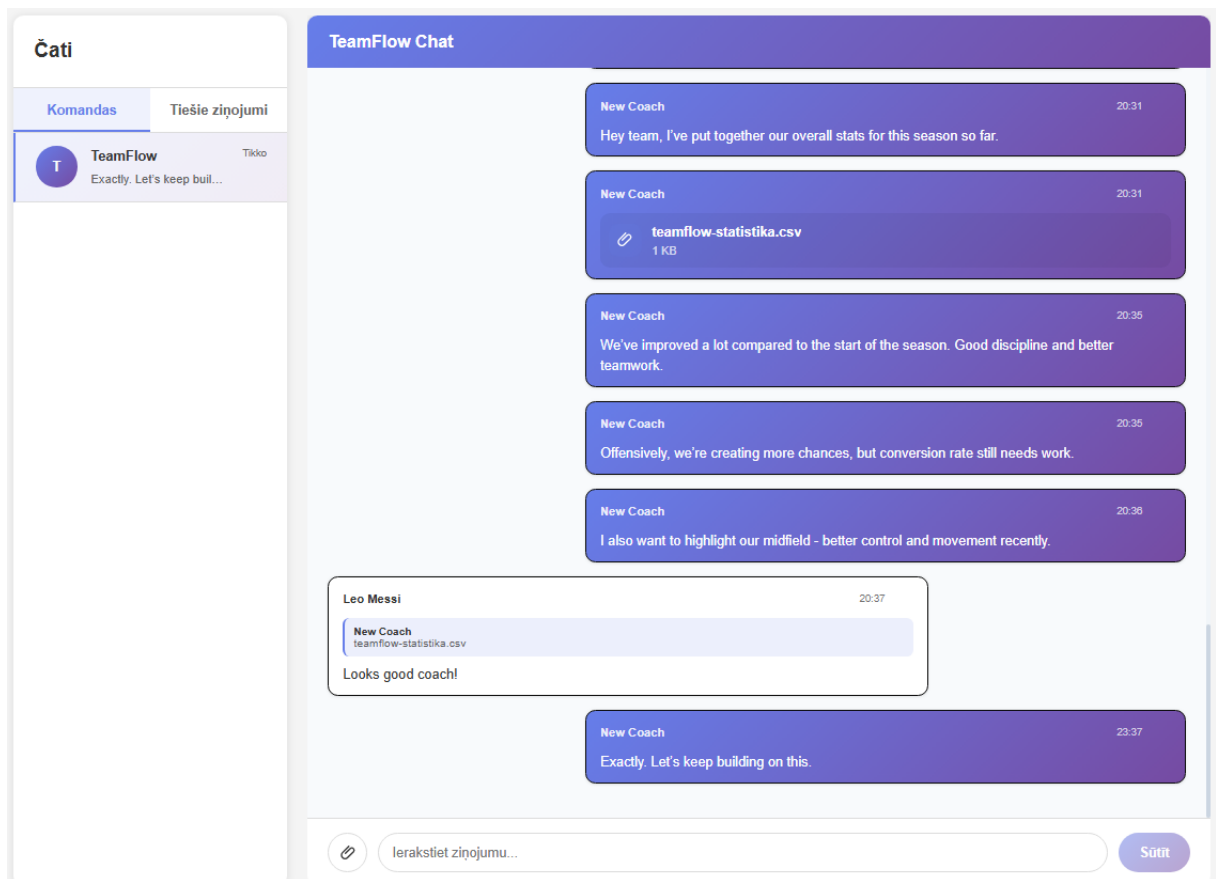
SPĒLĒTĀJS	APMEKLĒJA	IZLAIDA	NAV ATBILDES	PROCENTS
Leo Messi 5 / 5	5	0	0	100%
Kylan Mbappe 4 / 5	1	3	1	20%
Achraf Hakimi 0 / 5	0	0	5	0%
Alvaro Morata 0 / 5	0	0	5	0%
Bukayo Saka 0 / 5	0	0	5	0%
Erling Haaland 0 / 5	0	0	5	0%

Aizvērt

47. att. Treniņu apmeklējuma statistika

Komandas čats

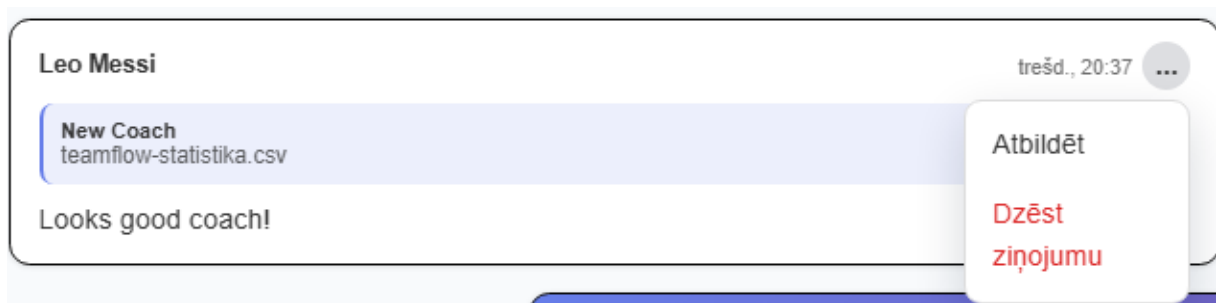
Čata lapā ir sānu josla ar čatu sarakstu un galvenā sarakstes zona. Mobilajā skatā sānu joslu atver ar pogu “Čati” un aizver ar aizvēršanas pogu vai nospiežot ārpus sānu joslas. Cilnē “Komandas” lietotājs izvēlas komandas čatu. Pēc čata izvēles galvenajā zonā parādās ziņojumi un ziņojuma ievades rinda. Lietotājs ieraksta tekstu ievades laukā un nospiež “Sūtīt” vai Enter taustiņu. (skat. 48. att.).



48. att. Komandas čats

Pielikuma poga atver faila izvēli. Pēc faila izvēles lietotājs var nosūtīt ziņojumu ar pielikumu vai noņemt izvēlēto failu ar pogu “Noņemt”. Sistēma pieņem atbalstītos failu tipus un pārbauda faila izmēru. Ziņojuma darbību izvēlne ļauj atbildēt uz ziņojumu. Ja ziņojuma autors ir pašreizējais lietotājs, viņš var arī dzēst savu ziņojumu. Ja čatā ir pielikums, lietotājs to var atvērt vai lejupielādēt, nospiežot uz pielikuma.

Novietojot kursoru uz ziņojuma, tiek parādīta papildu darbību poga ar trīs punktu ikonu. Nospiežot šo pogu, tiek atvērta izvēlne ar darbībām “Atbildēt” un “Dzēst ziņojumu” (skat. 49. att.). Jebkurš komandas čata lietotājs var atbildēt uz ziņojumu. Atbildes gadījumā virs ievades lauka tiek parādīts ziņojums, uz kuru lietotājs atbild, un pēc nosūtīšanas jaunais ziņojums tiek sasaistīts ar sākotnējo ziņojumu.

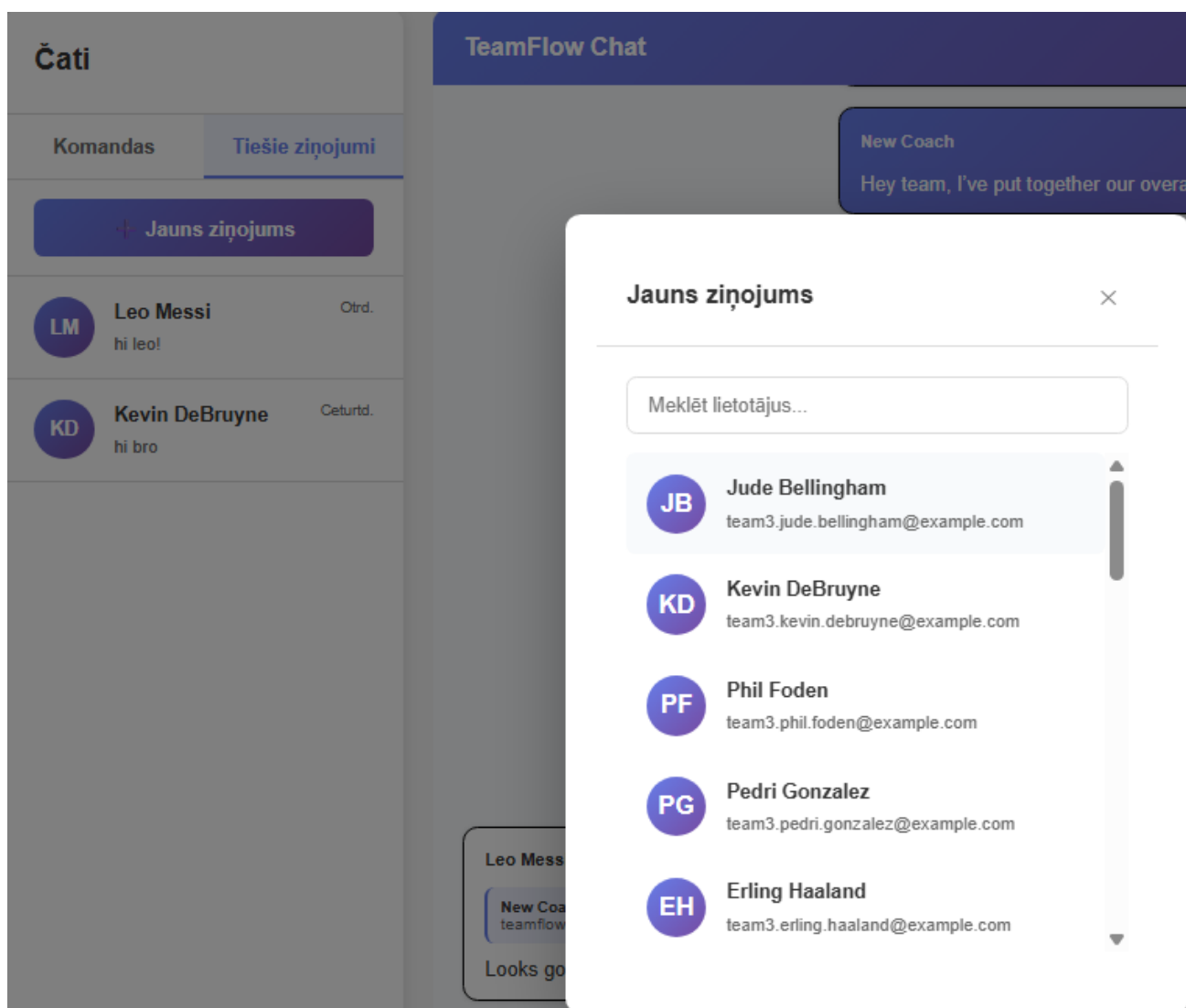


49. att. Čata ziņojuma darbību izvēlne

Dzēšanas tiesības ir atkarīgas no lietotāja lomas. Spēlētājs var dzēst tikai savus nosūtītos ziņojumus. Treneris var dzēst jebkuru ziņojumu komandas čatā, ja tas nepieciešams sarakstes pārvaldībai vai neatbilstoša satura noņemšanai.

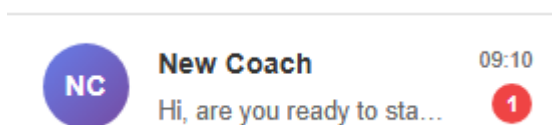
Tiešie ziņojumi

Tiešo ziņojumu cilnē lietotājs redz privāto sarakstu sarakstu (skat. 50. att.). Lai sāktu jaunu sarunu, jānospiež poga “Jauns ziņojums”. Atvērtajā logā jāizmanto lietotāju meklēšanas lauks un jāizvēlas adresāts no saraksta. Pēc adresāta izvēles logs aizveras un tiek atvērta privātā sarakste.



50. att. Tiešo ziņojumu sākšana

Privātajā sarakstē ziņojumu nosūtīšana darbojas tāpat kā komandas čātā: lietotājs ievada tekstu, pēc vajadzības pievieno failu un nospiež “Sūtīt”. Ja ziņojums vēl nav uzrakstīts un pielikums nav izvēlēts, sūtīšanas poga ir atspējota. Neizlasīto ziņojumu skaits tiek rādīts pie sarunas sānu joslā. (skat. 51. att.).



51. att. Neizlasīts ziņojums privātā sarakstē

Administrators lietotāju pārvaldība

Administrators lapa (skat. 52. att.) ir pieejama tikai lietotājam ar administrators lomu. Sadaļā “Lietotāji” administrators var meklēt lietotājus, filtrēt pēc lomas, rediģēt lietotāja vārdu, uzvārdu, e-pastu, lomu un komandas ID. Ja rindā tiek veiktas izmaiņas, sistēma parāda statusu “Jāsaglabā”. Poga “Saglabāt” nosūta izmaiņas uz administrators servisu. Poga “Atiestatīt” atceļ nesaglabātās izmaiņas konkrētajā rindā. Poga “Dzēst” dzēš lietotāju pēc apstiprinājuma. Pašreizējā administrators dzēšana un lomas maiņa ir bloķēta.

PĀRVALDĪBAS CENTRS

Administrators darba vide

34 lietotāji

5 komandas

29 piesaistīti komandai

0 nesaglabātas izmaiņas

PIESLEDZIES KĀ

Main Admin

Šo darba vidi var atvērt tikai lietotāji ar administrators lomu.

KOPĀ KONTI

34

Visi reģistrētie lietotāji

KOMANDAS

5

Reģistrētie klubi un komandas

ADMINISTRATORI

2

Lietotāji ar pilnu piekļuvi

TRENERI

9

Komandu vadītāji un personāls

SPĒLĒTĀJI

23

Sastāva dalībnieki

PĀRVALDĪBAS TABULAS

Lietotāju tabula

Lietotāji

Komandas

Atjaunot

VĀRDS / E-PASTS

LOMA

Meklēt pēc vārda vai e-pasta adreses...

Visas lomas

LIETOTĀJS	VĀRDS	UZVārds	E-PASTS	LOMA	KOMANDAS ID	STATUSS	DARBĪBAS
TU Test User #1	Test	User	test@test.com	Administrators	Nav obligāts	ATJAUNINĀTS	Atiestatīt Saglabāt Dzēst
FC first coach #2	first	coach	firstcoach@gmail.com	Treners	1	ATJAUNINĀTS	Atiestatīt Saglabāt Dzēst
FP first player #3	first	player	firstplayer@gmail.com	Spēlētājs	2	ATJAUNINĀTS	Atiestatīt Saglabāt Dzēst
VA vladimirs avgustins #4	vladimirs	avgustins	vladimiravgustin123@g	Treners	2	ATJAUNINĀTS	Atiestatīt Saglabāt Dzēst

52. att. Administrators lietotāju pārvaldība

Lietotāju tabulas apakšā ir lapošanas zona. Poga “Iepriekšējā” atver iepriekšējo lapu, lapas numura poga atver konkrēto lapu, bet poga “Nākamā” atver nākamo lapu. Poga “Atjaunot” atkārtoti ielādē aktīvās sadaļas datus no servera.

Administratora komandu pārvaldība

Sadaļā “Komandas” administrators var meklēt komandas pēc nosaukuma, koda vai trenera. Tabulā redzams komandas nosaukums, kods, galvenais treneris, dalībnieku skaits un spēlētāju skaits (skat. 53. att.). Administrators var rediģēt komandas nosaukumu un komandas kodu. Poga “Saglabāt” nosūta izmaiņas uz komandu administrēšanas servisu, bet poga “Atiestatīt” atjauno rindas iepriekšējo stāvokli. Ja nosaukums vai kods nav derīgs, sistēma parāda kļūdu.

PĀRVALDĪBAS TABULAS

Komandu tabula

KOMANDAS NOSAUKUMS / KOMANDAS KODS

Meklēt pēc komandas nosaukuma, koda vai trenera...

KOMANDA	KOMANDAS NOSAUKUMS	KOMANDAS KODS	GALVENAIS TRENERIS	DALĪBNIKĒKI	SPĒLĒTĀJI	STATUSS	DARBĪBAS
1 123123 #2	123123	HDEGND	vladimirs avgustins	2	1	ATJAUNINĀTS	Atiestatīt Saglabāt
B Barcelona #5	Barcelona	SPL1D1	Jauns Speletajs	1	0	ATJAUNINĀTS	Atiestatīt Saglabāt
RM Real Madrid #4	Real Madrid	BA5MUD	Cristiano Ronaldo	2	0	ATJAUNINĀTS	Atiestatīt Saglabāt
T TeamFlow #1	TeamFlow	GWOM7J	first coach	1	0	ATJAUNINĀTS	Atiestatīt Saglabāt
T TeamFlow #3	TeamFlow	VCA06X	New Coach	23	21	ATJAUNINĀTS	Atiestatīt Saglabāt

53. att. Administratora komandu pārvaldība

Administrators var pārslēgties starp lietotāju un komandu sadaļām ar tabulu pārslēgšanas pogām “Lietotāji” un “Komandas”. Kopsavilkuma kartītes lapas augšdaļā atjauninās atbilstoši ielādētajiem lietotāju un komandu datiem.

7. PROGRAMMATŪRAS LIETOJAMĪBAS TESTĒŠANA

Testēšanas mērķis ir pārbaudīt, vai galvenās funkcionālās prasības strādā atbilstoši sagaidāmajam rezultātam.

7.1.tabula

Lietotāja reģistrācijas testēšana

Lietotāja loma:	viesis	
Prasības apraksts:		
Sistēmai jānodrošina jauna lietotāja konta izveide ar obligāto lauku pārbaudi un unikālu e-pastu		
Ievadītā/situācijas apraksts	Sagaidāmais rezultāts	Statuss
1. Ievada korektus datus: vārds, uzvārds, unikāls e-pasts, parole	Konts tiek veiksmīgi izveidots	pareizi
2. Atstāj tukšu obligāto lauku (piem., parole)	Konts netiek izveidots, kļūda par obligāto lauku	pareizi
3. Ievada jau reģistrētu e-pastu	Konts netiek izveidots, kļūda par dublikātu e-pastu	pareizi
4. Ievada nekorektu e-pasta formātu (piem., testmail.com)	Konts netiek izveidots, kļūda par nekorektu e-pastu	pareizi

7.2.tabula

Pievienošanās komandai

Lietotāja loma:	spēlētājs	
Prasības apraksts:		
Sistēmai jānodrošina lietotāja pievienošana komandai, ievadot derīgu komandas kodu.		
Ievadītā/situācijas apraksts	Sagaidāmais rezultāts	Statuss
1. Lietotājs ievada derīgu komandas kodu un nav piesaistīts citai komandai	Lietotājs veiksmīgi pievienojas komandai	pareizi
2. Lietotājs ievada nepareizu komandas kodu	Komandai pievienoties neizdodas, tiek parādīts paziņojums, ka komanda nav atrasta	pareizi
3. Lietotājs ievada tukšu komandas kodu	Pievienošanās netiek veikta, tiek parādīts paziņojums par obligātu lauka aizpildi	pareizi

Pasākumu grafika pārvaldība

Lietotāja loma:	treneris	
Prasības apraksts:		
Sistēmai jānodrošina komandas pasākumu pievienošana un rediģēšana, nepieļaujot laika konfliktus.		
Ievadītā/situācijas apraksts	Sagaidāmais rezultāts	Statuss
1. Treneris pievieno jaunu pasākumu ar korektiem datiem	Pasākums tiek veiksmīgi saglabāts grafikā	pareizi
2. Treneris mēģina pievienot pasākumu tajā pašā datumā un laikā, kur jau ir cits pasākums	Saglabāšana tiek noraidīta, parādās paziņojums par laika konfliktu	pareizi
3. Treneris iesniedz formu bez obligātajiem laukiem (piem., nav nosaukums vai datums)	Pasākums netiek izveidots, parādās kļūda par obligāto lauku aizpildi	pareizi

Paroles atjaunošana

Lietotāja loma:	jebkurš lietotājs	
Prasības apraksts:		
Sistēmai jānodrošina paroles atjaunošana ar e-pasta tokens un jaunās paroles validāciju.		
Ievadītā/situācijas apraksts	Sagaidāmais rezultāts	Statuss
1. Lietotājs ievada reģistrētu e-pastu paroles atjaunošanas formā	Sistēma pieņem pieprasījumu un nosūta paroles atjaunošanas ziņojumu uz e-pastu	pareizi
2. Lietotājs iesniedz tukšu e-pasta lauku	Pieprasījums netiek apstrādāts, tiek parādīts paziņojums par obligātu e-pasta ievadi	pareizi
3. Lietotājs atver atjaunošanas saiti ar derīgu tokenu un ievada jaunu paroli	Parole tiek veiksmīgi nomainīta, lietotājs saņem apstiprinājumu	pareizi

NOBEIGUMS

Kvalifikācijas darba izstrādes laikā tika sasniegts izvirzītais mērķis - izstrādāta tīmekļa lietojumprogramma "TeamFlow" futbola komandas ikdienas darba organizēšanai. Sistēmā realizēta lietotāju reģistrācija un pieslēgšanās, paroles atjaunošana, komandas izveide un pievienošanās komandai ar kodu, trenera pievienošanās pieprasījumu apstrāde, spēlētāju saraksta un profilu pārvaldība, grafika uzturēšana, apmeklētības atzīmēšana, spēlētāju statistikas pārvaldība un eksports CSV/XLS formātā. Papildus izveidota komandas tērēšana, privātās ziņas, pielikumu augšupielāde, kontaktforma un administratora panelis lietotāju un komandu pamatdatu pārvaldībai. Lietojumprogramma ir izstrādāta kā klienta-servera sistēma: klienta daļa veidota ar Vue.js, bet servera daļa - ar Node.js un Express, datu glabāšanai izmantojot SQLite datubāzi. Saskaņā ir pieejama latviešu un angļu valodā, un sistēmā ir ieviesta lomu balstīta piekļuves kontrole, kas nodrošina atšķirīgas iespējas spēlētājam, trenerim, galvenajam trenerim un administratoram. Izstrādātā sistēma darbojas stabili lokālā vidē, nodrošina galveno lietošanas scenāriju izpildi un atbilst kvalifikācijas darba sākumā noteiktajām funkcionālajām prasībām.

Pašlaik sistēma tiek izmantota kā kvalifikācijas darba ietvaros izstrādāts mācību projekts un ir piemērota lokālai palaišanai, funkcionalitātes pārbaudei un demonstrēšanai. Izstrādes laikā tika realizētas visas galvenās funkcijas, kas sākotnēji tika plānotas šim projektam, tāpēc sistēma jau nodrošina pilnvērtīgu pamatu futbola komandas darba organizēšanai. Turpmāk vispiemērotākais projekta attīstības virziens būtu sadarbība ar reālu futbola klubu vai komandu, kas sistēmu izmantotu praksē un sniegtu konkrētas prasības par papildu funkcijām. Šāda sadarbība ļautu attīstīt projektu nevis teorētiski, bet balstoties uz reālām lietotāju vajadzībām un ikdienas darba pieredzi.

INFORMĀCIJAS AVOTI

1. Vue.js. Documentation - <https://vuejs.org/guide/introduction.html> - (Resurss apskatīts 19.04.2025.)
2. Vite. Guide - <https://vite.dev/guide/> - (Resurss apskatīts 19.04.2025.)
3. Pinia. Documentation - <https://pinia.vuejs.org/> - (Resurss apskatīts 19.04.2025.)
4. Vue Router. Documentation - <https://router.vuejs.org/> - (Resurss apskatīts 19.04.2025.)
5. Vue I18n. Documentation - <https://vue-i18n.intlify.dev/> - (Resurss apskatīts 31.05.2024.)
6. Axios. Documentation - <https://axios-http.com/docs/intro> - (Resurss apskatīts 19.04.2025.)
7. Chart.js. Documentation - <https://www.chartjs.org/docs/latest/> - (Resurss apskatīts 19.04.2025.)
8. Day.js. Documentation - <https://day.js.org/docs/en/installation/installation> - (Resurss apskatīts 09.05.2025.)
9. Express. Documentation - <https://expressjs.com/> - (Resurss apskatīts 19.04.2025.)
10. SQLite. Documentation - <https://www.sqlite.org/docs.html> - (Resurss apskatīts 24.04.2025.)
11. Socket.IO. Documentation - <https://socket.io/docs/v4/> - (Resurss apskatīts 19.04.2025.)
12. jsonwebtoken. npm package documentation - <https://www.npmjs.com/package/jsonwebtoken> - (Resurss apskatīts 19.04.2025.)
13. bcrypt. npm package documentation - <https://www.npmjs.com/package/bcrypt> - (Resurss apskatīts 19.04.2025.)
14. Nodemailer. Documentation - <https://nodemailer.com/about/> - (Resurss apskatīts 17.02.2026.)

PIELIKUMI

1. pielikums

Programmas pirmteksts

Šajā pielikumā apkopoti sporta komandu pārvaldības sistēmas “TeamFlow” svarīgākie pirmkoda fragmenti.

server.js

```
/**
 * Šis fails ir TeamFlow servera daļas galvenais fails.
 * Tas inicializē Express serveri, SQLite datubāzi, Socket.IO savienojumus,
 * API maršrutus un kontaktformas e-pasta nosūtīšanu.
 * Galvenās funkcijas:
 * - corsOriginCheck(): pārbauda atļautos klienta domēnus.
 * - Datubāzes inicializācija: izveido galvenās sistēmas tabulas.
 * - app.post('/api/contact'): nosūta kontaktformas ziņojumu.
 * - io.use(): pārbauda Socket.IO JWT tokens.
 * - io.on('connection'): apstrādā čata un tiešo ziņojumu notikumus.
 * Izmantotās bibliotēkas: express, cors, sqlite3, socket.io, jsonwebtoken,
 * nodemailer, dotenv.
 * Izmantotie maršruti: auth.js, schedules.js, admin.js, chat.js.
 */
const path = require('path');
require('dotenv').config({ path: path.join(__dirname, '.env') });
const express = require('express');
const cors = require('cors');
const bodyParser = require('body-parser');
const sqlite3 = require('sqlite3').verbose();
const { Server } = require('socket.io');
const http = require('http');
const jwt = require('jsonwebtoken');
const fs = require('fs');

const app = express();
const server = http.createServer(app);
const PORT = 3000;
```

```

const HOST = '0.0.0.0';

const uploadsDir = path.join(__dirname, 'uploads', 'logos');
if (!fs.existsSync(uploadsDir)) fs.mkdirSync(uploadsDir, { recursive: true
});

const chatUploadsDir = path.join(__dirname, 'uploads', 'chat');
if (!fs.existsSync(chatUploadsDir)) fs.mkdirSync(chatUploadsDir, {
recursive: true });

/**
 * Pārbauda, vai klienta adrese drīkst piekļūt API.
 *
 * @param {string} origin - Klienta pieprasījuma avots.
 * @param {Function} callback - CORS pārbaudes rezultāts.
 */
const corsOriginCheck = (origin, callback) => {
  const allowedOrigins = [
    'http://localhost:5173',
    'http://localhost:5174',
    'https://localhost:5173',
    'https://localhost:5174',
    'http://127.0.0.1:5173',
    'http://127.0.0.1:5174',
    'https://127.0.0.1:5173',
    'https://127.0.0.1:5174'
  ];

  const isLocalNetworkIP = /^https?:\/\/(192\.168\.|10\.|172\. (1[6-9]|2[0-
9]|3[01])|127\.)/;

  if (!origin || allowedOrigins.includes(origin) ||
isLocalNetworkIP.test(origin)) {
    callback(null, true);
  } else {
    callback(new Error('CORS piekļuve nav atļauta'));
  }
};

const io = new Server(server, {
  cors: { origin: corsOriginCheck, methods: ['GET', 'POST'], credentials:
true }
});

app.use(cors({

```

```

    origin: corsOriginCheck,
    methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
    credentials: true
  }));
app.use(bodyParser.json({ limit: '10mb' }));
app.use(bodyParser.urlencoded({ limit: '10mb', extended: true }));

/**
 * Inicializē SQLite datubāzi un izveido galvenās tabulas, ja tās vēl
 * neeksistē.
 */
const dbPath = path.resolve(__dirname, 'database.sqlite');
const db = new sqlite3.Database(dbPath, (err) => {
  if (err) {
    console.error('Database connection error:', err.message);
  } else {
    db.run('PRAGMA foreign_keys = ON');
    db.serialize(() => {
      db.run(`CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        surname TEXT NOT NULL,
        email TEXT NOT NULL UNIQUE,
        password TEXT NOT NULL,
        role TEXT NOT NULL DEFAULT 'user',
        avatar TEXT,
        team_id INTEGER,
        createdAt DATETIME DEFAULT CURRENT_TIMESTAMP,
        updatedAt DATETIME DEFAULT CURRENT_TIMESTAMP
      )`);
      db.run(`CREATE TABLE IF NOT EXISTS teams (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        team_code TEXT,
        coach_id INTEGER,
        logo TEXT,
        createdAt DATETIME DEFAULT CURRENT_TIMESTAMP,
        updatedAt DATETIME DEFAULT CURRENT_TIMESTAMP
      )`);
      db.run(`CREATE TABLE IF NOT EXISTS player_stats (
        id INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

        user_id INTEGER NOT NULL UNIQUE,
        matches INTEGER DEFAULT 0,
        goals INTEGER DEFAULT 0,
        assists INTEGER DEFAULT 0,
        yellow_cards INTEGER DEFAULT 0,
        red_cards INTEGER DEFAULT 0,
        attendance INTEGER DEFAULT 0,
        FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
    )`);

db.run(`CREATE TABLE IF NOT EXISTS schedules (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    team_id INTEGER NOT NULL,
    event_name TEXT NOT NULL,
    event_date TEXT NOT NULL,
    location TEXT,
    description TEXT,
    event_time TEXT,
    event_type TEXT,
    createdAt TEXT,
    FOREIGN KEY (team_id) REFERENCES teams(id) ON DELETE CASCADE
)`);

db.run(`CREATE TABLE IF NOT EXISTS attendance (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    event_id INTEGER NOT NULL,
    status TEXT CHECK(status IN ('present', 'absent', 'late',
'excused')) DEFAULT 'absent',
    checked_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    notes TEXT,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (event_id) REFERENCES schedules(id) ON DELETE CASCADE,
    UNIQUE(user_id, event_id)
)`);

db.run(`CREATE TABLE IF NOT EXISTS chat_rooms (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    team_id INTEGER NOT NULL,
    name TEXT NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (team_id) REFERENCES teams(id) ON DELETE CASCADE
)`);

db.run(`CREATE TABLE IF NOT EXISTS messages (
    id INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

        room_id INTEGER NOT NULL,
        user_id INTEGER NOT NULL,
        username TEXT NOT NULL,
        message TEXT NOT NULL,
        attachment_url TEXT,
        attachment_name TEXT,
        attachment_type TEXT,
        attachment_size INTEGER,
        reply_to_message_id INTEGER,
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP
    ));
db.run('CREATE TABLE IF NOT EXISTS direct_messages (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    sender_id INTEGER NOT NULL,
    receiver_id INTEGER NOT NULL,
    message TEXT NOT NULL,
    attachment_url TEXT,
    attachment_name TEXT,
    attachment_type TEXT,
    attachment_size INTEGER,
    is_read INTEGER DEFAULT 0,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
    ));
});
}
});

/**
 * Nosūta kontaktformas ziņojumu uz sistēmas e-pastu.
 *
 * @route POST /api/contact
 */
app.post('/api/contact', async (req, res) => {
    const { name, email, subject, message } = req.body;
    if (!name || !email || !subject || !message) {
        return res.status(400).json({ error: 'Visi lauki ir obligāti' });
    }

    const mailOptions = {
        from: `TeamFlow kontakti` <${emailUser}>,
        to: 'vladimiravgustin123@gmail.com',
        replyTo: email,

```

```

    subject: `${subjectLabels[subject] || subject} - no ${name}`,
    html: `
      <p><strong>Vārds:</strong> ${name}</p>
      <p><strong>E-pasts:</strong> ${email}</p>
      <p><strong>Tēma:</strong> ${subjectLabels[subject] || subject}</p>
      <hr/>
      <p><strong>Ziņojums:</strong></p>
      <p>${message.replace(/\n/g, '<br>')}</p>
    `;
  };

  try {
    await transporter.sendMail(mailOptions);
    res.status(200).json({ message: 'E-pasts veiksmīgi nosūtīts' });
  } catch (err) {
    res.status(500).json({ error: 'Neizdevās nosūtīt e-pastu.' });
  }
});

/**
 * Pārbauda Socket.IO lietotāja JWT tokens.
 */
io.use((socket, next) => {
  const token = socket.handshake.auth.token;
  if (!token) return next(new Error('Autentifikācijas kļūda'));

  try {
    const decoded = jwt.verify(token, 'your_jwt_secret');
    socket.userId = decoded.id;
    socket.userFullName = `${decoded.name || ''} ${decoded.surname || ''}`.trim();
    next();
  } catch (err) {
    next(new Error('Autentifikācijas kļūda'));
  }
});

/**
 * Apstrādā komandas čata un tiešo ziņojumu reāllaika notikumus.
 */
io.on('connection', (socket) => {
  socket.join(`user_${socket.userId}`);

```

```

socket.on('join_room', async (roomIdValue) => {
  const roomId = normalizePositiveInteger(roomIdValue);
  const roomMembership = await getRoomMembership(roomId, socket.userId);
  if (!roomMembership) {
    socket.emit('message_error', { error: 'Piekļuve liegta' });
    return;
  }
  socket.join(`room_${roomId}`);
});

```

```

socket.on('send_message', async (data) => {
  const roomId = normalizePositiveInteger(data?.roomId);
  const message = normalizeChatText(data?.message);
  const attachment = normalizeAttachment(data?.attachment);
  if (!roomId || (!message && !attachment)) {
    socket.emit('message_error', { error: 'Ziņojums nav derīgs' });
    return;
  }

```

```

const result = await dbRun(
  `INSERT INTO messages (
    room_id, user_id, username, message,
    attachment_url, attachment_name, attachment_type, attachment_size
  ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)`,
  [
    roomId,
    socket.userId,
    socket.userFullName,
    message,
    attachment?.attachment_url || null,
    attachment?.attachment_name || null,
    attachment?.attachment_type || null,
    attachment?.attachment_size || null
  ]
);

```

```

io.to(`room_${roomId}`).emit('new_message', {
  id: result.lastID,
  roomId,
  userId: socket.userId,
  username: socket.userFullName,

```

```

        message,
        createdAt: new Date().toISOString()
    });
});

socket.on('send_dm', (data) => {
    const { receiverId } = data;
    const message = normalizeChatText(data?.message);
    const attachment = normalizeAttachment(data?.attachment);
    if (!message && !attachment) return;

    db.run(
        `INSERT INTO direct_messages (
            sender_id, receiver_id, message,
            attachment_url, attachment_name, attachment_type, attachment_size
        ) VALUES (?, ?, ?, ?, ?, ?, ?)` ,
        [
            socket.userId,
            receiverId,
            message,
            attachment?.attachment_url || null,
            attachment?.attachment_name || null,
            attachment?.attachment_type || null,
            attachment?.attachment_size || null
        ],
        function(err) {
            if (err) {
                socket.emit('dm_error', { error: 'Neizdevās nosūtīt ziņojumu' });
                return;
            }

            const dmData = {
                id: this.lastID,
                sender_id: socket.userId,
                receiver_id: receiverId,
                message,
                is_read: 0,
                created_at: new Date().toISOString()
            };

            io.to(`user_${receiverId}`).emit('new_dm', dmData);
            socket.emit('dm_sent', dmData);
        }
    );
});

```



```

    }
  );
});
});

```

```
---
```

middleware/auth.js

```

/**
 * Šis fails nodrošina API pieprasījumu autentifikāciju un lietotāja lomas
 * pārbaudi.
 *
 * Galvenās funkcijas:
 * - authenticateToken(): pārbauda Bearer JWT tokens.
 * - requireRole(): pārbauda, vai lietotājam ir vajadzīgā loma.
 *
 * Nosaukums: backend/middleware
 * Izmantotās bibliotēkas: jsonwebtoken
 */
const jwt = require('jsonwebtoken');

function authenticateToken(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({ error: 'Autorizācijas tokens nav
norādīts' });
  }

  jwt.verify(token, process.env.JWT_SECRET || 'your_jwt_secret', (err,
user) => {
    if (err) {
      return res.status(403).json({ error: 'Nederīgs autorizācijas tokens'
});
    }
    req.user = user;
    next();
  });
}

function requireRole(role) {

```

```

    return (req, res, next) => {
      if (req.user.role !== role) {
        return res.status(403).json({ error: 'Piekļuve liegta' });
      }
      next();
    };
  }
}

module.exports = { authenticateToken, requireRole };

```

routes/auth.js

```

/**
 * Šis fails nodrošina lietotāju reģistrāciju, pieslēgšanos, paroles
atjaunošanu,
 * komandas izveidi, pievienošanos komandai, spēlētāju statistiku un
komandas
 * kopsavilkumu.
 *
 * Galvenās funkcijas:
 * - authenticateToken(): pārbauda API pieprasījuma JWT marķieri.
 * - getUserProfile(): iegūst lietotāja profilu.
 * - ensureTeamChatRoom(): izveido komandas čata istabu, ja tā vēl
neeksistē.
 * - /register: reģistrē lietotāju.
 * - /login: pieslēdz lietotāju.
 * - /teams: izveido komandu.
 * - /join-team: pievieno spēlētāju komandai vai izveido trenera
pieprasījumu.
 * - /players/:id/stats: atjaunina spēlētāja statistiku.
 * - /teams/:teamId/stats: aprēķina komandas statistiku.
 *
 * Nosaukums: backend/routes
 * Izmantotās bibliotēkas: express, bcrypt, jsonwebtoken, crypto,
nodemailer
 * Izmantotās tabulas: users, teams, player_stats, chat_rooms,
password_resets.
 */
const express = require('express');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');

```

```

const crypto = require('crypto');
const nodemailer = require('nodemailer');

const JWT_SECRET = 'your_jwt_secret';
const MIN_PASSWORD_LENGTH = 8;
const normalizeRole = (role) => (typeof role === 'string' ?
role.trim().toLowerCase() : '');

const dbGet = (sql, params = []) => new Promise((resolve, reject) => {
  db.get(sql, params, (err, row) => err ? reject(err) : resolve(row));
});
const dbAll = (sql, params = []) => new Promise((resolve, reject) => {
  db.all(sql, params, (err, rows) => err ? reject(err) : resolve(rows));
});
const dbRun = (sql, params = []) => new Promise((resolve, reject) => {
  db.run(sql, params, function runCallback(err) {
    if (err) reject(err);
    else resolve(this);
  });
});
const getUserProfile = (userId) => dbGet(
  'SELECT id, name, surname, email, role, team_id, avatar FROM users WHERE
id = ?',
  [userId]
);

/**
 * Izveido komandas čata istabu, ja tā vēl nav izveidota.
 */
const ensureTeamChatRoom = async (teamId, teamName) => {
  const existingRoom = await dbGet('SELECT id FROM chat_rooms WHERE team_id
= ?', [teamId]);
  if (!existingRoom) {
    await dbRun('INSERT INTO chat_rooms (team_id, name) VALUES (?, ?)', [
      teamId,
      `${teamName} čats`
    ]);
  }
};

/**
 * Pārbauda Bearer JWT tokens.
 */

```

```

const authenticateToken = (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Autorizācijas tokens nav norādīts' });
  }

  try {
    req.user = jwt.verify(authHeader.split(' ')[1], JWT_SECRET);
    next();
  } catch (err) {
    return res.status(403).json({ error: 'Nederīgs autorizācijas tokens' });
  }
};

/**
 * Reģistrē jaunu lietotāju un, ja lietotājs ir spēlētājs, izveido viņam
 * sākotnējo statistikas ierakstu.
 *
 * @route POST /api/auth/register
 */
router.post('/register', async (req, res) => {
  const { name, surname, email, password, role } = req.body;
  if (!name || !surname || !email || !password) {
    return res.status(400).json({ error: 'Lūdz, aizpildiet visus laukus.' });
  }

  if (String(password).length < MIN_PASSWORD_LENGTH) {
    return res.status(400).json({ error: `Parolei jābūt vismaz ${MIN_PASSWORD_LENGTH} rakstzīmes garai` });
  }

  db.get('SELECT * FROM users WHERE email = ?', [email], async (err, row)
=> {
    if (row) return res.status(400).json({ error: 'Lietotājs ar šo e-pastu jau pastāv' });

    const hashedPassword = await bcrypt.hash(password, 10);
    const userRole = role || 'user';

    db.run(

```

```

    'INSERT INTO users (name, surname, role, email, password, team_id)
VALUES (?, ?, ?, ?, ?, ?)',
    [name, surname, userRole, email, hashedPassword, null],
    function(insertErr) {
        if (insertErr) return res.status(500).json({ error: 'Reģistrācijas
kļūda' });

        const userId = this.lastID;
        if (userRole.toLowerCase() === 'player') {
            db.run(
                'INSERT INTO player_stats (user_id, matches, goals, assists,
yellow_cards, red_cards) VALUES (?, 0, 0, 0, 0, 0)',
                [userId],
                () => res.status(201).json({ message: 'Lietotājs veiksmīgi
reģistrēts', userId })
            );
        } else {
            res.status(201).json({ message: 'Lietotājs veiksmīgi reģistrēts',
userId });
        }
    }
);
});
});

/**
 * Pieslēdz lietotāju, pārbaudot e-pastu un paroli, un izsniedz JWT tokens.
 *
 * @route POST /api/auth/login
 */
router.post('/login', (req, res) => {
    const { email, password } = req.body;
    if (!email || !password) {
        return res.status(400).json({ error: 'Ievadiet e-pastu un paroli' });
    }

    db.get('SELECT * FROM users WHERE email = ?', [email], async (err, user)
=> {
        if (!user) return res.status(400).json({ error: 'Lietotājs nav atrasts'
});

        const isPasswordValid = await bcrypt.compare(password, user.password);
        if (!isPasswordValid) {
            return res.status(400).json({ error: 'Nepareiza parole' });

```

```

    }

    const token = jwt.sign(
      { id: user.id, name: user.name, surname: user.surname, role:
user.role },
      JWT_SECRET,
      { expiresIn: '7d' }
    );

    res.status(200).json({
      message: 'Pieslēgšanās veiksmīga',
      user: { id: user.id, name: user.name, surname: user.surname, role:
user.role },
      token
    });
  });
});

/**
 * Izveido paroles atjaunošanas tokens un saglabā tā hash datubāzē.
 *
 * @route POST /api/auth/forgot-password
 */
router.post('/forgot-password', (req, res) => {
  const { email } = req.body;
  if (!email) return res.status(400).json({ error: 'E-pasts ir obligāts'
});

  db.get('SELECT id, email FROM users WHERE email = ?', [email], (err,
user) => {
    if (!user) return res.status(404).json({ error: 'Konts ar šo e-pastu
nav atrasts.' });

    const rawToken = crypto.randomBytes(32).toString('hex');
    const tokenHash =
crypto.createHash('sha256').update(rawToken).digest('hex');
    const expiresAt = Math.floor(Date.now() / 1000) + (60 * 60);

    db.run(
      'INSERT INTO password_resets (user_id, token_hash, expires_at) VALUES
(?, ?, ?)',
      [user.id, tokenHash, expiresAt],
      () => res.status(200).json({ message: 'Paroles atiestatīšanas e-pasts
ir nosūtīts.' })
    );
  });
});

```

```

    });
  });

/**
 * Izveido jaunu komandu, izveido komandas čatu un piesaista treneri komandai.
 *
 * @route POST /api/auth/teams
 */
router.post('/teams', authenticateToken, (req, res) => {
  const coachId = req.user.id;
  const { name, teamCode } = req.body;
  if (!name || !teamCode) {
    return res.status(400).json({ error: 'Lūdzu, aizpildiet visus komandas laukus' });
  }

  db.run(
    `INSERT INTO teams (name, team_code, coach_id) VALUES (?, ?, ?)`,
    [name, teamCode, coachId],
    function(err) {
      if (err) return res.status(500).json({ error: 'Kļūda, izveidojot komandu' });

      const newTeamId = this.lastID;
      db.run('INSERT INTO chat_rooms (team_id, name) VALUES (?, ?)', [
        newTeamId,
        `${name} čats`
      ]);
      db.run('UPDATE users SET team_id = ? WHERE id = ?', [newTeamId, coachId], () => {
        res.status(201).json({
          message: 'Komanda veiksmīgi izveidota',
          team: { id: newTeamId, name, teamCode, coachId }
        });
      });
    }
  );
});

/**
 * Pievieno lietotāju komandai pēc komandas koda.
 *
 * Spēlētājs tiek pievienots uzreiz, bet trenerim tiek izveidots asistenta pieprasījums.

```

```

*
* @route POST /api/auth/join-team
*/
router.post('/join-team', authenticateToken, async (req, res) => {
  const normalizedTeamCode = typeof req.body.teamCode === 'string'
    ? req.body.teamCode.trim().toUpperCase()
    : '';
  const user = await getUserProfile(req.user.id);
  const team = await dbGet('SELECT id, name, coach_id FROM teams WHERE
team_code = ?', [
    normalizedTeamCode
  ]);

  if (!team) return res.status(404).json({ message: 'Komanda ar šo kodu nav
atrasta' });

  if (user.team_id) return res.status(400).json({ message: 'Lietotājs jau
ir komandā' });

  if (normalizeRole(user.role) === 'coach') {
    await dbRun(
      `INSERT INTO coach_join_requests (team_id, requester_user_id, status)
VALUES (?, ?, 'pending')`,
      [team.id, user.id]
    );
    return res.status(202).json({
      message: 'Trenera asistenta pieprasījums nosūtīts galvenajam
trenerim.',
      requestCreated: true
    });
  }

  await dbRun('UPDATE users SET team_id = ? WHERE id = ?', [team.id,
user.id]);
  await ensureTeamChatRoom(team.id, team.name);

  if (normalizeRole(user.role) === 'player') {
    await dbRun(
      `INSERT INTO player_stats (user_id, matches, goals, assists,
yellow_cards, red_cards)
VALUES (?, 0, 0, 0, 0, 0, 0)
ON CONFLICT(user_id) DO UPDATE SET matches = 0, goals = 0, assists =
0`,
      [user.id]
    );
  }
}

```



```

    }

    res.status(200).json({
      message: 'Jūs veiksmīgi pievienojāties komandai',
      user: await getUserProfile(user.id),
      team
    });
  });
});

/**
 * Atjaunina spēlētāja statistiku, ja pieprasījumu veic tās pašas komandas
 * treneris.
 *
 * @route PUT /api/auth/players/:id/stats
 */
router.put('/players/:id/stats', authenticateToken, (req, res) => {
  const playerId = req.params.id;
  const { matches, goals, assists, yellow_cards, red_cards } = req.body;

  (async () => {
    const coach = await getUserProfile(req.user.id);
    const player = await dbGet('SELECT team_id, role FROM users WHERE id = ? ', [playerId]);

    if (!coach || normalizeRole(coach.role) !== 'coach' || !coach.team_id) {
      return res.status(403).json({ error: 'Statistiku var atjaunināt tikai treneris' });
    }

    if (!player || Number(player.team_id) !== Number(coach.team_id)) {
      return res.status(403).json({ error: 'Nav atļauts atjaunināt šo spēlētāju' });
    }

    await dbRun(
      `INSERT INTO player_stats
      (user_id, matches, goals, assists, yellow_cards, red_cards)
      VALUES (?, ?, ?, ?, ?, ?)
      ON CONFLICT(user_id) DO UPDATE SET
        matches = excluded.matches,
        goals = excluded.goals,
        assists = excluded.assists,
        yellow_cards = excluded.yellow_cards,`
    );
  })();
});

```

```

        red_cards = excluded.red_cards`,
        [playerId, matches, goals, assists, yellow_cards, red_cards]
    );

    res.status(200).json({ message: 'Statistika veiksmīgi atjaunināta' });
  })();
});

/**
 * Aprēķina komandas statistikas kopsavilkumu.
 *
 * @route GET /api/auth/teams/:teamId/stats
 */
router.get('/teams/:teamId/stats', (req, res) => {
  const { teamId } = req.params;

  db.all(
    `SELECT u.id as user_id, u.name, u.surname,
      COALESCE(ps.matches, 0) as matches,
      COALESCE(ps.goals, 0) as goals,
      COALESCE(ps.assists, 0) as assists,
      COALESCE(ps.yellow_cards, 0) as yellow_cards,
      COALESCE(ps.red_cards, 0) as red_cards
    FROM users u
    LEFT JOIN player_stats ps ON ps.user_id = u.id
    WHERE u.team_id = ? AND LOWER(u.role) != 'coach'`,
    [teamId],
    (playersErr, players) => {
      const safePlayers = Array.isArray(players) ? players : [];
      res.json({
        totalPlayers: safePlayers.length,
        totalGoals: safePlayers.reduce((sum, player) => sum +
          (Number(player.goals) || 0), 0),
        totalAssists: safePlayers.reduce((sum, player) => sum +
          (Number(player.assists) || 0), 0),
        totalYellowCards: safePlayers.reduce((sum, player) => sum +
          (Number(player.yellow_cards) || 0), 0),
        totalRedCards: safePlayers.reduce((sum, player) => sum +
          (Number(player.red_cards) || 0), 0),
        topScorers: safePlayers
          .filter((player) => (Number(player.goals) || 0) > 0)
          .sort((a, b) => Number(b.goals) - Number(a.goals))
          .slice(0, 5)
      });
    }
  );
});

```

```

    });
  }
);
});

```

```
---
```

routes/schedules.js

```

/**
 * Šis fails nodrošina komandas grafiku un apmeklējuma uzskaiti.
 *
 * Galvenās funkcijas:
 * - GET /teams/:id/schedule: ielādē grafiku.
 * - POST /teams/:id/schedule: pievieno notikumu.
 * - PUT /teams/:teamId/schedule/:eventId: atjaunina notikumu.
 * - DELETE /teams/:teamId/schedule/:eventId: dzēš notikumu.
 * - GET /attendance/full: ielādē treniņa apmeklējumu.
 * - POST /attendance: saglabā spēlētāja apmeklējuma statusu.
 * - GET /attendance/stats: aprēķina komandas apmeklējuma statistiku.
 *
 * Nosaukums: backend/routes
 * Izmantotās tabulas: schedules, attendance, users.
 */
const express = require('express');
const router = express.Router();

module.exports = (db) => {
  /**
   * Ielādē komandas notikumus.
   */
  router.get('/teams/:id/schedule', (req, res) => {
    const teamId = req.params.id;
    db.all(
      `SELECT * FROM schedules
      WHERE team_id = ?
      ORDER BY event_date ASC, time(COALESCE(NULLIF(event_time, ''),
'23:59:59')) ASC, id ASC`,
      [teamId],
      (err, rows) => {

```

```

        if (err) return res.status(500).json({ error: 'Kļūda, ielādējot
grafiku' });
        res.json(rows);
    }
    );
});

/**
 * Pievieno jaunu grafika notikumu un pārbauda laika konfliktu.
 */
router.post('/teams/:id/schedule', (req, res) => {
    const teamId = req.params.id;
    const { event_name, event_date, location, event_time, event_type,
description } = req.body;
    const createdAt = new Date().toISOString();

    if (!event_name || !event_date) {
        return res.status(400).json({ error: 'Notikuma nosaukums un datums ir
obligāti' });
    }

    db.get(
        `SELECT id FROM schedules WHERE team_id = ? AND event_date = ? AND
event_time = ?`,
        [teamId, event_date, event_time],
        (err, existing) => {
            if (existing) return res.status(409).json({ error: 'Šajā datumā un
laikā jau ir notikums' });

            db.run(
                `INSERT INTO schedules
                (team_id, event_name, event_date, location, description,
event_time, event_type, createdAt)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?)`,
                [teamId, event_name, event_date, location, description || '',
event_time, event_type, createdAt],
                function(err) {
                    if (err) return res.status(500).json({ error: 'Kļūda,
pievienojot notikumu' });
                    res.json({ id: this.lastID, team_id: teamId, event_name,
event_date, location, description, event_time, event_type, createdAt });
                }
            );
        }
    );
});

```

```

});

/**
 * Atjaunina notikumu.
 */
router.put('/teams/:teamId/schedule/:eventId', (req, res) => {
  const { teamId, eventId } = req.params;
  const { event_name, event_date, location, event_time, event_type,
description } = req.body;

  db.run(
    `UPDATE schedules
      SET event_name = ?, event_date = ?, location = ?, description = ?,
event_time = ?, event_type = ?
      WHERE id = ? AND team_id = ?`,
    [event_name, event_date, location, description || '', event_time,
event_type, eventId, teamId],
    function(err) {
      if (err) return res.status(500).json({ error: 'Kļūda, atjauninot
notikumu' });
      if (this.changes === 0) return res.status(404).json({ error:
'Notikums nav atrasts' });
      res.json({ success: true });
    }
  );
});

/**
 * Dzēš notikumu no grafika.
 */
router.delete('/teams/:teamId/schedule/:eventId', (req, res) => {
  const { teamId, eventId } = req.params;
  db.run(`DELETE FROM schedules WHERE id = ? AND team_id = ?`, [eventId,
teamId], function(err) {
    if (err) return res.status(500).json({ error: 'Kļūda, dzēšot
notikumu' });
    if (this.changes === 0) return res.status(404).json({ error:
'Notikums nav atrasts' });
    res.json({ success: true });
  });
});

/**
 * Ielādē pilnu treniņa apmeklējuma sarakstu.

```

```

    */
    router.get('/teams/:teamId/events/:eventId/attendance/full', (req, res)
=> {
    const { teamId, eventId } = req.params;
    db.all(`
        SELECT u.id as user_id, u.name, u.surname,
            (u.name || ' ' || u.surname) as username,
            u.email, u.avatar,
            COALESCE(a.status, 'unmarked') as status,
            a.checked_at, a.notes
        FROM users u
        LEFT JOIN attendance a ON u.id = a.user_id AND a.event_id = ?
        WHERE u.team_id = ? AND LOWER(u.role) = 'player'
        ORDER BY u.surname, u.name
    `, [eventId, teamId], (err, rows) => {
        if (err) return res.status(500).json({ error: 'Kļūda, ielādējot
apmeklējumu' });
        res.json(rows);
    });
});

/**
 * Saglabā spēlētāja apmeklējuma statusu.
 */
router.post('/teams/:teamId/events/:eventId/attendance', (req, res) => {
    const { eventId } = req.params;
    const { user_id, status, notes } = req.body;
    const validStatuses = new Set(['present', 'absent', 'late',
'excused']);
    if (!user_id || !validStatuses.has(status)) {
        return res.status(400).json({ error: 'Nederīgs apmeklējuma statuss'
});
    }

    db.run(`
        INSERT INTO attendance (user_id, event_id, status, notes)
        VALUES (?, ?, ?, ?)
        ON CONFLICT(user_id, event_id) DO UPDATE SET
            status = excluded.status,
            notes = excluded.notes,
            checked_at = CURRENT_TIMESTAMP
    `, [user_id, eventId, status, notes || null], function(err) {
        if (err) return res.status(500).json({ error: 'Kļūda, saglabājot
apmeklējumu' });
    });
}

```

```

    res.json({ success: true, id: this.lastID, user_id, event_id:
eventId, status, notes });
  });
});

/**
 * Aprēķina komandas treniņu apmeklējuma statistiku.
 */
router.get('/teams/:teamId/attendance/stats', (req, res) => {
  const { teamId } = req.params;
  db.get(
    `SELECT COUNT(*) as total_practices FROM schedules
    WHERE team_id = ? AND LOWER(event_type) = 'practice'`,
    [teamId],
    (err, practiceCount) => {
      const totalPractices = practiceCount?.total_practices || 0;
      db.all(`
        SELECT u.id as user_id, (u.name || ' ' || u.surname) as username,
        u.avatar,
        SUM(CASE WHEN a.status = 'present' THEN 1 ELSE 0 END) as
present_count,
        SUM(CASE WHEN a.status = 'absent' OR a.status = 'excused'
THEN 1 ELSE 0 END) as absent_count,
        SUM(CASE WHEN a.status = 'late' THEN 1 ELSE 0 END) as
late_count
        FROM users u
        LEFT JOIN attendance a ON u.id = a.user_id
        LEFT JOIN schedules s ON a.event_id = s.id AND s.team_id = ?
        WHERE u.team_id = ? AND LOWER(u.role) = 'player'
        GROUP BY u.id
        ORDER BY present_count DESC
      `, [teamId, teamId], (err, rows) => {
        const stats = rows.map((row) => {
          const presentCount = Number(row.present_count) || 0;
          const absentCount = Number(row.absent_count) || 0;
          const lateCount = Number(row.late_count) || 0;
          const markedCount = presentCount + absentCount + lateCount;
          return {
            ...row,
            total_practices: totalPractices,
            unmarked_count: Math.max(totalPractices - markedCount, 0),
            attendance_rate: totalPractices > 0 ?
Math.round((presentCount / totalPractices) * 100) : 0
          };
        });
      });
    }
  );
});

```

```

        });
        res.json(stats);
    });
}
);
});
};

```

routes/chat.js

```

/**
 * Šis fails nodrošina čata API: pielikumus, komandas čata istabas,
 * ziņojumu vēsturi un tiešās sarakstes.
 *
 * Galvenās funkcijas:
 * - sanitizeFileName(): attīra faila nosaukumu.
 * - parseDataUrl(): nolasa base64 failu.
 * - /upload: saglabā pielikumu.
 * - /team/:teamId/room: iegūst vai izveido komandas čatu.
 * - /room/:roomId/messages: ielādē čata ziņojumus.
 * - /dm/:userId: ielādē tiešo ziņojumu vēsturi.
 *
 * Nosaukums: backend/routes
 * Izmantotās bibliotēkas: express, crypto, fs, path
 */
const express = require('express');
const crypto = require('crypto');
const fs = require('fs');
const path = require('path');
const router = express.Router();
const { authenticateToken } = require('../middleware/auth');

const MAX_CHAT_UPLOAD_BYTES = 6 * 1024 * 1024;
const CHAT_UPLOAD_DIR = path.join(__dirname, '..', 'uploads', 'chat');
const CHAT_ATTACHMENT_API_PREFIX = '/api/chat/attachments/';

const sanitizeFileName = (fileName) => {
    const baseName = path.basename(String(fileName || 'attachment'));
    return baseName.replace(/^[^\\w\\.\\- ]+/g, '').replace(/\\s+/g, '_').slice(0,
80) || 'attachment';

```



```

};

const parseDataURL = (dataUrl) => {
  const match = String(dataUrl ||
  '').match(/^data:([^;]+);base64,(.+)$/s);
  if (!match) return null;
  return { mimeType: match[1].toLowerCase(), base64: match[2] };
};

/**
 * Augšupielādē čata pielikumu serverī.
 *
 * @route POST /api/chat/upload
 */
router.post('/upload', authenticateToken, (req, res) => {
  const parsedData = parseDataURL(req.body?.data);
  const mimeType = parsedData?.mimeType || String(req.body?.mimeType ||
  '').toLowerCase();

  if (!parsedData || !MIME_EXTENSIONS[mimeType]) {
    return res.status(400).json({ error: 'Neatbalstīts faila tips' });
  }

  const fileBuffer = Buffer.from(parsedData.base64, 'base64');
  if (!fileBuffer.length || fileBuffer.length > MAX_CHAT_UPLOAD_BYTES) {
    return res.status(400).json({ error: 'Failam jābūt mazākam par 6 MB'
  });
  }

  fs.mkdirSync(CHAT_UPLOAD_DIR, { recursive: true });
  const originalName = sanitizeFileName(req.body?.fileName);
  const extension = MIME_EXTENSIONS[mimeType];
  const storedName = `${Date.now()}-${
  crypto.randomBytes(8).toString('hex')}${extension}`;
  const storedPath = path.join(CHAT_UPLOAD_DIR, storedName);

  fs.writeFile(storedPath, fileBuffer, (err) => {
    if (err) return res.status(500).json({ error: 'Neizdevās augšupielādēt
    failu' });
    res.status(201).json({
      attachmentUrl: `${CHAT_ATTACHMENT_API_PREFIX}${storedName}`,
      attachmentName: originalName,
      attachmentType: mimeType,
      attachmentSize: fileBuffer.length
    });
  });
});

```

```

    });
  });
});

/**
 * Iegūst vai izveido komandas čata istabu.
 *
 * @route GET /api/chat/team/:teamId/room
 */
router.get('/team/:teamId/room', authenticateToken, (req, res) => {
  const { teamId } = req.params;
  const userId = req.user.id;

  db.get(`SELECT * FROM users WHERE id = ? AND team_id = ?`, [userId,
    teamId], (err, userTeam) => {
    if (!userTeam) return res.status(403).json({ error: 'Jūs neesat šīs
    komandas dalībnieks' });

    db.get(`SELECT * FROM chat_rooms WHERE team_id = ?`, [teamId], (err,
    room) => {
      if (room) return res.json(room);

      db.run(`INSERT INTO chat_rooms (team_id, name) VALUES (?, ?)`,
        [teamId, `Komandas ${teamId} čats`], function(err) {
          if (err) return res.status(500).json({ error: 'Neizdevās izveidot
          čata istabu' });

          res.json({
            id: this.lastID,
            team_id: teamId,
            name: `Komandas ${teamId} čats`,
            created_at: new Date().toISOString()
          });
        });
      });
    });
  });
});

/**
 * Ielādē komandas čata ziņojumu vēsturi.
 *
 * @route GET /api/chat/room/:roomId/messages
 */
router.get('/room/:roomId/messages', authenticateToken, (req, res) => {

```

```

const { roomId } = req.params;
const { limit = 50, before } = req.query;
const userId = req.user.id;

db.get(
  `SELECT cr.* FROM chat_rooms cr
  INNER JOIN teams t ON cr.team_id = t.id
  INNER JOIN users u ON u.team_id = t.id
  WHERE cr.id = ? AND u.id = ?`,
  [roomId, userId],
  (err, room) => {
    if (!room) return res.status(403).json({ error: 'Piekļuve liegta' });

    let query = `SELECT * FROM messages WHERE room_id = ?`;
    const params = [roomId];
    if (before) {
      query += ` AND created_at < ?`;
      params.push(before);
    }
    query += ` ORDER BY created_at DESC LIMIT ?`;
    params.push(parseInt(limit));

    db.all(query, params, (err, messages) =>
      res.json(messages.reverse()));
  }
);

/**
 * Ielādē tiešo ziņojumu vēsturi ar konkrētu lietotāju.
 */
router.get('/dm/:userId', authenticateToken, (req, res) => {
  const currentUserId = req.user.id;
  const otherUserId = req.params.userId;

  db.all(
    `SELECT dm.*, sender.name as sender_name, receiver.name as
    receiver_name
    FROM direct_messages dm
    INNER JOIN users sender ON dm.sender_id = sender.id
    INNER JOIN users receiver ON dm.receiver_id = receiver.id
    WHERE (dm.sender_id = ? AND dm.receiver_id = ?)
    OR (dm.sender_id = ? AND dm.receiver_id = ?)

```

```

    ORDER BY dm.created_at ASC
    LIMIT ?`,
    [currentUserId, otherUserId, otherUserId, currentUserId, 50],
    (err, messages) => res.json(messages)
  );
});

---
```

routes/admin.js

```

/**
 * Šis fails nodrošina administratora API lietotāju un komandu pārvaldībai.
 *
 * Galvenās funkcijas:
 * - requireRole(): pārbauda administratora lomu.
 * - /users: ielādē lietotājus.
 * - /teams: ielādē komandas.
 * - PUT /users/:id: atjaunina lietotāju.
 * - DELETE /users/:id: dzēš lietotāju.
 * - PUT /teams/:id: atjaunina komandu.
 *
 * Nosaukums: backend/routes
 * Izmantotās tabulas: users, teams
 */
function requireRole(role) {
  return (req, res, next) => {
    if (!req.user || (req.user.role || '').toLowerCase() !==
    role.toLowerCase()) {
      return res.status(403).json({ error: 'Piekļuve liegta' });
    }
    next();
  };
}

const normalizeTeamId = (value) => {
  if (value === null || value === undefined || value === '') return null;
  const parsedValue = Number(value);
  return Number.isInteger(parsedValue) && parsedValue > 0 ? parsedValue :
  NaN;
};
```

```

const normalizeTeamCode = (value) => (
  typeof value === 'string' ? value.trim().toUpperCase() : ''
);

/**
 * Ielādē lietotājus un komandas administratora panelim.
 */
router.get('/users', authenticateToken, requireRole('admin'), async (req,
res) => {
  try {
    const users = await dbAll('SELECT * FROM users ORDER BY id ASC');
    res.json({ users });
  } catch (error) {
    res.status(500).json({ error: 'Kļūda, ielādējot lietotājus' });
  }
});

router.get('/teams', authenticateToken, requireRole('admin'), async (req,
res) => {
  try {
    const teams = await dbAll(
      `SELECT t.*,
        CASE
          WHEN coach.id IS NULL THEN NULL
          ELSE TRIM(COALESCE(coach.name, '') || ' ' ||
COALESCE(coach.surname, ''))
        END AS coach_name,
        (SELECT COUNT(*) FROM users u WHERE u.team_id = t.id) AS
member_count,
        (SELECT COUNT(*) FROM users u WHERE u.team_id = t.id AND
LOWER(u.role) = 'player') AS player_count
      FROM teams t
      LEFT JOIN users coach ON coach.id = t.coach_id
      ORDER BY LOWER(t.name) ASC`
    );
    res.json({ teams });
  } catch (error) {
    res.status(500).json({ error: 'Kļūda, ielādējot komandas' });
  }
});

/**
 * Atjaunina lietotāja vai komandas datus administratora panelī.
 */

```

```

router.put('/users/:id', authenticateToken, requireRole('admin'), async
(req, res) => {
  const { name, surname, email, role, team_id } = req.body;
  const normalizedTeamId = normalizeTeamId(team_id);
  if (!String(email || '').trim()) {
    return res.status(400).json({ error: 'E-pasts ir obligāts' });
  }

  const result = await dbRun(
    `UPDATE users
      SET name = ?, surname = ?, email = ?, role = ?, team_id = ?, updatedAt
= CURRENT_TIMESTAMP
      WHERE id = ?`,
    [name, surname, email, role, normalizedTeamId, req.params.id]
  );

  if (!result.changes) return res.status(404).json({ error: 'Lietotājs nav
atrasts' });
  res.json({ success: true });
});

router.put('/teams/:id', authenticateToken, requireRole('admin'), async
(req, res) => {
  const teamId = Number(req.params.id);
  const name = typeof req.body.name === 'string' ? req.body.name.trim() :
'';
  const teamCode = normalizeTeamCode(req.body.team_code);

  const duplicateTeam = await dbGet('SELECT id FROM teams WHERE team_code =
? AND id != ?', [
    teamCode,
    teamId
  ]);

  if (duplicateTeam) return res.status(409).json({ error: 'Komandas kodam
jābūt unikālam' });

  await dbRun(
    `UPDATE teams SET name = ?, team_code = ?, updatedAt =
CURRENT_TIMESTAMP WHERE id = ?`,
    [name, teamCode, teamId]
  );

  const team = await getTeamDetails(teamId);
  res.json({ success: true, team });
});

```

stores/auth.js

```
/**
 * Šis Pinia store pārvalda lietotāja autentifikāciju frontend pusē.
 *
 * Galvenās funkcijas:
 * - login(): pieslēdz lietotāju un saglabā JWT tokens.
 * - fetchUser(): ielādē pašreizējo lietotāju.
 * - logout(): dzēš sesijas datus.
 * - initializeAuth(): atjauno sesiju no localStorage.
 *
 * Nosaukums: frontend/src/stores
 */
import { defineStore } from 'pinia';
import axios from 'axios';

const setAxiosAuthHeader = (token) => {
  if (token) axios.defaults.headers.common['Authorization'] = `Bearer ${token}`;
  else delete axios.defaults.headers.common['Authorization'];
};

export const useAuthStore = defineStore('auth', {
  state: () => ({
    user: JSON.parse(localStorage.getItem('user')) || null,
    token: localStorage.getItem('token') || null
  }),
  getters: {
    isAuthenticated: (state) => !!state.token
  },
  actions: {
    async login(email, password) {
      const res = await fetch('/api/auth/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ email, password })
      });
      const data = await res.json();
      if (!res.ok) throw new Error(data.error || 'Pieslēgšanās kļūda');
    }
  }
});
```

```

    this.token = data.token;
    localStorage.setItem('token', data.token);
    setAxiosAuthHeader(data.token);
    await this.fetchUser();
  },
  async fetchUser() {
    if (!this.token) return;
    const res = await fetch('/api/auth/me', {
      headers: { Authorization: `Bearer ${this.token}` }
    });
    if (!res.ok) {
      this.logout();
      return;
    }
    const data = await res.json();
    this.user = data.user;
    localStorage.setItem('user', JSON.stringify(this.user));
  },
  logout() {
    this.user = null;
    this.token = null;
    localStorage.removeItem('token');
    localStorage.removeItem('user');
    setAxiosAuthHeader(null);
    window.location.href = '/';
  }
}
});
---

```

stores/chat.js

```

/**
 * Šis Pinia store pārvalda komandas čatu un tiešos ziņojumus.
 *
 * Galvenās funkcijas:
 * - connect(): izveido Socket.IO savienojumu.
 * - joinRoom(): atver komandas čatu.
 * - sendMessage(): nosūta čata ziņojumu.
 * - uploadAttachment(): augšupielādē pielikumu.
 * - openDM(): atver tiešo saraksti.

```



```

* - sendDM(): nosūta tiešo ziņojumu.
*
* Nosaukums: frontend/src/stores
*/
import { defineStore } from 'pinia';
import { io } from 'socket.io-client';
import { useAuthStore } from '../auth';

const getApiBaseUrl = () => {
  const hostname = window.location.hostname;
  return hostname === 'localhost' || hostname === '127.0.0.1'
    ? 'http://localhost:3000'
    : `http://${hostname}:3000`;
};

const API_BASE_URL = getApiBaseUrl();

export const useChatStore = defineStore('chat', {
  state: () => ({
    socket: null,
    currentRoom: null,
    messages: [],
    rooms: [],
    typingUsers: [],
    isConnected: false,
    currentDM: null,
    dmMessages: [],
    dmConversations: [],
    allUsers: []
  }),
  getters: {
    sortedMessages: (state) => [...state.messages].sort((a, b) =>
      new Date(a.createdAt || a.created_at) - new Date(b.createdAt ||
b.created_at)
    )
  },
  actions: {

    /**
     * Izveido Socket.IO savienojumu un klausās jaunus ziņojumus.
     */
    connect() {
      const authStore = useAuthStore();

```

```

    if (!authStore.token || this.isConnected) return;

    this.socket = io(API_BASE_URL, {
      auth: { token: authStore.token },
      transports: ['websocket', 'polling']
    });

    this.socket.on('connect', () => { this.isConnected = true; });
    this.socket.on('disconnect', () => { this.isConnected = false; });

    this.socket.on('new_message', (message) => {
      if (this.currentRoom && message.roomId === this.currentRoom.id) {
        this.messages.push(message);
        this.scrollToBottom();
      }
    });

    this.socket.on('new_dm', (dmData) => {
      if (this.currentDM && dmData.sender_id === this.currentDM.userId) {
        this.dmMessages.push(dmData);
        this.scrollToBottom();
        this.markDMAsRead(dmData.sender_id);
      }
      this.fetchDMConversations();
    });
  },

  /**
   * Atver komandas čata istabu.
   */
  async joinRoom(roomId) {
    if (!this.socket || !roomId) return;

    const response = await
    fetch(`${API_BASE_URL}/api/chat/room/${roomId}/messages`, {
      headers: { Authorization: `Bearer ${useAuthStore().token}` }
    });

    this.messages = await response.json();

    this.currentRoom = this.rooms.find((room) => room.id === roomId) || {
    id: roomId };

    this.socket.emit('join_room', roomId);
  },

  /**

```

```

    * Nosūta komandas čata ziņojumu.
    */
    sendMessage(message, attachment = null, replyTo = null) {
      const text = typeof message === 'string' ? message.trim() : '';
      if (!this.socket || !this.currentRoom || (!text && !attachment))
        return;

      this.socket.emit('send_message', {
        roomId: this.currentRoom.id,
        message: text,
        attachment,
        replyTo: replyTo?.id ? { id: replyTo.id } : null
      });
    },

    /**
     * Atver un nosūta tiešos ziņojumus.
     */
    async openDM(userId, username) {
      const response = await fetch(`${API_BASE_URL}/api/chat/dm/${userId}`,
    {
      headers: { Authorization: `Bearer ${useAuthStore().token}` }
    });
      this.dmMessages = await response.json();
      this.currentDM = { userId, username };
      this.currentRoom = null;
      await this.markDMAsRead(userId);
    },
    sendDM(receiverId, message, attachment = null) {
      const text = typeof message === 'string' ? message.trim() : '';
      if (!this.socket || (!text && !attachment)) return;
      this.socket.emit('send_dm', { receiverId, message: text, attachment
    });
  }
}
});
---

```

views/CreateTeam.vue

```

/**
 * Šī lapa nodrošina jaunas komandas izveidi.

```

```

*
* Galvenās funkcijas:
* - generateTeamCode(): ģenerē sešu simbolu komandas kodu.
* - createTeam(): nosūta komandas datus serverim.
*
* Nosaukums: frontend/src/views
* Izmantotais API maršruts: /api/auth/teams
*/
const teamName = ref('');
const teamCode = ref('');

function generateTeamCode() {
  const charset = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789';
  let code = '';
  for (let i = 0; i < 6; i++) {
    code += charset.charAt(Math.floor(Math.random() * charset.length));
  }
  teamCode.value = code;
}

async function createTeam() {
  if (!auth.token) {
    alert(copy.value.loginRequired);
    return;
  }

  try {
    await axios.post('/api/auth/teams', {
      name: teamName.value,
      teamCode: teamCode.value
    }, {
      headers: { Authorization: `Bearer ${auth.token}` }
    });
    await auth.fetchUser();
    router.push('/');
  } catch (error) {
    alert(t('messages.error') + ': ' + (error.response?.data.error || error.message));
  }
}

onMounted(() => generateTeamCode());

```

views/TeamDashboard.vue

```
/**
 * Šī lapa ir komandas galvenais panelis.
 *
 * Galvenās funkcijas:
 * - playerScore(): aprēķina spēlētāja snieguma punktus.
 * - rankedPlayers(): sakārto spēlētājus pēc snieguma.
 * - upcomingEvents(): atlasa tuvākos notikumus.
 * - fetchDashboard(): ielādē komandas datus, statistiku un grafiku.
 * - handleLogoUpload(): augšupielādē komandas logotipu.
 * - reviewCoachRequest(): apstrādā trenera asistenta pieprasījumu.
 *
 * Nosaukums: frontend/src/views
 */
const playerScore = (player) => (
  (player.stats?.goals || 0) * 4 +
  (player.stats?.assists || 0) * 3 +
  (player.stats?.matches || 0)
);

const rankedPlayers = computed(() => [...players.value].sort((a, b) => {
  const scoreDiff = playerScore(b) - playerScore(a);
  if (scoreDiff !== 0) return scoreDiff;
  return fullName(a).localeCompare(fullName(b));
}));

const upcomingEvents = computed(() => {
  const now = Date.now();
  return sortEvents(schedule.value).filter((event) => {
    const eventStamp = new Date(`${event.event_date}T${event.event_time} ||
'00:00'`).getTime();
    return eventStamp >= now;
  });
});

const fetchDashboard = async () => {
  try {
```

```

    const [bundle, nextSummary, nextSchedule] = await Promise.all([
      fetchTeamBundle(teamId),
      fetchTeamSummary(teamId),
      fetchTeamSchedule(teamId)
    ]);
    team.value = bundle.team;
    players.value = bundle.players;
    summary.value = nextSummary;
    schedule.value = nextSchedule;
    await loadCoachRequests();
  } catch (error) {
    showToast(copy.value.dashboardLoadError, 'error');
  }
};

/**
 * Augšupielādē komandas logotipu un apstrādā trenera pieprasījumu.
 */
const handleLogoUpload = async (event) => {
  const file = event.target.files?.[0];
  if (!file) return;
  if (file.size > 2 * 1024 * 1024) {
    showToast(copy.value.imageTooLarge, 'error');
    return;
  }

  try {
    team.value.logo = await uploadTeamLogo(teamId, file);
    showToast(copy.value.logoUpdated);
  } catch (error) {
    showToast(copy.value.logoUpdateError, 'error');
  } finally {
    event.target.value = '';
  }
};

const reviewCoachRequest = async (request, decision) => {
  if (!confirm(copy.value.reviewConfirm(decision,
    coachRequestName(request)))) return;

  if (decision === 'approve') {
    await approveCoachJoinRequest(teamId, request.id);
  }
};

```

```

    } else {
      await rejectCoachJoinRequest(teamId, request.id);
    }
    coachRequests.value = coachRequests.value.filter((entry) => entry.id !==
request.id);
  };
}

```

views/TeamPlayers.vue

```

/**
 * Šī lapa nodrošina komandas spēlētāju saraksta un statistikas pārvaldību.
 *
 * Galvenās funkcijas:
 * - fetchTeamData(): ielādē spēlētājus.
 * - sortedPlayers(), filteredPlayers(): kārtot un filtrēt spēlētājus.
 * - adjustStat(): maina statistikas vērtību.
 * - updatePlayerStats(): saglabā statistiku serverī.
 * - confirmRemovePlayer(): noņem spēlētāju no komandas.
 *
 * Nosaukums: frontend/src/views
 */
const sortedPlayers = computed(() => {
  const roster = [...players.value];
  switch (sortBy.value) {
    case 'goals':
      return roster.sort((a, b) => b.stats.goals - a.stats.goals);
    case 'assists':
      return roster.sort((a, b) => b.stats.assists - a.stats.assists);
    case 'matches':
      return roster.sort((a, b) => b.stats.matches - a.stats.matches);
    default:
      return roster.sort((a, b) => fullName(a).localeCompare(fullName(b)));
  }
});

const filteredPlayers = computed(() => {
  if (!searchQuery.value) return sortedPlayers.value;
  const query = searchQuery.value.toLowerCase();
  return sortedPlayers.value.filter((player) =>

```

```

        fullName(player).toLowerCase().includes(query) ||
        (player.email || '').toLowerCase().includes(query)
    );
});

const fetchTeamData = async () => {
    loading.value = true;
    try {
        const response = await fetchTeamBundle(teamId);
        team.value = response.team;
        players.value = response.players;
    } finally {
        loading.value = false;
    }
};

/**
 * Maina un saglabā spēlētāja statistiku.
 */
const adjustStat = (player, key, delta) => {
    const nextValue = Math.max(0, Number(player.stats[key] || 0) + delta);
    player.stats[key] = nextValue;
    updatePlayerStats(player);
};

const updatePlayerStats = async (player) => {
    try {
        await axios.put(`/api/auth/players/${player.id}/stats`, player.stats);
        showToast(t('messages.statsUpdated'), 'success');
    } catch (error) {
        showToast(t('teamPage.errorUpdatingStats'), 'error');
        fetchTeamData();
    }
};

const confirmRemovePlayer = async (player) => {
    if (!isMainCoach.value) return;
    if (!confirm(t('teamPage.removePlayerConfirm'))) return;

    await axios.delete(`/api/auth/players/${player.id}/team`);
    showToast(t('teamPage.playerRemoved'), 'success');
    fetchTeamData();
};

```



```
};
```

```
---
```

views/TeamSchedule.vue

```
/**
 * Šī lapa nodrošina komandas grafiku un apmeklējuma uzskaiti.
 *
 * Galvenās funkcijas:
 * - fetchData(): ielādē grafiku un spēlētājus.
 * - hasTimeConflict(): pārbauda laika konfliktu.
 * - addEvent(), saveEdit(), deleteEvent(): pārvalda notikumus.
 * - fetchEventAttendance(): ielādē apmeklējumu.
 * - setMyAttendance(): saglabā spēlētāja atbildi.
 *
 * Nosaukums: frontend/src/views
 */
const compareScheduleEvents = (a, b) =>
  new Date(`${a.event_date}T${a.event_time} || '23:59'`) -
  new Date(`${b.event_date}T${b.event_time} || '23:59'`);

const fetchData = async () => {
  loading.value = true;
  try {
    const [scheduleResponse, playersResponse] = await Promise.all([
      axios.get(`/api/teams/${teamId}/schedule`),
      axios.get(`/api/teams/${teamId}/players`)
    ]);
    events.value = Array.isArray(scheduleResponse.data)
      ? scheduleResponse.data.sort(compareScheduleEvents)
      : [];
    players.value = playersResponse.data?.players || [];
  } finally {
    loading.value = false;
  }
};

const hasTimeConflict = (date, time, excludeId = null) => {
  return events.value.some((event) =>
    event.event_date === date &&
```

```

        event.event_time === time &&
        String(event.id) !== String(excludeId)
    );
};

/**
 * Pievieno, rediģē un dzēš grafika notikumus.
 */
const addEvent = async () => {
    if (hasTimeConflict(currentEvent.value.event_date,
        currentEvent.value.event_time)) {
        error.value = t('schedule.timeConflict');
        return;
    }

    const response = await axios.post(`/api/teams/${teamId}/schedule`,
        currentEvent.value);
    events.value.push(response.data);
    events.value.sort(compareScheduleEvents);
    closeModal();
};

const saveEdit = async () => {
    if (hasTimeConflict(currentEvent.value.event_date,
        currentEvent.value.event_time, currentEvent.value.id)) {
        error.value = t('schedule.timeConflict');
        return;
    }

    await axios.put(`/api/teams/${teamId}/schedule/${currentEvent.value.id}`,
        currentEvent.value);

    const index = events.value.findIndex((event) => event.id ===
        currentEvent.value.id);
    if (index !== -1) events.value[index] = { ...currentEvent.value };
    closeModal();
};

const deleteEvent = async (event) => {
    if (!confirm(t('schedule.deleteConfirm'))) return;
    await axios.delete(`/api/teams/${teamId}/schedule/${event.id}`);
    events.value = events.value.filter((item) => item.id !== event.id);
    selectedEvent.value = null;
};

/**
 * Ielādē un saglabā apmeklējumu konkrētām treniņam.

```

```

*/
const fetchEventAttendance = async (eventId) => {
  if (!eventId) return;
  const response = await
  axios.get(`/api/teams/${teamId}/events/${eventId}/attendance/full`);
  eventAttendance.value = Array.isArray(response.data) ? response.data :
  [];

  const currentUserAttendance = eventAttendance.value.find(
    (entry) => Number(entry.user_id) === Number(authStore.user?.id)
  );
  myAttendanceStatus.value = currentUserAttendance?.status || null;
  myAttendanceNotes.value = currentUserAttendance?.notes || '';
};

const setMyAttendance = async (status) => {
  if (!selectedEvent.value || !authStore.user?.id) return;
  await
  axios.post(`/api/teams/${teamId}/events/${selectedEvent.value.id}/attendance`, {
    user_id: authStore.user.id,
    status,
    notes: status === 'present' ? null : myAttendanceNotes.value
  });
  await fetchEventAttendance(selectedEvent.value.id);
};

---

```

views/TeamStatistics.vue

```

/**
 * Šī lapa nodrošina komandas statistikas analītiku un eksportu.
 *
 * Galvenās funkcijas:
 * - rankedPlayers(): kārtot spēlētājus pēc rezultātiem.
 * - contributionBreakdown(): aprēķina vārtu un piespēļu sadalījumu.
 * - fetchStatistics(): ielādē statistiku.
 * - exportStatisticsCsv(): eksportē CSV failu.
 *
 * Nosaukums: frontend/src/views
 */

```

```

const rankedPlayers = computed(() => [...players.value].sort((a, b) => {
  const goalsDiff = (b.stats?.goals || 0) - (a.stats?.goals || 0);
  if (goalsDiff !== 0) return goalsDiff;
  const assistsDiff = (b.stats?.assists || 0) - (a.stats?.assists || 0);
  if (assistsDiff !== 0) return assistsDiff;
  return fullName(a).localeCompare(fullName(b));
}));

const contributionBreakdown = computed(() => {
  const list = performanceLeaders.value.length
    ? performanceLeaders.value
    : [...rankedPlayers.value].slice(0, 5);

  const total = list.reduce((sum, player) =>
    sum + (player.stats?.goals || 0) + (player.stats?.assists || 0), 0);

  return list.map((player, index) => {
    const goals = Number(player.stats?.goals) || 0;
    const assists = Number(player.stats?.assists) || 0;
    const contributionTotal = goals + assists;
    return {
      player,
      rank: index + 1,
      goals,
      assists,
      total: contributionTotal,
      goalShare: contributionTotal ? (goals / contributionTotal) * 100 : 0,
      assistShare: contributionTotal ? (assists / contributionTotal) * 100
: 0,
      teamShare: total ? (contributionTotal / total) * 100 : 0
    };
  });
});

/**
 * Ielādē statistiku un eksportē to CSV formātā.
 */
const fetchStatistics = async () => {
  loading.value = true;
  try {
    const [bundle, stats] = await Promise.all([
      fetchTeamBundle(teamId),

```

```

        fetchTeamSummary(teamId)
    });
    team.value = bundle.team;
    players.value = bundle.players;
    summary.value = stats;
  } finally {
    loading.value = false;
  }
};

const escapeCsvValue = (value) => {
  const stringValue = String(value ?? '');
  if (/[";\n]/.test(stringValue)) {
    return `"${stringValue.replace(/"/g, '""')}"`;
  }
  return stringValue;
};

const exportStatisticsCsv = () => {
  const delimiter = ';';
  const headers = copy.value.exportHeaders;
  const lines = [
    `sep=${delimiter}`,
    headers.join(delimiter),
    ...exportRows.value.map((row) => [
      row.rank,
      row.player,
      row.email,
      row.matches,
      row.goals,
      row.assists,
      row.yellowCards,
      row.redCards
    ].map(escapeCsvValue).join(delimiter))
  ];
  downloadBlob(`\uFEFF${lines.join('\n')}`, 'text/csv;charset=utf-8;',
  createFilename('csv'));
};

---
```

components/ChatComponent.vue

```

/**
 * Šis komponents nodrošina komandas čata ziņojumu attēlošanu un sūtīšanu.
 *
 * Galvenās funkcijas:
 * - sendMessage(): nosūta tekstu vai pielikumu.
 * - getAttachment(): sagatavo pielikuma datus.
 * - replyToMessage(): sāk atbildi uz ziņojumu.
 * - deleteMessage(): dzēš ziņojumu.
 *
 * Nosaukums: frontend/src/components
 * Izmantotais store: useChatStore
 */
const sendMessage = async () => {
  const text = newMessage.value.trim();
  if (!text && !selectedFile.value) return;

  try {
    isUploading.value = true;
    uploadError.value = '';
    const attachment = selectedFile.value
      ? await chatStore.uploadAttachment(selectedFile.value)
      : null;

    chatStore.sendMessage(text, attachment, replyingTo.value);
    newMessage.value = '';
    cancelReply();
    removeSelectedFile();
  } catch (error) {
    uploadError.value = error.message || 'Neizdevās augšupielādēt failu';
  } finally {
    isUploading.value = false;
  }
};

const getAttachment = (message) => {
  const url = message.attachment_url || message.attachmentUrl;
  if (!url) return null;

  return {
    url,
    objectUrl: chatStore.getAttachmentObjectUrl(url),
  };
};

```

```

        name: message.attachment_name || message.attachmentName || 'Pielikums',
        type: message.attachment_type || message.attachmentType || '',
        size: message.attachment_size || message.attachmentSize || 0
    };
};

const replyToMessage = (message) => {
    replyingTo.value = {
        id: message.id,
        username: message.username || 'Message',
        preview: getMessagePreview(message)
    };
    activeMessageMenuId.value = null;
};

const deleteMessage = (message) => {
    activeMessageMenuId.value = null;
    if (!canDeleteMessage(message)) return;
    if (replyingTo.value?.id === message.id) cancelReply();
    chatStore.deleteMessage(message.id);
};

```